

UMinho

Better Call Solve

André Pinheiro, Ângelo Fernandes, Gonçalo Faria

MIUP Team Reference

Contents

1 Contest 1
2 Mathematics 1
3 Data structures 3
4 Numerical 6
5 Number theory 9
6 Combinatorial 11
7 Graph 12
8 Geometry 17
9 Strings 20
10 Sequences and DP 22

Contest (1)

template.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;
template<class T> using vec = vector<T>;
#define rep(i,a,b) for (int i = (a); i < (b); ++i)
#define pb push_back
#define all(x) (x).begin(), (x).end()
constexpr int INF = 1000000000;
constexpr ll INFLL = LLONG_MAX/4;
#define sz(x) (int)(x).size()

#ifdef LOCAL
template<class T, class = void>
struct dbg_pair : false_type {};
template<class T> struct dbg_pair<T, void_t<
    decltype(declval<const T&>().first),
    decltype(declval<const T&>().second)>> : true_type {};
template<class T, class = void>
struct dbg_range : false_type {};
template<class T> struct dbg_range<T, void_t<
    decltype(begin(declval<const T&>())),
    decltype(end(declval<const T&>()))>> : true_type {};
template<class T> void dbg_print(const T& x) {
    if constexpr (
        is_convertible_v<const T&, string_view>)
        cerr << '"' << x << '"';
    else if constexpr (dbg_pair<T>::value) {
        cerr << '('; dbg_print(x.first);
        cerr << ','; dbg_print(x.second); cerr << ')';
    } else if constexpr (dbg_range<T>::value) {
        cerr << '[';
        const char* sep = "";
        for (const auto& v : x) {
            cerr << sep; dbg_print(v); sep = ",";
        }
        cerr << ']';
    } else cerr << x;
}
#define DEBUG(x) do { \
    cerr << #x << " = "; \
    dbg_print(x); cerr << '\n'; \
} while (0)
#else
#define DEBUG(x) ((void)0)
#endif

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

}
```

config.sh

```sh
# Adiciona a ~/.bashrc; depois: source ~/.bashrc
compilecp() {
  local f="${1%.cpp}"
  g++ -std=gnu++17 -O2 \
    -Wall -Wextra -Wshadow \
    "$f.cpp" -o "$f"
}

debugcp() {
  local f="${1%.cpp}"
  g++ -std=gnu++17 -O0 -g \
    -Wall -Wextra -Wshadow \
    -DLOCAL -D_GLIBCXX_DEBUG \
    -fsanitize=address,undefined \
    -fno-omit-frame-pointer \
    "$f.cpp" -o "$f.dbg"
}
```

troubleshoot.txt

```
Antes de submeter:
Testa exemplos, casos limite e input máximo.
Confirma tempo/memória e o ficheiro a submeter.

Wrong answer:
Relê o enunciado; verifica índices e output.
Overflow, valores por iniciar, dados por limpar?
Segue um caso pequeno; compara com força bruta.
Pede a um colega que reveja o raciocínio.

Runtime error:
Índices inválidos, iterators, divisão por zero?
Stack overflow ou algum assert falhou?
Corre sanitizers e os testes de STL debug.

Time limit exceeded:
Má complexidade, ciclos infinitos, cópias inúteis?
Demasiado I/O ou output de debug?

Memory limit exceeded:
Alocações, cópias ou containers demasiado grandes?
Limpa dados entre casos; verifica a memória total.
```

1.1 Conventions / using a header

- Copy the Contest template, the selected header and the dependencies listed above its code. All snippets use GNU C++17; no `vi` alias. `vec<T>` means `vector<T>`; `ll` means `long long`. `rep(i,a,b)` iterates over $[a, b)$; `pb`, `all` and `sz` abbreviate push_back, the iterator pair and an int size.
- Vertices/indices start at 0. Array ranges are $[l, r)$, prefix arrays have $n + 1$ entries, and jump tables use `jmp[k][v]`. Closed geometric segments and value-domain bounds are labelled explicitly.
- Tree roots have `parent[root]=-1`. Shortest-path parents are also -1 at a source and at unreachable vertices: check the distance or the documented reachability result before reconstructing a path. Sentinelas de pais não são distâncias.
- Each header defines its own result contract: `INF` for weighted unreachable distances, -1 for unweighted BFS distances, and `-INF` for Bellman–Ford vertices affected by a reachable negative cycle. Use the exact sentinel returned by that implementation.
- `auto [x,y]` unpacks a pair; `if(ans)` checks an optional, `*ans` reads it, `nullopt` means no result. Template types are usually inferred; `Fenwick<ll>` explicitly chooses the sum type.
- `[&](int x){...}` is a local callback that can access outer variables. Internal helpers are implementation details unless the header documents them as part of its interface.
- Keep asserts in local testing. Widen *before* arithmetic: `1LL*a*b`, `(__int128)a*b`. An assignment to ll does not repair an earlier int overflow.

1.2 STL quick reference

- `sort(all(a))`: $O(n \log n)$, pairs compare lexicographically. A comparator must be strict: use <, never <=. `stable_sort` preserves equal-key order.
- Sorted vector: `lower_bound` finds the first value $\geq x$; `upper_bound` the first $> x$. Both $O(\log n)$; their iterator difference counts equal values. Do not dereference end().
- `priority_queue<T>` is a max-heap. For a min-heap use `priority_queue<T,vec<T>,greater<T>>`. Push/pop $O(\log n)$, top $O(1)$; copy top before pop. Empty top/pop is invalid.
- `map`/`set`: ordered unique keys, $O(\log n)$ operations. `map[key]` inserts a default value; find() does not insert. Use the container's lower_bound, not the generic iterator algorithm.
- `multiset` allows duplicates. erase(value) removes all copies; erase(iterator) removes one. For a map, `it->second` is the value.
- `unordered_map`/`unordered_set`: expected $O(1)$, worst $O(n)$ per operation. reserve() reduces reallocations; never rely on iteration order. Rehash invalidates iterators.
- `a.erase(unique(all(a)),a.end())` removes adjacent duplicates; sort first to deduplicate arbitrary values. Vector insert/erase in the middle costs $O(n)$; reallocation invalidates references/iterators.
- `next_permutation(all(a))` visits lexicographic permutations; start sorted, include the first arrangement with do/while.

Mathematics (2)

2.1 Quadratic equations

- $ax^2 + bx + c = 0$, $a \neq 0$, $D = b^2 - 4ac$:

$$x_{1,2} = \frac{-b \pm \sqrt{D}}{2a}, \qquad x_1 + x_2 = -b/a, \quad x_1 x_2 = c/a.$$

- $D > 0, = 0, < 0$: two, one, no real roots. Vertex $(-b/(2a), -D/(4a))$: minimum if $a > 0$, maximum if $a < 0$.
- $a = 0$: linear. $a = b = 0$: all x if $c = 0$, otherwise none.
- Widen intermediates; verify integer bounds exactly.

2.2 Integer division and alignment

IntDivision.h – Floor/ceiling integer division, including negative operands.

floorDiv(a,b) rounds down, ceilDiv(a,b) rounds up; C++ / rounds toward zero. Operands use the same integer type, b!=0; quotient must fit (exclude MIN/-1). O(1) time/memory. Useful for integer inequalities and number of required batches.

Usage: `assert(ceilDiv(17LL,5LL) == 4);`
`assert(floorDiv(-7LL,3LL) == -3);`

```cpp
template<class T> T floorDiv(T a, T b) {
  T q = a / b, r = a % b;
  if (r != 0 && ((r < 0) != (b < 0))) --q;
  return q;
}

template<class T> T ceilDiv(T a, T b) {
  T q = a / b, r = a % b;
  if (r != 0 && ((r < 0) == (b < 0))) ++q;
  return q;
}
```

ModAlign.h – Next multiple or periodic event, including the present time.
nextMultiple(x,k) returns the smallest multiple of k >=x. nextTerm(x,t0,k) returns the first t0+j*k >=x with j>=0. Integer operands, k>0; result must fit. O(1) time/memory.
Usage: `assert(nextTerm(17LL,3LL,5LL) == 18);`

```
template<class T> T nextMultiple(T x, T k) {
  T r = x % k;
  if (r > 0) return x + (k - r);
  return x - r;
}

template<class T> T nextTerm(T x, T t0, T k) {
  if (x <= t0) return t0;
  T a = x % k, b = t0 % k;
  if (a < 0) a += k;
  if (b < 0) b += k;
  if (a <= b) return x + (b - a);
  return x + (k - (a - b));
}
```

2.3 Sums and progressions

- $n \geq 0$; empty sum $= 0$. Arithmetic: $a_i = a_0 + id$.

$$\sum_{i=0}^{n-1} a_i = na_0 + \frac{dn(n-1)}{2}.$$

- Geometric: $a_i = a_0 r^i$.

$$\sum_{i=0}^{n-1} a_i = \begin{cases} a_0(1-r^n)/(1-r), & r \neq 1, \\ na_0, & r = 1. \end{cases}$$

  Infinite sum $a_0/(1-r)$ requires $|r| < 1$.
- Power sums:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2},$$

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6},$$

$$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2,$$

$$\sum_{i=1}^{n} i^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}.$$

- For integers $0 \leq a \leq b$ and $c \neq 1$:

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c-1}.$$

  For $c = 1$, the sum is $b - a + 1$.
- $S(n) = \sum_{i=0}^{n-1} f(i)$: sum on $[l, r)$ is $S(r) - S(l)$.
- Cancel exact factors or widen before multiplying. Division modulo m requires an invertible denominator.

2.4 Series

- For all real x:

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots.$$

- $-1 < x \leq 1$:

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \cdots.$$

- $-1 \leq x \leq 1$:

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{x^3}{16} - \frac{5x^4}{128} + \cdots.$$

- For all real x (radians):

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots,$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots.$$

2.5 Exact arithmetic and overflow

- Signed b-bit range: $[-2^{b-1}, 2^{b-1} - 1]$; $b = 64, 128$.
- Promote before arithmetic: `i128(a)*b`. Check intermediate bounds, including negating the signed minimum.

Int128.h – Read/print signed 128-bit integers exactly.
toString(x) returns decimal text. read128(in,x) reads a signed integer; false for malformed input, overflow or EOF, leaving x unchanged. Cast BEFORE arithmetic: i128(a)*b, not i128(a*b). O(d) time/memory for d digits.
Usage: `i128 x = i128(1000000000000LL)*1000000000000LL;`
`cout << toString(x);`

```
using i128 = __int128_t;
using u128 = __uint128_t;

string toString(i128 x) {
  u128 v = x < 0 ? -u128(x) : u128(x);
  string s;
  do {
    s += char('0' + v % 10);
    v /= 10;
  } while (v);
  if (x < 0) s += '-';
  reverse(s.begin(), s.end());
  return s;
}

bool read128(istream& in, i128& x) {
  string s;
  if (!(in >> s)) return false;
  bool neg = s[0] == '-';
  int i = neg || s[0] == '+';
  if (i == sz(s)) return false;
  u128 v = 0, lim = (u128(1) << 127) - !neg;
  for (; i < sz(s); ++i) {
    int d = s[i] - '0';
    if (d < 0 || d > 9) return false;
    if (v > (lim - d) / 10) return false;
    v = v * 10 + d;
  }
  x = neg && v ? -i128(v - 1) - 1 : i128(v);
  return true;
}
```

- cpp_int: arbitrary-precision integers and stream I/O; cost depends on bit length. Range-check before fixed-width conversion.

```
#include <boost/multiprecision/cpp_int.hpp>
using boost::multiprecision::cpp_int;
```

2.6 Probability

- $P(A \mid B) = P(A \cap B)/P(B)$, requiring $P(B) > 0$.
- Independence: $P(A \cap B) = P(A)P(B)$. Pairwise independence need not imply mutual independence.
- Partition (B_i), omitting $P(B_i) = 0$:

$$P(A) = \sum_i P(A \mid B_i)P(B_i),$$

$$P(B_j \mid A) = \frac{P(A \mid B_j)P(B_j)}{P(A)} \quad (P(A) > 0).$$

2.7 Variance and common distributions

- $\mathrm{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2$. For independent variables, $\mathrm{Var}(\sum a_i X_i) = \sum a_i^2 \mathrm{Var}(X_i)$.
- Hypergeometric: sample n without replacement from N objects, K marked; $0 \leq n, K \leq N$: $P(X = k) = \binom{K}{k}\binom{N-K}{n-k}/\binom{N}{n}$; impossible choices give zero.
- Poisson, rate $\lambda \geq 0$: $P(X = k) = e^{-\lambda}\lambda^k/k!$, $k \geq 0$; mean and variance λ.
- Uniform on $[a, b]$: mean $(a+b)/2$, variance $(b-a)^2/12$. Exponential rate $\lambda > 0$: $P(T > t) = e^{-\lambda t}$, $t \geq 0$; mean $1/\lambda$, variance $1/\lambda^2$.

2.8 Expectation

- Discrete X: $\mathbb{E}[X] = \sum_x xP(X = x)$. For finite expectations, independence is unnecessary:

$$\mathbb{E}\left[c + \sum_i a_i X_i\right] = c + \sum_i a_i \mathbb{E}[X_i].$$

- Indicator I_A: $\mathbb{E}[I_A] = P(A)$; expected count $= \sum_i P(A_i)$.
- Partition (B_i): $\mathbb{E}[X] = \sum_i P(B_i)\mathbb{E}[X \mid B_i]$; omit zero-probability events.

2.9 Binomial and geometric distributions

- Binomial: successes in $n \geq 0$ independent trials, fixed $0 \leq p \leq 1$.

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}, \quad 0 \leq k \leq n.$$

  $\mathbb{E}[X] = np$; $p = 0, 1$ are deterministic. Sampling without replacement is generally not binomial.
- Geometric: independent trials, fixed $0 < p \leq 1$; T includes the first success.

$$P(T = k) = (1-p)^{k-1}p, \quad k \geq 1; \qquad \mathbb{E}[T] = 1/p.$$

  Failures $T - 1$ have mean $(1-p)/p$. $p = 1$: $T = 1$; $p = 0$: infinite waiting time.

Probability.h – Probabilities for independent repeated trials.
binomialProb(n,p) returns n+1 probabilities: entry k is exactly k successes. Sum entries for an interval. $O(n^2)$ time, O(n) memory; n>=0 must be moderate. atLeastOne(n,p) returns $1 - (1-p)^n$ stably; geometricProb(k,p) returns probability of first success at k>=1. Both O(1). Fixed p in [0,1], independent trials. Long double is approximate; extremely small probabilities may underflow. p=0/1 and n=0 are handled.
Usage: `auto prob = binomialProb(4,0.5L); // prob[2]=0.375`
`assert(abs(atLeastOne(4,0.5L)-0.9375L) < 1e-15L);`

```
vec<long double> binomialProb(int n, long double p) {
  assert(n >= 0 && 0 <= p && p <= 1);
  vec<long double> prob((size_t)n + 1);
  prob[0] = 1;
  for (int i = 0; i < n; ++i) {
    // Descer: prob[k-1] ainda tem de descrever os i tiros
    anteriores.
```

```
    for (int k = i+1; k >= 1; --k)
      prob[k] = prob[k]*(1-p) + prob[k-1]*p;
    prob[0] *= 1-p;
  }
  return prob;
}

long double atLeastOne(long long n, long double p) {
  assert(n >= 0 && 0 <= p && p <= 1);
  if (n == 0 || p == 0) return 0;
  if (p == 1) return 1;
  // Forma estável de 1-(1-p)^n; condições evitam log(0).
  return -expm1l(n*log1pl(-p));
}

long double geometricProb(long long k, long double p) {
  assert(k >= 1 && 0 <= p && p <= 1);
  if (k == 1) return p;
  if (p == 0 || p == 1) return 0;
  return p*expl((k-1)*log1pl(-p));
}
```

2.10 Random processes on states

- State contains all relevant history. $p_{st} = P(\text{next} = t \mid \text{current} = s)$; $p_{st} \geq 0$, $\sum_t p_{st} = 1$.
- Hitting target before failure: $h = 1$ at targets; $h = 0$ at failures and states unable to reach a target; otherwise $h_s = \sum_t p_{st} h_t$.
- Expected steps: $E = 0$ at terminals; $E_s = 1 + \sum_t p_{st} E_t$. For expected immediate cost c_s, replace 1 by c_s.
- Self-loop, $p_{ss} < 1$:

$$E_s = \frac{c_s + \sum_{t \neq s} p_{st} E_t}{1 - p_{ss}}.$$

  For hitting probability, omit c_s.
- DAG: reverse topological DP, $O(V + E)$. Cycles: real Gaussian elimination, $O(V^3)$.
- Finite chain: expected steps are finite iff every reachable state can reach a terminal through positive-probability edges. A reachable closed nonterminal class implies infinite expected time.

2.11 Trigonometry and recurrences

- Angles in radians: $\sin(u+v) = \sin u \cos v + \cos u \sin v$; $\cos(u+v) = \cos u \cos v - \sin u \sin v$.
  $a \cos x + b \sin x = \sqrt{a^2 + b^2} \cos(x - \text{atan2}(b, a))$.
- For $s_n = \sum_{i=1}^{k} c_i s_{n-i}$ with $c_k \neq 0$: polynomial $x^k - \sum_i c_i x^{k-i}$. Distinct roots r_i give $s_n = \sum_i d_i r_i^n$; a root of multiplicity m contributes a polynomial of degree m-1 times r^n. Prefer `LinearRecurrence.h` for modular computation.

Data structures (3)

- Public indices are 0-based; every range is $[l, r)$. Prefix arrays have n+1 entries. Internal Fenwick/DSU bookkeeping may differ.
- T defaults to ll where applicable; arithmetic must fit the chosen type. Use sparse tables for static minima, Fenwick for additive queries, segment trees for general aggregates, lazy trees for range updates.

3.1 Arrays and disjoint sets

ArrayBasics.h – Static range sums and offline range additions.

- prefixSums(a) returns n+1 entries, p[0]=0; rangeSum(p,l,r) returns the sum on $[l, r)$. Negative values allowed; all sums must fit ll.
- Start a difference array d(n+1,0). addRange(d,l,r,x) adds x on $[l, r)$; finishDiff(d) returns the n final values without changing d. Add the initial array afterwards, if needed; no intermediate queries.
- Prefix construction/finalisation $O(n)$, query/add $O(1)$; space $O(n)$.

Usage: `auto p = prefixSums(vec<int>{4,-2,7});`
`ll x = rangeSum(p,1,3); // 5`

```
template<class T> vec<ll> prefixSums(const vec<T>& a) {
  vec<ll> p(sz(a)+1);
  rep(i,0,sz(a)) p[i+1] = p[i]+ll(a[i]);
  return p;
}
ll rangeSum(const vec<ll>& p, int l, int r) {
  assert(0 <= l && l <= r && r < sz(p));
  return p[r]-p[l];
}
void addRange(vec<ll>& d, int l, int r, ll x) {
  assert(0 <= l && l <= r && r < sz(d));
  if (l == r) return;
  d[l] += x; d[r] -= x;
}
vec<ll> finishDiff(const vec<ll>& d) {
  assert(!d.empty());
  vec<ll> a(sz(d)-1);
  ll value = 0;
  rep(i,0,sz(a)) a[i] = value += d[i];
  return a;
}
```

UnionFind.h – DSU: merge components and test connectivity.

- UF(n) starts with n separate vertices. join(a,b) returns true only when two components merge; find(v) returns its representative, sameSet(a,b) tests connectivity, size(v) gives the component size.
- e[root] is the negative size; other entries are DSU parents (internal convention, not tree parents). Union by size and path compression.
- Build/space $O(n)$; operations amortised $O(\alpha(n))$.

Usage: `UF uf(3); uf.join(0,1); bool yes = uf.sameSet(0,1);`

```
struct UF {
  vec<int> e;
  UF(int n) : e(n, -1) {}
  bool sameSet(int a, int b) { return find(a) == find(b); }
  int size(int x) { return -e[find(x)]; }
  int find(int x) {
    if (e[x] < 0) return x;
    return e[x] = find(e[x]); // comprime o caminho
  }
  bool join(int a, int b) {
    a = find(a); b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    e[a] += e[b]; e[b] = a;
    return true;
  }
};
```

UnionFindRollback.h – DSU snapshots: temporarily merge components, then undo.

- Use when merges must be undone. Start with n separate vertices; find(v) gives the root, size(v) the group size; join(a,b) returns true if merged.
- Save t=time(), then rollback(t) undoes later merges. t must be a snapshot of the current history. e[root]=-size; st saves changed entries for undo. No path compression; unchanged joins save nothing.
- Find/join $O(\log n)$, snapshot $O(1)$, undo k joins $O(k)$; build/space $O(n)$.

Usage: `RollbackUF uf(n); int t = uf.time();`
`uf.join(a,b); uf.rollback(t); // undo that merge`

```
struct RollbackUF {
  vec<int> e; vec<pair<int,int>> st;
  RollbackUF(int n) : e(n, -1) {}
  int size(int x) { return -e[find(x)]; }
  int find(int x) {
    while (e[x] >= 0) x = e[x];
    return x;
  }
  int time() { return sz(st); }
  void rollback(int t) {
    assert(0 <= t && t <= time());
    while (time() > t) {
      auto [i, old] = st.back(); st.pop_back();
      e[i] = old;
    }
  }
  bool join(int a, int b) {
    a = find(a); b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    st.pb({a, e[a]});
    st.pb({b, e[b]});
    e[a] += e[b]; e[b] = a;
    return true;
  }
};
```

DynamicConnectivity.h – Offline connectivity with edge additions and removals.

- Use for graph edges added AND removed, when all operations are known in advance. If there are only removals, reverse time and use ordinary UF.
- Construct (n,Q): vertices and time steps. add(l,r,u,v) makes edge u-v active at $l, \ldots, r-1$. Give one interval per active copy of repeated edges.
- run(answer) calls your answer(t,uf) at each time; save answers there using find/size, without changing uf. run returns nothing. s groups lifetimes; dfs temporarily merges edges, visits times, then undoes.
- I intervals: $O(n + Q + I \log Q \log n)$ plus answer calls; space $O(n + Q + I \log Q)$.

Usage: `dc.run([&](int t, RollbackUF& uf) {`
`ans[t] = uf.find(u[t]) == uf.find(v[t]);`
`}); // dc was built and intervals added above`
`"UnionFindRollback.h"`

```
struct DynamicConnectivity {
  int q, n = 1;
  RollbackUF uf;
  vec<vec<pair<int,int>>> s;
  DynamicConnectivity(int N, int Q)
    : q(Q), uf(N) {
    while (n < q) n *= 2;
    s.resize(2*n);
  }
  void add(int l, int r, int u, int v) {
    assert(0 <= l && l <= r && r <= q);
    for (l += n, r += n; l < r; l /= 2, r /= 2) {
      if (l & 1) s[l++].pb({u,v});
      if (r & 1) s[--r].pb({u,v});
    }
  }
  template<class F> void dfs(int k, F& f) {
    int t = uf.time();
    for (auto [u,v] : s[k]) uf.join(u,v);
    if (k >= n) {
      if (k-n < q) f(k-n,uf);
    } else {
      dfs(2*k,f); dfs(2*k+1,f);
    }
    uf.rollback(t);
  }
  template<class F> void run(F f) { dfs(1,f); }
};
```

3.2 Range queries

Fenwick.h – Point additions, range sums and prefix selection.

- Fenwick<T>(n) starts with n zeros (T defaults to ll). add(i,x) adds x; query(r) sums $[0, r)$; query(l,r) sums $[l, r)$. Indices are 0-based externally; t uses 1-based partial sums internally.
- lowerBound(s) returns the first i with query(i+1)>=s, or n if absent; returns -1 for s<=0. This search requires all array values nonnegative; ordinary sums allow negatives. All sums must fit T.
- Build/space $O(n)$; add/query/search $O(\log n)$.

Usage: `Fenwick f(3); f.add(1,7); int i=f.lowerBound(7); // 1`

```
template<class T = ll> struct Fenwick {
```

```
  int n;
  vec<T> t;
  Fenwick(int m = 0) : n(m), t(size_t(m)+1) {
    assert(n >= 0);
  }
  void add(int i, T delta) {
    assert(0 <= i && i < n);
    for (ll k = ll(i)+1; k <= n; k += k & -k)
      t[k] += delta;
  }
  T query(int r) const {
    assert(0 <= r && r <= n);
    T ans{};
    for (; r > 0; r -= r & -r) ans += t[r];
    return ans;
  }
  T query(int l, int r) const {
    assert(0 <= l && l <= r && r <= n);
    return query(r) - query(l);
  }
  int lowerBound(T sum) const {
    if (sum <= T(0)) return -1;
    int pos = 0, step = 1;
    while (step <= n/2) step *= 2;
    for (; step; step /= 2)
      if (step <= n-pos && t[pos+step] < sum) {
        pos += step;
        sum -= t[pos];
      }
    return pos;
  }
};
```

Fenwick2D.h – Sparse point additions and rectangle sums.

- Use for point additions and rectangle sums with large/sparse coordinates. Pass ALL possible update points first; values initially zero. Later, add(x,y,v) adds v only at a registered point.
- query(X,Y) sums points with x<X and y<Y; query(x1,y1,x2,y2) sums $[x_1, x_2) \times [y_1, y_2)$. Query boundaries need not be registered. xs/ys compress coordinates; pos finds an index, ft holds Fenwick trees. Both tree levels use 1-based internal indices.
- m registered points: build $O(m \log^2 m)$, space $O(m \log m)$, add/query $O(\log^2 m)$. T is the sum type, C the coordinate type; sums fit T.

Usage: `Fenwick2D<ll,ll> f(points); f.add(x,y,7); ll sum = f.query(x1,y1,x2,y2);`

"Fenwick.h"

```
template<class T = ll, class C = int>
struct Fenwick2D {
  vec<C> xs;
  vec<vec<C>> ys;
  vec<Fenwick<T>> ft;
  static int pos(const vec<C>& v, C x) {
    return int(lower_bound(v.begin(), v.end(), x)
      - v.begin());
  }
  Fenwick2D(vec<pair<C,C>> p = {}) {
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    for (auto [x,y] : p) xs.pb(x);
    xs.erase(unique(xs.begin(), xs.end()),
      xs.end());
    ys.resize(xs.size()+1);
    for (auto [x,y] : p)
      for (ll i = ll(pos(xs,x))+1; i <= sz(xs);
        i += i & -i)
        ys[i].pb(y);
    for (auto& v : ys) {
      sort(v.begin(), v.end());
      v.erase(unique(v.begin(), v.end()),
        v.end());
      ft.emplace_back(sz(v));
    }
  }
  void add(C x, C y, T delta) {
    int at = pos(xs,x);
    assert(at < sz(xs) && xs[at] == x);
    for (ll i = ll(at)+1; i <= sz(xs); i += i & -i) {
      int j = pos(ys[i],y);
      assert(j < sz(ys[i]) && ys[i][j] == y);
      ft[i].add(j, delta);
    }
  }
  T query(C x, C y) const {
    T ans{};
    for (int i = pos(xs,x); i; i &= i-1)
      ans += ft[i].query(pos(ys[i],y));
    return ans;
  }
  T query(C x1, C y1, C x2, C y2) const {
    assert(x1 <= x2 && y1 <= y2);
    return query(x2,y2) - query(x1,y2)
      - query(x2,y1) + query(x1,y1);
  }
};
```

SegmentTree.h – Point assignment and range aggregate (maximum by default).

- SegTree(n) starts with n neutral values. update(i,x) assigns a[i]=x; query(l,r) returns max on $[l, r)$, unit for an empty range. T is ll. To adapt, change T, merge and unit together: merge must be associative and unit its identity (sum/0, min/LLONG_MAX, etc.).
- Left/right accumulators preserve order for noncommutative operations. s stores partial results. Build/space $O(n)$; update/query $O(\log n)$. Arithmetic must fit T.

Usage: `SegTree st(3); st.update(1,7); ll x=st.query(0,2); // 7`

```
struct SegTree {
  using T = ll;
  static constexpr T unit = LLONG_MIN;
  static T merge(T a, T b) { return max(a,b); }
  vec<T> s; int n;
  SegTree(int m = 0) : s(2*m,unit), n(m) { assert(m >= 0); }
  void update(int i, T x) {
    assert(0 <= i && i < n);
    s[i += n] = x;
    for (i /= 2; i; i /= 2)
      s[i] = merge(s[2*i],s[2*i+1]);
  }
  T query(int l, int r) const {
    assert(0 <= l && l <= r && r <= n);
    T left = unit, right = unit;
    for (l += n, r += n; l < r; l /= 2, r /= 2) {
      if (l & 1) left = merge(left,s[l++]);
      if (r & 1) right = merge(s[--r],right);
    }
    return merge(left,right);
  }
};
```

LazySegTree.h – Range add/set, range sum and range maximum.

- Construct from vec<T> a (T defaults to ll). add(l,r,x) adds x to every value in $[l, r)$; set(l,r,x) overwrites that range. query(l,r) returns its sum; queryMax(l,r) its maximum. Empty ranges return 0 / numeric_limits<T>::lowest().
- sum/mx store aggregates. assigned marks a pending assignment; tag then stores its value, otherwise an addition. apply updates a node, push passes pending work to children, pull recomputes the parent. Use the short public methods, not the recursive overloads.
- Build/space $O(n)$; operations $O(\log n)$. T must be an ordered numeric type; all intermediate sums, products and CUMULATIVE lazy tags must fit T, even when the final individual values fit.

Usage: `LazySegTree st(vec<ll>{1,2,3}); st.set(0,2,5); st.add(1,3,2); ll total=st.query(0,3); // 17`

```
template<class T = ll> struct LazySegTree {
  int n;
  vec<T> sum, mx, tag;
  vec<bool> assigned;
  LazySegTree(const vec<T>& a) : n(sz(a)), sum(4*n),
    mx(4*n), tag(4*n), assigned(4*n) {
    if (n) build(1,0,n,a);
  }
  void pull(int p) {
    sum[p] = sum[2*p]+sum[2*p+1];
    mx[p] = max(mx[2*p],mx[2*p+1]);
  }
  void build(int p, int l, int r, const vec<T>& a) {
    if (r-l == 1) { sum[p] = mx[p] = a[l]; return; }
    int m = l+(r-l)/2;
    build(2*p,l,m,a); build(2*p+1,m,r,a); pull(p);
  }
  void apply(int p, int len, T x, bool replace) {
    if (replace) {
      sum[p] = x*len; mx[p] = tag[p] = x;
      assigned[p] = true;
    } else {
      sum[p] += x*len; mx[p] += x; tag[p] += x;
    }
  }
  void push(int p, int l, int r) {
    if (!assigned[p] && tag[p] == 0) return;
    int m = l+(r-l)/2;
    apply(2*p,m-l,tag[p],assigned[p]);
    apply(2*p+1,r-m,tag[p],assigned[p]);
    tag[p] = 0; assigned[p] = false;
  }
  void update(int l, int r, T x, bool replace,
      int p, int a, int b) {
    if (r <= a || b <= l || l == r) return;
    if (l <= a && b <= r) { apply(p,b-a,x,replace); return; }
    push(p,a,b);
    int m = a+(b-a)/2;
    update(l,r,x,replace,2*p,a,m);
    update(l,r,x,replace,2*p+1,m,b); pull(p);
  }
  T query(int l, int r, bool maximum, int p, int a, int b) {
    if (r <= a || b <= l || l == r)
      return maximum ? numeric_limits<T>::lowest() : T(0);
    if (l <= a && b <= r) return maximum ? mx[p] : sum[p];
    push(p,a,b); int m = a+(b-a)/2;
    T x = query(l,r,maximum,2*p,a,m);
    T y = query(l,r,maximum,2*p+1,m,b);
    return maximum ? max(x,y) : x+y;
  }
  void check(int l, int r) const {
    assert(0 <= l && l <= r && r <= n);
  }
  void add(int l, int r, T x) {
    check(l,r); update(l,r,x,false,1,0,n);
  }
  void set(int l, int r, T x) {
    check(l,r); update(l,r,x,true,1,0,n);
  }
  T query(int l, int r) {
    check(l,r); return query(l,r,false,1,0,n);
  }
  T queryMax(int l, int r) {
    check(l,r); return query(l,r,true,1,0,n);
  }
};
```

SparseLazySegTree.h – Range add/set and maximum on a huge integer domain.

- SparseLazySegTree(n) represents n zeros on $[0, n)$, $0 < n \leq LLONG_MAX$. add/set(l,r,x) add/assign x; query(l,r) returns maximum, LLONG_MIN for an empty interval. Useful when coordinates arrive online and cannot be compressed first.

- Nodes are created only when descending; child IDs index t. hasSet distinguishes an assignment from an addition (no reserved value). push creates children and sends pending tags; modify/query recurse. Public indices are ll because the domain can be large.
- Operations $O(\log n)$; after q operations $O(q \log n)$ space. Queries may allocate children. Stored values must fit ll; cumulative lazy tags use __int128, so cancelling large additions remain safe.

Usage: `SparseLazySegTree st(1000000000000LL); st.add(10,20,3); ll best=st.query(0,30); // 3`

```
struct SparseLazySegTree {
  struct Node {
    int left = -1, right = -1;
    ll value = 0;
    __int128 tag = 0;
    bool hasSet = false;
  };
  ll n;
  vec<Node> t = vec<Node>(1);
  SparseLazySegTree(ll N) : n(N) { assert(n > 0); }
  void apply(int p, __int128 x, bool replace) {
    __int128 value = replace ? x : (__int128)t[p].value+x;
    assert(LLONG_MIN <= value && value <= LLONG_MAX);
    t[p].value = ll(value);
    if (replace) { t[p].tag = x; t[p].hasSet = true; }
    else t[p].tag += x;
  }
  void push(int p) {
    if (t[p].left < 0) {
      int id = sz(t); t.resize(id+2);
      t[p].left = id; t[p].right = id+1;
    }
    apply(t[p].left,t[p].tag,t[p].hasSet);
    apply(t[p].right,t[p].tag,t[p].hasSet);
    t[p].tag = 0; t[p].hasSet = false;
  }
  void modify(ll l, ll r, ll x, bool replace, int p, ll a, ll
   b) {
    if (r <= a || b <= l || l == r) return;
    if (l <= a && b <= r) { apply(p,x,replace); return; }
    push(p); ll m = a+(b-a)/2;
    modify(l,r,x,replace,t[p].left,a,m);
    modify(l,r,x,replace,t[p].right,m,b);
    t[p].value = max(t[t[p].left].value,t[t[p].right].value);
  }
  ll query(ll l, ll r, int p, ll a, ll b) {
    if (r <= a || b <= l || l == r) return LLONG_MIN;
    if (l <= a && b <= r) return t[p].value;
    push(p); ll m = a+(b-a)/2;
    return max(query(l,r,t[p].left,a,m),query(l,r,t[p].right,
    m,b));
  }
  void check(ll l, ll r) const { assert(0 <= l && l <= r && r
   <= n); }
  void add(ll l, ll r, ll x) { check(l,r); modify(l,r,x,false
  ,0,0,n); }
  void set(ll l, ll r, ll x) { check(l,r); modify(l,r,x,true,
  0,0,n); }
  ll query(ll l, ll r) { check(l,r); return query(l,r,0,0,n);
   }
};
```

RMQ.h – Sparse table: minimum on a static range.

- Construct RMQ(a); query(l,r) returns the minimum on nonempty $[l, r)$. Store (value,index) pairs to retrieve the first minimum position. No updates. jmp[k][i] covers 2^k values starting at i.
- Change both min calls to max for maxima. Overlapping blocks require an idempotent operation: this query formula does not work for sums.
- Build/space $O(n \log n)$, query $O(1)$; $n < 2^{30}$.

Usage: `RMQ rmq(vec<ll>{5,1,3}); ll x=rmq.query(0,2); // 1`

```
template<class T = ll> struct RMQ {
  vec<vec<T>> jmp;
  RMQ(const vec<T>& a) : jmp(1,a) {
    for (int k = 1; (1 << k) <= sz(a); ++k) {
      jmp.emplace_back(sz(a) - (1 << k) + 1);
      rep(i,0,sz(jmp[k]))
        jmp[k][i] = min(jmp[k-1][i],
          jmp[k-1][i + (1 << (k-1))]);
    }
  }
  T query(int l, int r) const {
    assert(0 <= l && l < r && r <= sz(jmp[0]));
    int k = __lg(r-l);
    return min(jmp[k][l], jmp[k][r - (1 << k)]);
  }
};
```

SqrtDecomposition.h – Point assignment and range counts of values <=x.

- Use for changes a[i]=x and queries asking how many values are <=x in a range. Construct from a; set(i,x) assigns; countLE(l,r,x) returns the count in $[l, r)$. a holds values, b holds sorted blocks. New values are allowed; T only needs ordering.
- $B \approx \sqrt{n}$: build $O(n \log B)$, set $O(B)$, query $O(B + (n/B) \log B)$; space $O(n)$.

Usage: `vec<ll> a{3,1,3}; SqrtDecomposition d(a); d.set(2,0); int k = d.countLE(0,3,2); // 2`

```
template<class T = ll> struct SqrtDecomposition {
  int n, len;
  vec<T> a;
  vec<vec<T>> b;
  SqrtDecomposition(const vec<T>& v = {})
    : n(sz(v)), len(max(1, int(sqrt(n)))), a(v),
      b((n+len-1)/len) {
    rep(i,0,n)
      b[i/len].pb(a[i]);
    for (auto& v1 : b)
      sort(v1.begin(), v1.end());
  }
  void set(int i, T x) {
    assert(0 <= i && i < n);
    auto& v = b[i/len];
    v.erase(lower_bound(v.begin(), v.end(),
      a[i]));
    v.insert(lower_bound(v.begin(), v.end(),
      x), x);
    a[i] = x;
  }
  int countLE(int l, int r, const T& x) const {
    assert(0 <= l && l <= r && r <= n);
    int ans = 0;
    while (l < r) {
      if (l % len == 0 && r-l >= len) {
        const auto& v = b[l/len];
        auto it = upper_bound(v.begin(),
          v.end(), x);
        ans += int(it-v.begin());
        l += len;
      } else {
        ans += !(x < a[l]);
        ++l;
      }
    }
    return ans;
  }
};
```

PrefixSum2D.h – Static rectangle sums.

- PrefixSum2D(rows,cols,a) receives a rectangular matrix with those dimensions. query(r1,c1,r2,c2) sums rows $[r_1, r_2)$ and columns $[c_1, c_2)$; an empty rectangle gives zero. p has an extra zero row/column.
- Build/space $O((n+1)(m+1))$, query $O(1)$; intermediate sums fit T.

Usage: `PrefixSum2D p(2,2,vec<vec<ll>>{{1,2},{3,4}}); ll x=p.query(0,1,2,2); // 6`

```
template<class T = ll> struct PrefixSum2D {
  int n, m;
  vec<vec<T>> p;
  PrefixSum2D(int rows, int cols,
    const vec<vec<T>>& a)
    : n(rows), m(cols), p(n+1, vec<T>(m+1)) {
    assert(n >= 0 && m >= 0 && sz(a) == n);
    rep(r,0,n) {
      assert(sz(a[r]) == m);
      rep(c,0,m)
        p[r+1][c+1] = a[r][c] + p[r][c+1]
          + p[r+1][c] - p[r][c];
    }
  }
  T query(int r1, int c1, int r2, int c2) const {
    assert(0 <= r1 && r1 <= r2 && r2 <= n);
    assert(0 <= c1 && c1 <= c2 && c2 <= m);
    return p[r2][c2] - p[r1][c2]
      - p[r2][c1] + p[r1][c1];
  }
};
```

Difference2D.h – Offline rectangle additions.

- Difference2D<T>(n,m) starts with a zero matrix. add(r1,c1,r2,c2,x) adds x on $[r_1, r_2) \times [c_1, c_2)$. values() returns the final matrix, without consuming the structure.
- Use when queries are needed only after all updates. d stores corner changes; values computes their 2D prefix sums. Add $O(1)$; values/space $O(n(m+1))$. Intermediate values must fit T.

Usage: `Difference2D d(2,3); d.add(0,1,2,3,5); auto a=d.values();`

```
template<class T = ll> struct Difference2D {
  int n, m;
  vec<vec<T>> d;
  Difference2D(int rows, int cols)
    : n(rows), m(cols), d(n) {
    assert(n >= 0 && m >= 0);
    for (auto& row : d) row.resize(m);
  }
  void add(int r1, int c1, int r2, int c2,
      T delta) {
    assert(0 <= r1 && r1 <= r2 && r2 <= n);
    assert(0 <= c1 && c1 <= c2 && c2 <= m);
    if (r1 == r2 || c1 == c2) return;
    d[r1][c1] += delta;
    if (r2 < n) d[r2][c1] -= delta;
    if (c2 < m) d[r1][c2] -= delta;
    if (r2 < n && c2 < m) d[r2][c2] += delta;
  }
  vec<vec<T>> values() const {
    auto a = d;
    rep(r,0,n)
      rep(c,0,m) {
        if (r) a[r][c] += a[r-1][c];
        if (c) a[r][c] += a[r][c-1];
        if (r && c) a[r][c] -= a[r-1][c-1];
      }
    return a;
  }
};
```

3.3 Ordered and editable containers

OrderedSet.h – Ordered set with ranks and kth-smallest selection.

- Use when insert/erase must coexist with ranks or kth-smallest queries. OST<int> stores unique ints in increasing order; insert(x), erase(x) work like set. `order_of_key(x)` counts values <x.
- `find_by_order(k)` returns an iterator to the 0-based kth value; check against end() before reading *it. For duplicates, store pair<T,int> with a unique id. Custom comparator C must be strict.
- Operations $O(\log n)$, space $O(n)$.

Usage: `OST<int> s; s.insert(7); s.insert(3); auto it = s.find_by_order(0); // *it = 3`

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

template<class T, class C = less<T>>
using OST = __gnu_pbds::tree<T,
  __gnu_pbds::null_type, C,
  __gnu_pbds::rb_tree_tag,
  __gnu_pbds::tree_order_statistics_node_update>;
```

HashMap.h – Integer hash map with a mixed, randomly salted hash.

- HashMap<V> maps ll keys to values V; standard unordered_map interface (find, erase, count, operator[]). operator[] inserts a default value when absent; find does not. Negative keys are valid.
- reserve(expected) avoids repeated growth. Expected $O(1)$ operations, worst-case $O(n)$; salt reduces predictable collision attacks but gives no deterministic guarantee. Hashing preserves exact key equality (unlike polynomial string fingerprints).

Usage: `HashMap<int> freq; freq.reserve(1000); ++freq[-7];`

```
struct IntHash {
  static uint64_t mix(uint64_t x) {
    x += 0x9e3779b97f4a7c15ULL;
    x = (x^(x>>30))*0xbf58476d1ce4e5b9ULL;
    x = (x^(x>>27))*0x94d049bb133111ebULL;
    return x^(x>>31);
  }
  size_t operator()(ll x) const {
    static const uint64_t salt = chrono::steady_clock::now()
      .time_since_epoch().count();
    return mix(uint64_t(x)+salt);
  }
};
template<class V> using HashMap = unordered_map<ll,V,IntHash
>;
```

Treap.h – Implicit treap: editable sequence with split/merge.

- ImplicitTreap starts empty. insert(pos,x) inserts before pos; erase(l,r), reverse(l,r) and query(l,r) remove/reverse/sum $[l, r)$. move(l,r,k) removes that block and inserts it before index k of the REMAINING sequence. size() is current length; values() materialises it.
- Nodes are IDs in t (0=null). cnt and sum aggregate each subtree; rev is pending reversal. split(v,k) separates the first k values; merge(a,b) concatenates disjoint trees; push propagates reversal, pull updates aggregates. Normally use the public sequence methods.
- Expected $O(\log n)$ per operation; values $O(n)$. Recursion depth is expected $O(\log n)$, worst-case $O(n)$. Random priorities; sums fit ll. Erased nodes remain allocated: memory $O(I)$ for I total insertions; destroy the object to reclaim.

Usage: `ImplicitTreap t; t.insert(0,3); t.insert(1,7); t.reverse(0,2); auto a=t.values(); // {7,3}`

```
struct ImplicitTreap {
  struct Node {
    int l = 0, r = 0, cnt = 1;
    ll val, sum;
    uint64_t priority;
    bool rev = false;
    Node(ll x, uint64_t p) : val(x), sum(x), priority(p) {}
  };
  vec<Node> t{Node(0,0)};
  int root = 0;
  mt19937_64 rng;
  ImplicitTreap(uint64_t seed = chrono::steady_clock::now()
    .time_since_epoch().count()) : rng(seed) { t[0].cnt = 0;
    }
  int size() const { return t[root].cnt; }
  void pull(int v) {
    t[v].cnt = 1+t[t[v].l].cnt+t[t[v].r].cnt;
    t[v].sum = t[v].val+t[t[v].l].sum+t[t[v].r].sum;
  }
  void push(int v) {
    if (!v || !t[v].rev) return;
    swap(t[v].l,t[v].r);
    if (t[v].l) t[t[v].l].rev = !t[t[v].l].rev;
    if (t[v].r) t[t[v].r].rev = !t[t[v].r].rev;
    t[v].rev = false;
  }
  pair<int,int> split(int v, int k) {
    if (!v) return {0,0};
    push(v);
    if (t[t[v].l].cnt >= k) {
      auto [a,b] = split(t[v].l,k);
      t[v].l = b; pull(v); return {a,v};
    }
    auto [a,b] = split(t[v].r,k-t[t[v].l].cnt-1);
    t[v].r = a; pull(v); return {v,b};
  }
  int merge(int a, int b) {
    if (!a || !b) return a ? a : b;
    if (t[a].priority > t[b].priority) {
      push(a); t[a].r = merge(t[a].r,b); pull(a); return a;
    }
    push(b); t[b].l = merge(a,t[b].l); pull(b); return b;
  }
  void check(int l, int r) const { assert(0 <= l && l <= r &&
   r <= size()); }
  void insert(int pos, ll x) {
    check(pos,pos);
    auto [a,b] = split(root,pos);
    int v = sz(t); t.emplace_back(x,rng());
    root = merge(merge(a,v),b);
  }
  void erase(int l, int r) {
    check(l,r); auto [a,bc] = split(root,l);
    auto [b,c] = split(bc,r-l); (void)b; root = merge(a,c);
  }
  void reverse(int l, int r) {
    check(l,r); auto [a,bc] = split(root,l);
    auto [b,c] = split(bc,r-l);
    if (b) t[b].rev = !t[b].rev;
    root = merge(a,merge(b,c));
  }
  ll query(int l, int r) {
    check(l,r); auto [a,bc] = split(root,l);
    auto [b,c] = split(bc,r-l); ll ans = t[b].sum;
    root = merge(a,merge(b,c)); return ans;
  }
  void move(int l, int r, int k) {
    check(l,r); assert(0 <= k && k <= size()-(r-l));
    auto [a,bc] = split(root,l); auto [b,c] = split(bc,r-l);
    auto [x,y] = split(merge(a,c),k); root = merge(x,merge(b,
    y));
  }
  void collect(int v, vec<ll>& a) {
    if (!v) return;
    push(v); collect(t[v].l,a); a.pb(t[v].val); collect(t[v].
    r,a);
  }
  vec<ll> values() { vec<ll> a; collect(root,a); return a; }
};
```

Numerical (4)

4.1 Binary and unimodal search

BinarySearch.h – First/last feasible integer in a half-open search range.

firstTrue(l,r,check) requires false then true; lastTrue requires true then false. Only evaluates candidates in $[l, r)$. Returns optional answer; nullopt for no answer or empty range. The excluded endpoints act as conceptual sentinels, never evaluated. Midpoints use 128 bits, including searches spanning negative and positive ll values. $O(\log(r - l + 1))$ predicate calls, $O(1)$ memory.

Usage: `auto x = firstTrue(0,11,[](ll t) { return t/2+t/3 >= 5; }); assert(x && *x == 6);`

```
template<class F>
optional<ll> firstTrue(ll l, ll r, F check) {
  assert(l <= r);
  ll end = r;
  while (l < r) {
    ll m = ll((__int128)l + ((__int128)r-l)/2);
    if (check(m)) r = m;
    else l = m+1;
  }
  if (l == end) return nullopt;
  return l;
}
template<class F>
optional<ll> lastTrue(ll l, ll r, F check) {
  assert(l <= r);
  ll begin = l;
  while (l < r) {
    ll m = ll((__int128)l + ((__int128)r-l)/2);
    if (check(m)) l = m+1;
    else r = m;
  }
  if (l == begin) return nullopt;
  return l-1;
}
```

Bisection.h – Approximate the first feasible real value.

Finite l<r with f(l)=false, f(r)=true; f switches only from false to true. Returns the true-side boundary approximation, not f(x). Floating type T, k>=0 iterations: O(k) evaluations, O(1) memory. Stops if rounding prevents progress.

Usage: `double x = bisect(0.,2.,[](double x) { return x*x >= 2; });`

```
template<class T, class F>
T bisect(T l, T r, F f, int k = 100) {
  static_assert(is_floating_point_v<T>);
  assert(l < r && k >= 0);
  while (k--) {
    T m = l < 0 && r > 0 ? (l+r)/2
      : l+(r-l)/2;
    if (m == l || m == r) break;
    if (f(m)) r = m;
    else l = m;
  }
  return r;
}
```

TernarySearch.h – Approximate the minimum of a unimodal real function.

Pass finite l<=r, function f and positive iteration count. On [l,r], f must decrease then increase (either side may be empty). Returns an approximate minimizer x, not f(x). Flat minimum is allowed; other flat regions can invalidate the search. Negate f to maximize. O(iterations) evaluations, O(1) memory. Use binary search for monotone predicates.

Usage: `auto x = ternaryMin(0,10,[](long double x) { return (x-3)*(x-3); });`

```
template<class F>
long double ternaryMin(long double l, long double r,
    F f, int iterations = 100) {
  assert(isfinite(l) && isfinite(r) && l <= r && iterations
   > 0);
  rep(it,0,iterations) {
    long double a = l+(r-l)/3, b = r-(r-l)/3;
    if (f(a) <= f(b)) r = b;
    else l = a;
  }
  return l+(r-l)/2;
}
```

4.2 Modular arithmetic and matrices

ModPower.h – Modular product and fast exponentiation.

modMul(a,b,m) expects normalized residues $0 \le a, b < m$, m>0. modPow(b,e,m) accepts a signed negative base; e is nonnegative. Base and modulus have the same integer type, at most 64 bits. Returns a residue in $[0, m)$. Products use unsigned 128 bits. Product $O(1)$, power $O(\log(e+1))$, memory $O(1)$.

Usage: `assert(modPow(2LL,10,7LL) == 2);`

```
using ull = unsigned long long;

template<class T> T modMul(T a, T b, T m) {
  static_assert(is_integral_v<T> && sizeof(T) <= 8);
  assert(m > 0 && a >= 0 && a < m && b >= 0 && b < m);
  return T((__uint128_t)a * b % m);
}
template<class T> T modPow(T b, ull e, T m) {
  assert(m > 0);
  T ans = 1 % m;
  b %= m;
  if (b < 0) b += m;
  while (e > 0) {
    if (e & 1) ans = modMul(ans,b,m);
    b = modMul(b,b,m);
    e /= 2;
  }
  return ans;
}
```

Matrix.h – Multiply modular matrices or raise a transition matrix to a power.

Square matrices with entries in $[0, mod)$; matMul(a,b,mod) returns ab and matPow(a,e,mod) returns a^e. e>=0, integer mod>=1; e=0 gives identity. State as a column: next[i]=sum A[i][j]*state[j]. 128-bit products support integer types through 64 bits. For real arithmetic remove reductions and use long double. Product $O(n^3)$; power $O(n^3 \log(e+1))$; $O(n^2)$ memory.

Usage: `vec<vec<ll>> a{{1,1},{1,0}};`
`auto f = matPow(a,10,1000000007LL); // f[0][1]=55`

```
template<class T>
vec<vec<T>> matMul(const vec<vec<T>>& a,
    const vec<vec<T>>& b, T mod) {
  static_assert(is_integral_v<T> && sizeof(T) <= 8);
  assert(mod >= 1 && sz(a) == sz(b));
  int n = sz(a);
  for (int i = 0; i < n; ++i) {
    assert(sz(a[i]) == n && sz(b[i]) == n);
    for (int j = 0; j < n; ++j) {
      assert(a[i][j] >= 0 && a[i][j] < mod);
      assert(b[i][j] >= 0 && b[i][j] < mod);
    }
  }
  vec<vec<T>> c(n,vec<T>(n));
  for (int i = 0; i < n; ++i)
    for (int k = 0; k < n; ++k)
      for (int j = 0; j < n; ++j)
        c[i][j] = T((c[i][j] + (__uint128_t)
          a[i][k]*b[k][j]) % mod);
  return c;
}
template<class T>
vec<vec<T>> matPow(vec<vec<T>> a,
    ll e, T mod) {
  assert(e >= 0 && mod >= 1);
  int n = sz(a);
  for (const auto& row : a) {
    assert(sz(row) == n);
    for (T x : row) assert(x >= 0 && x < mod);
  }
  vec<vec<T>> r(n,vec<T>(n));
  for (int i = 0; i < n; ++i) r[i][i] = T(1)%mod;
  while (e) {
    if (e&1) r = matMul(r,a,mod);
    e /= 2;
    if (e) a = matMul(a,a,mod);
  }
  return r;
}
```

4.3 Linear systems

Each equation is a row, each variable a column. For $Ax = b$, pass `[A|b]`: m coefficients then the right-hand side. Rank -1 means impossible; rank m means unique; otherwise free variables exist. Choose real Gauss for measurements/expectations, modular Gauss for a prime field, and binary Gauss for XOR constraints.

Gauss.h – Solve real linear equations; return rank and one solution.

Pass augmented rows a[i]={coefficients...,rhs}, variable count m, output x and absolute tolerance eps. Modifies a. Returns -1 if inconsistent, m for a unique solution, otherwise rank with free variables zero. x has m entries. Partial pivoting reduces numerical error; finite inputs/intermediates required and scaling matters. $O(nm \min(n,m))$ time, $O(m)$ extra memory.

Usage: `vec<vec<double>> a{{2,1,5},{1,-1,1}}; vec<double> x;`
`int rank = gauss(a,2,x); // rank=2, x={2,1}`

```
template<class T>
int gauss(vec<vec<T>>& a, int m,
    vec<T>& x, T eps = T(1e-12L)) {
  static_assert(is_floating_point_v<T>);
  assert(m >= 0 && eps > 0);
  int n = sz(a), rank = 0;
  vec<int> col;
  x.assign(m, T{});
  for (auto& row : a) assert(sz(row) == m+1);
  for (int c = 0; c < m && rank < n; ++c) {
    int pivot = rank;
    for (int i = rank+1; i < n; ++i)
      if (abs(a[i][c]) > abs(a[pivot][c])) pivot = i;
    if (abs(a[pivot][c]) <= eps) continue;
    swap(a[rank], a[pivot]);
    for (int i = rank+1; i < n; ++i) {
      T factor = a[i][c] / a[rank][c];
      a[i][c] = 0;
      for (int j = c+1; j <= m; ++j)
        a[i][j] -= factor * a[rank][j];
    }
    col.pb(c);
    ++rank;
  }
  for (int i = rank; i < n; ++i)
    if (abs(a[i][m]) > eps) return -1;
  for (int i = rank-1; i >= 0; --i) {
    int c = col[i];
    T value = a[i][m];
    for (int j = c+1; j < m; ++j)
      value -= a[i][j] * x[j];
    x[c] = value / a[i][c];
  }
  return rank;
}
```

GaussMod.h – Solve linear equations modulo a prime.

Pass augmented rows a, number of variables m, output x, and PRIME p with $2 \le p \le$ INT_MAX. Copies and normalizes a, so negative coefficients are allowed. Returns -1 if inconsistent, m if unique, otherwise rank; x is one solution in $[0, p)$ with free variables zero. where[c] records each pivot row. $O(nm \min(n,m) + \min(n,m) \log p)$ time and $O(nm)$ copied memory. Not for composite moduli.

Usage: `vec<vec<int>> a{{2,1,5},{1,-1,1}}; vec<int> x;`
`int rank = gaussMod(a,2,x,7); // rank=2, x={2,1}`
`"ModPower.h"`

```
int gaussMod(vec<vec<int>> a, int m,
    vec<int>& x, int p) {
  assert(m >= 0 && p >= 2);
  int n = sz(a), rank = 0;
  vec<int> where(m,-1);
  x.assign(m, 0);
  for (auto& row : a) {
    assert(sz(row) == m+1);
    for (int& v : row) {
      v %= p;
      if (v < 0) v += p;
    }
  }
  for (int c = 0; c < m && rank < n; ++c) {
    int pivot = rank;
    while (pivot < n && !a[pivot][c]) ++pivot;
    if (pivot == n) continue;
    swap(a[rank], a[pivot]);
    where[c] = rank;
    ll inv = modPow(a[rank][c], p-2, p);
    for (int j = c; j <= m; ++j)
      a[rank][j] = int(a[rank][j] * inv % p);
    for (int i = 0; i < n; ++i) if (i != rank) {
      int factor = a[i][c];
      a[i][c] = 0;
      for (int j = c+1; j <= m; ++j)
        a[i][j] = int((a[i][j]
          - ll(factor)*a[rank][j]%p + p)%p);
    }
    ++rank;
  }
  for (int i = rank; i < n; ++i)
    if (a[i][m]) return -1;
  for (int c = 0; c < m; ++c)
    if (where[c] != -1) x[c] = a[where[c]][m];
  return rank;
}
```

GaussBinary.h – Solve XOR/parity equations over bits.

a[i][j] indicates whether equation i uses variable j; a[i][m] is its RHS bit. Choose bitset<K> with K>m. Pass a,m,x; modifies a and fills x with m bits. Returns -1 if impossible, m if unique, otherwise rank with free variables zero. XOR replaces subtraction. $O(nm\lceil K/w \rceil)$ time and $O(m + K/w)$ extra memory.

Usage: `vec<bitset<4>> a{bitset<4>(5),bitset<4>(6)};`
`vec<int> x;`
`int rank = gaussBinary(a,2,x); // x={1,1}`

```
template<size_t K>
int gaussBinary(vec<bitset<K>>& a, int m, vec<int>& x) {
  assert(0 <= m && size_t(m) < K);
  int n = sz(a), rank = 0;
  vec<int> col;
  x.assign(m,0);
  for (int c = 0; c < m && rank < n; ++c) {
    int pivot = rank;
    while (pivot < n && !a[pivot][c]) ++pivot;
    if (pivot == n) continue;
    swap(a[pivot],a[rank]);
    for (int i = 0; i < n; ++i)
      if (i != rank && a[i][c]) a[i] ^= a[rank];
    col.pb(c); ++rank;
  }
  for (int i = rank; i < n; ++i)
    if (a[i][m]) return -1;
  for (int i = 0; i < rank; ++i) x[col[i]] = a[i][m];
  return rank;
}
```

Determinant.h – Determinant of a real square matrix.

Copies a and returns det(a); det(empty)=1. Row swaps flip the sign. Partial pivoting; pivots with absolute value <=eps count as zero. Finite arithmetic required. $O(n^3)$ time, $O(n^2)$ copied memory.

Usage: `assert(abs(determinant({{2,1},{1,3}})-5) < 1e-9L);`

```
long double determinant(vec<vec<long double>> a,
      long double eps = 1e-12L) {
    int n = sz(a); long double ans = 1;
```

```
    assert(eps > 0);
    for (auto& row : a) assert(sz(row) == n);
    rep(i,0,n) {
        int p = i;
        rep(j,i+1,n) if (abs(a[j][i]) > abs(a[p][i])) p = j;
        if (abs(a[p][i]) <= eps) return 0;
        if (p != i) { swap(a[p],a[i]); ans = -ans; }
        ans *= a[i][i];
        rep(j,i+1,n) {
            long double f = a[j][i]/a[i][i];
            rep(k,i+1,n) a[j][k] -= f*a[i][k];
        }
    }
    return ans;
}
```

IntDeterminant.h – Determinant modulo any positive modulus, including composite.

Copies square matrix a, normalizes signed entries, returns det(a) in [0,mod). Euclidean row reductions avoid modular division; useful for Matrix–Tree counts even when mod is not prime. $O(n^3 \log mod)$ time upper bound, $O(n^2)$ copied memory.

Usage: `assert(detMod({{2,1},{1,3}},6) == 5);`

```
ll detMod(vec<vec<ll>> a, ll mod) {
    assert(mod > 0); int n = sz(a); ll ans = 1%mod;
    for (auto& row : a) {
        assert(sz(row) == n);
        for (ll& x : row) { x %= mod; if (x < 0) x += mod; }
    }
    rep(i,0,n) {
        rep(j,i+1,n) while (a[j][i]) {
            ll q = a[i][i]/a[j][i];
            rep(k,i,n) {
                a[i][k] = ll((a[i][k]-(__int128)q*a[j][k])%
                mod);
                if (a[i][k] < 0) a[i][k] += mod;
            }
            swap(a[i],a[j]); if (ans) ans = mod-ans;
        }
        ans = ll((__int128)ans*a[i][i]%mod);
    }
    return ans;
}
```

MatrixInverse.h – Invert a real square matrix, with explicit failure.

matrixInverse(a,eps) copies a and returns optional inverse; nullopt if a pivot is <=eps in absolute value. Uses partial pivoting. eps is absolute: scale inputs, and expect floating-point error. $O(n^3)$ time and $O(n^2)$ memory. Empty matrix has an empty inverse.

Usage: `auto inv = matrixInverse({{2,1},{1,3}}); // test inv before *inv`

```
optional<vec<vec<long double>>> matrixInverse(
        vec<vec<long double>> a, long double eps = 1e-12L) {
    int n = sz(a); assert(eps > 0);
    vec<vec<long double>> b(n,vec<long double>(n));
    rep(i,0,n) { assert(sz(a[i]) == n); b[i][i] = 1; }
    rep(i,0,n) {
        int p = i;
        rep(j,i+1,n) if (abs(a[j][i]) > abs(a[p][i])) p = j;
        if (abs(a[p][i]) <= eps) return nullopt;
        swap(a[p],a[i]); swap(b[p],b[i]);
        long double v = a[i][i];
        rep(k,0,n) { a[i][k] /= v; b[i][k] /= v; }
        rep(j,0,n) if (j != i) {
            long double f = a[j][i];
            rep(k,0,n) { a[j][k] -= f*a[i][k]; b[j][k] -= f*b
            [i][k]; }
        }
    }
    return b;
}
```

4.4 Polynomials and recurrences

PolyInterpolate.h – Recover polynomial coefficients from distinct sample points.

x,y have n entries, all x distinct. Returns a of length n such that $p(t) = \sum_i a_i t^i$ and p(x[i])=y[i]; empty input gives empty output. Uses divided differences then converts to ordinary coefficients. Floating point: clustered/large x or high degree may be ill-conditioned. $O(n^2)$ time, O(n) extra memory. Evaluate with Horner from highest degree.

Usage: `auto a = interpolate({0,1,2},{1,4,9}); // {1,2,1}`

```
vec<long double> interpolate(const vec<long double>& x,
        vec<long double> y) {
    int n = sz(x); assert(sz(y) == n);
    rep(k,0,n) rep(i,k+1,n) {
        assert(x[i] != x[k]);
        y[i] = (y[i]-y[k])/(x[i]-x[k]);
    }
    vec<long double> a(n), basis(n+1); basis[0] = 1;
    rep(k,0,n) {
        rep(i,0,k+1) a[i] += y[k]*basis[i];
        for (int i = k+1; i > 0; --i)
            basis[i] = basis[i-1]-x[k]*basis[i];
        basis[0] *= -x[k];
    }
    return a;
}
```

LinearRecurrence.h – Find a far-away term of a known linear recurrence.

s has the first d terms, c has d coefficients, d>0: s[t]=c[0]*s[t-1]+...+c[d-1]*s[t-d]. Pass k>=0 and mod>=1; signed values are normalized. Returns s[k] modulo mod (composite allowed). Reduces powers of x modulo the recurrence polynomial; combine multiplies those coefficient vectors. $O(d^2 \log(k+1))$ time, O(d) extra memory.

Usage: `assert(linearRec({0,1},{1,1},10,1000000007) == 55);`

```
ll linearRec(vec<ll> s, vec<ll> c, unsigned long long k, ll
mod) {
    int d = sz(c);
    assert(d > 0 && sz(s) == d && mod >= 1);
    for (auto* v : {&s,&c}) for (ll& x : *v) {
        x %= mod; if (x < 0) x += mod;
    }
    auto combine = [&](const vec<ll>& a, const vec<ll>& b) {
        vec<ll> r(2*d-1);
        rep(i,0,d) rep(j,0,d)
            r[i+j] = ll((r[i+j]+(__int128)a[i]*b[j])%mod);
        for (int i = 2*d-2; i >= d; --i) rep(j,0,d)
            r[i-1-j] = ll((r[i-1-j]+(__int128)r[i]*c[j])%mod)
            ;
        r.resize(d); return r;
    };
    vec<ll> a(d), x(d); a[0] = 1%mod;
    if (d == 1) x[0] = c[0]; else x[1] = 1%mod;
    while (k) {
        if (k&1) a = combine(a,x);
        k >>= 1; if (k) x = combine(x,x);
    }
    ll ans = 0;
    rep(i,0,d) ans = ll((ans+(__int128)a[i]*s[i])%mod);
    return ans;
}
```

4.5 Convolution

NTT.h – Exact convolution modulo 998244353.

- Use for polynomial products or counting pairs by sum. Set a[i],b[j] to coefficients/frequencies; conv(a,b) returns c with c[k]=sum of a[i]*b[k-i] modulo 998244353. Length is a.size()+b.size()-1; empty if either input is empty. Inputs are copied.
- Values must be in $[0, mod)$; output length at most 2^{23}. conv pads to a power of two and calls ntt; ntt(a,false) transforms a in place, true reverses it. Only call ntt directly on a positive power-of-two length up to 2^{23}. Root 3 is specific to this prime.
- Padded length N: $O(N \log N)$ time, $O(N)$ space; counts wrap modulo mod.

Usage: `auto c = NTT::conv(vec<int>{1,2},vec<int>{3,4});`
`// c = {3,10,8}`
`"ModPower.h"`

```
namespace NTT {
constexpr int mod = 998244353;

void ntt(vec<int>& a, bool inv = false) {
  int n = sz(a);
  assert(n > 0 && !(n & (n-1)) && n <= (1<<23));
  for (int i = 1, j = 0; i < n; ++i) {
    int bit = n >> 1;
    for (; j & bit; bit >>= 1) j ^= bit;
    j ^= bit;
    if (i < j) swap(a[i], a[j]);
  }
  int g = inv ? int(modPow(3, mod-2, mod)) : 3;
  for (int k = 1; k < n; k *= 2) {
    int z = int(modPow(g, (mod-1)/(2*k), mod));
    for (int i = 0; i < n; i += 2*k) {
      ll w = 1;
      for (int j = 0; j < k; ++j) {
        int u = a[i+j];
        int v = int(w*a[i+j+k] % mod);
        a[i+j] = u+v < mod ? u+v : u+v-mod;
        a[i+j+k] = u-v < 0 ? u-v+mod : u-v;
        w = w*z % mod;
      }
    }
  }
  if (inv) {
    ll r = modPow(n, mod-2, mod);
    for (int& x : a) x = int(x*r % mod);
  }
}

vec<int> conv(vec<int> a, vec<int> b) {
  if (a.empty() || b.empty()) return {};
  int s = sz(a)+sz(b)-1, n = 1;
  assert(s <= (1<<23));
  while (n < s) n *= 2;
  a.resize(n); b.resize(n);
  ntt(a); ntt(b);
  for (int i = 0; i < n; ++i)
    a[i] = int(ll(a[i])*b[i] % mod);
  ntt(a, true);
  a.resize(s);
  return a;
}
}
```

FastFourierTransform.h – Approximate convolution of real coefficient/frequency arrays.

FFT::conv(a,b)[k]=sum a[i]*b[k-i]; length |a|+|b|-1, empty if either input is empty. No modulus; accepts negative/real entries. Round only when the accumulated error is comfortably below 0.5; large coefficients/lengths need an independent error bound or exact NTT. fft changes a power-of-two complex array in place; inverse=true undoes it. Padded size N: O(N log N) time, O(N) memory.

Usage: `auto c = FFT::conv({1,2},{3,4}); // approximately {3,10,8}`

```
namespace FFT {
```

```
using C = complex<long double>;
void fft(vec<C>& a, bool inverse = false) {
    int n = sz(a); assert(n > 0 && !(n&(n-1)));
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n>>1;
        for (; j&bit; bit >>= 1) j ^= bit;
        j ^= bit; if (i < j) swap(a[i],a[j]);
    }
    for (int len = 2; len <= n; ) {
        C root = polar(1.0L, (inverse?-2:2)*acosl(-1.0L)/len)
        ;
        for (int i = 0; i < n; i += len) {
            C w = 1;
            rep(j,0,len/2) {
                C u = a[i+j], v = a[i+j+len/2]*w;
                a[i+j] = u+v; a[i+j+len/2] = u-v; w *= root;
            }
        }
        if (len == n) break;
        len *= 2;
    }
    if (inverse) for (C& x : a) x /= n;
}
vec<long double> conv(const vec<long double>& a,
    const vec<long double>& b) {
    if (a.empty() || b.empty()) return {};
    int s = sz(a)+sz(b)-1, n = 1;
    assert(s > 0 && s <= (1<<29));
    while (n < s) n *= 2;
    vec<C> x(n), y(n);
    rep(i,0,sz(a)) x[i] = a[i];
    rep(i,0,sz(b)) y[i] = b[i];
    fft(x); fft(y);
    rep(i,0,n) x[i] *= y[i];
    fft(x,true); vec<long double> c(s);
    rep(i,0,s) c[i] = x[i].real();
    return c;
}
}
```

Number theory (5)

5.1 GCD and Diophantine equations

- `gcd(a,b)`: nonnegative GCD; `gcd(0,0)=0`. Absolute values must fit the common integer type. Time $O(\log(\min(|a|,|b|)+1))$.
- For $a,b \geq 0$, LCM is zero if either is zero; otherwise $(a/\gcd(a,b))b$. `lcm(a,b)` requires a representable result. Before multiplying, test $a/\gcd(a,b) \leq \text{MAX}/b$.

ExtGCD.h – Bezout coefficients for gcd and integer equations.

- Find integer coefficients for ax+by=gcd(a,b). Use for modular inverses or integer equations (notes above).
- Pass signed a,b and variables x,y; returns g>=0 and writes x,y. Neither input may be its type's minimum. For a=b=0, g=0. x1,y1 solve the smaller recursive equation.
- $O(\log(\max(|a|,|b|)+1))$ time/recursion space.

Usage: `ll x,y; ll g = extGcd(30LL,18LL,x,y);`
`// g=6 and 30*x+18*y=6`

```
template<class T>
T extGcd(T a, T b, T& x, T& y) {
  if (!b) {
    x = a < 0 ? -1 : 1; y = 0;
    return a < 0 ? -a : a;
  }
  T x1, y1;
  T g = extGcd(b,T(a%b),x1,y1);
  x = y1;
  y = x1 - (a/b)*y1;
  return g;
}
```

- $ax+by=c$, $(a,b) \neq (0,0)$, $g=\gcd(a,b)$: solvable iff $g \mid c$. Scale Bézout coefficients by c/g. All solutions from a particular (x_0,y_0):

$$x = x_0 + (b/g)t, \qquad y = y_0 - (a/g)t, \quad t \in \mathbb{Z}.$$

- Bound integer t with `floorDiv/ceilDiv` (Mathematics), reversing inequalities for negative steps; zero steps impose only feasibility. Widen intermediate products.
- $a=b=0$: all pairs if $c=0$, otherwise no solution.

5.2 Modular arithmetic

- m>0: normalize signed residues with `a%=m; if(a<0) a+=m;`. Residues do not preserve the order of the original values.
- Product/power: `modMul/modPow` from Numerical's `ModPower.h`. Cast before multiplying; ordinary / is not modular division.
- Divide by b using its inverse, which exists iff $\gcd(b,m)=1$ for m>1. For PRIME p and $p \nmid b$, inverse b^{p-2} mod p. Reducing exponents modulo $\varphi(m)$ requires $\gcd(b,m)=1$.

ModInv.h – Modular division when an inverse exists.

modInv(a,m), signed a and m>1, returns optional x in $[0,m)$ with ax=1 modulo m. Works for composite moduli too; nullopt when gcd(a,m)!=1. Test the optional before dereferencing. Extended Euclid with 128-bit intermediates: $O(\log m)$ time, $O(1)$ extra memory.

Usage: `auto inv = modInv(3,7); // *inv=5; 4/3 mod 7 is 4*5 mod 7`
`assert(!modInv(2,6));`

```
optional<ll> modInv(ll a, ll m) {
  assert(m > 1);
  a %= m;
  if (a < 0) a += m;
  ll r = m, next = a;
  __int128 x = 0, y = 1;
  while (next) {
    ll q = r / next;
    ll rem = r % next;
    r = next; next = rem;
    __int128 z = x - (__int128)q*y;
    x = y; y = z;
  }
  if (r != 1) return nullopt;
  x %= m;
  if (x < 0) x += m;
  return ll(x);
}
```

GeneralCRT.h – Combine congruences with possibly non-coprime moduli.

- Combine two repeating schedules: x has remainder a modulo m and remainder b modulo n. m,n>0; any signed a,b.
- crtMerge(a,m,b,n) returns (r,M): all solutions r+k*M, 0<=r<M, M=lcm(m,n). (-1,0): incompatible; (0,-1): M exceeds ll. Moduli need not be coprime.
- $O(\log(\max(m,n)))$ time/space.

Usage: `auto [r,m] = crtMerge(2,6,5,9);`
`// (14,18); check m>0 before using r`
`"ExtGCD.h"`

```
pair<ll,ll> crtMerge(ll a, ll m, ll b, ll n) {
  assert(m > 0 && n > 0);
  a %= m; if (a < 0) a += m;
  b %= n; if (b < 0) b += n;
  ll x, y, g = extGcd(m,n,x,y), d = b-a;
  if (d%g) return {-1,0};
  if (m/g > LLONG_MAX/n) return {0,-1};
  ll M = (m/g)*n;
  ll k = ll((__int128)(d/g)*x % (n/g));
  ll r = ll(((__int128)m*k+a)%M);
  return {r < 0 ? r+M : r,M};
}
```

ModLog.h – Discrete logarithm: smallest positive exponent $a^x = b$ modulo m.

Signed a,b are normalized; $1 \leq m \leq 10^{12}$. Returns x>=1, or -1 if impossible. Works even when gcd(a,m)>1; modLog(a,1,m) gives the multiplicative order if gcd(a,m)=1. For nonnegative x, check b==1 modulo m first and return 0 yourself. Baby-step giant-step: O(sqrt(m)) expected time and memory; the modulus bound limits the table to about one million entries.

Usage: `assert(modLog(2,8,13) == 3);`
`"../numerical/ModPower.h"`

```
ll modLog(ll a, ll b, ll m) {
    assert(1 <= m && m <= 1000000000000LL);
    a %= m; if (a < 0) a += m;
    b %= m; if (b < 0) b += m;
    ll n = ll(sqrtl(m))+1, e = 1%m, f = e, j = 1;
    unordered_map<ll,ll> baby;
    while (j <= n) {
        e = f = modMul(e,a,m);
        if (e == b) return j;
        baby[modMul(e,b,m)] = j++;
    }
    if (gcd(m,e) == gcd(m,b))
        for (ll i = 2; i <= n+1; ++i) {
            e = modMul(e,f,m);
            auto it = baby.find(e);
            if (it != baby.end()) return n*i-it->second;
        }
    return -1;
}
```

ModSqrt.h – Solve $x^2 = a$ modulo a prime with Tonelli–Shanks.

Signed a and PRIME p>=2 (up to LLONG_MAX). Returns optional root x in [0,p), nullopt if no root; the other root is (p-x)%p. Handles p=2 and a=0. Products use 128 bits. Main loop $O(\log^2 p)$ modular operations, plus the search for a quadratic non-residue; no small worst-case bound claimed for that search. O(1) memory.

Usage: `auto x = modSqrt(10,13); // roots 6 and 7`
`"../numerical/ModPower.h"`

```
optional<ll> modSqrt(ll a, ll p) {
    assert(p >= 2); a %= p; if (a < 0) a += p;
    if (p == 2 || a == 0) return a;
    if (modPow(a,(p-1)/2,p) != 1) return nullopt;
    if (p%4 == 3) return modPow(a,p/4+1,p);
    ll d = p-1, z = 2; int r = 0;
    while (d%2 == 0) { d /= 2; ++r; }
    while (modPow(z,(p-1)/2,p) != p-1) ++z;
    ll x = modPow(a,(d+1)/2,p), t = modPow(a,d,p);
    ll c = modPow(z,d,p);
    while (t != 1) {
        ll u = t; int i = 0;
        while (u != 1) { u = modMul(u,u,p); ++i; }
        ll b = modPow(c,1ULL<<(r-i-1),p);
        x = modMul(x,b,p); c = modMul(b,b,p);
        t = modMul(t,c,p); r = i;
    }
    return x;
}
```

ModSum.h – Sum floors/remainders of an arithmetic progression without iterating n times.

floorSum(n,m,a,b)=sum floor((a*i+b)/m), 0<=i<n. modSum uses mathematical nonnegative remainder instead of floor. n>=0, m>0; all four parameters $\leq 10^9$ in absolute value. Signed a,b allowed. Returns signed __int128; print using Int128.h. O(log m) time, O(1) memory. Cast before products outside this function too.

Usage: `assert(floorSum(4,3,2,1) == 4); // 0+1+1+2`

```
__int128 floorSum(ll n, ll m, ll a, ll b) {
    assert(0 <= n && n <= 1000000000 && 0 < m && m <=
    1000000000);
    assert(-1000000000 <= a && a <= 1000000000);
```

```
    assert(-1000000000 <= b && b <= 1000000000);
    ll qa = a/m-(a%m < 0), qb = b/m-(b%m < 0);
    __int128 ans = (__int128)qa*n*(n-1)/2+(__int128)qb*n;
    a -= qa*m; b -= qb*m;
    for (;;) {
        ans += (__int128)(a/m)*n*(n-1)/2+(__int128)(b/m)*n;
        a %= m; b %= m;
        ll y = a*n+b;
        if (y < m) return ans;
        n = y/m; b = y%m; swap(m,a);
    }
}
__int128 modSum(ll n, ll m, ll a, ll b) {
    __int128 floors = floorSum(n,m,a,b);
    return (__int128)a*n*(n-1)/2+(__int128)b*n-(__int128)m*
    floors;
}
```

5.3 Primes and factorization

SPF.h – Smallest-prime-factor sieve for many factorizations.

SPF(lim) covers [0,lim), lim>=0. p[x] is the least prime factor; for x>=2, p[x]==x means prime; p[0]=0 and p[1]=1 when present. factor(x), 1<=x<lim, returns sorted prime factors with repetition; factor(1) is empty. Build O(lim log log lim) time, O(lim) memory; query O(log x).

Usage: `SPF s(101); auto f = s.factor(60); // {2,2,3,5}`

```
struct SPF {
  vec<int> p;
  SPF(int lim) : p(lim) {
    iota(p.begin(), p.end(), 0);
    for (int i = 2; i <= (lim-1)/i; ++i)
      if (p[i] == i)
        for (ll j = ll(i)*i; j < lim; j += i)
          if (p[j] == j) p[j] = i;
  }
  vec<int> factor(int x) const {
    assert(1 <= x && x < sz(p));
    vec<int> f;
    while (x > 1) {
      f.pb(p[x]);
      x /= p[x];
    }
    return f;
  }
};
```

TrialDivision.h – Factor a small integer without preprocessing.

trialDivision(n), integer n>=1, returns sorted prime factors with repetition. n=1 returns empty. O(sqrt(n)) time, O(log n) output size; slow for large primes. Use SPF for many bounded queries or factor64 for large 64-bit integers.

Usage: `auto f = trialDivision(84LL); // {2,2,3,7}`

```
template<class T> vec<T> trialDivision(T n) {
  assert(n >= 1);
  vec<T> f;
  for (T p = 2; p <= n/p; ++p) {
    while (n % p == 0) {
      f.pb(p);
      n /= p;
    }
  }
  if (n > 1) f.pb(n);
  return f;
}
```

Prime64.h – Deterministic primality testing over the full unsigned 64-bit range.

- Ask whether one large integer is prime. isPrime64(n) returns bool for any unsigned 64-bit n; 0 and 1 return false. It does not return factors.
- Miller–Rabin: write $n-1=d\cdot 2^s$, then test seven bases. These bases make the answer exact throughout the 64-bit range. O(log n) modular products, O(1) space.

Usage: `bool prime = isPrime64(1000000007ULL);`

`"../numerical/ModPower.h"`

```
bool isPrime64(ull n) {
  if (n < 2 || n % 2 == 0) return n == 2;
  int s = 0;
  ull d = n-1;
  while (d % 2 == 0) { d /= 2; ++s; }
  for (ull a : {2ULL, 325ULL, 9375ULL, 28178ULL,
    450775ULL, 9780504ULL, 1795265022ULL}) {
    if (a % n == 0) continue;
    ull x = modPow(a%n, d, n);
    if (x == 1 || x == n-1) continue;
    bool passed = false;
    for (int j = 1; j < s; ++j) {
      x = modMul(x, x, n);
      if (x == n-1) { passed = true; break; }
    }
    if (!passed) return false;
  }
  return true;
}
```

Factor64.h – Prime factorization of large unsigned 64-bit integers.

- Split a large integer into prime factors when trial division is too slow. factor64(n), unsigned n>=1, returns a sorted vec with repeats; factor64(1) is empty.
- Call factor64, not rho64: rho64 needs a composite and finds just one divisor. It advances x/y, batches differences in q, then tries a gcd; a failed batch is retried step by step.
- Randomized Pollard–Rho/Brent: expected $O(n^{1/4})$ per split, O(log n) factorization space.

Usage: `auto f = factor64(360); // {2,2,2,3,3,5}`

`"Prime64.h"`

```
ull rho64(ull n) {
  if (n % 2 == 0) return 2;
  if (n % 3 == 0) return 3;
  static mt19937_64 rng(
    chrono::steady_clock::now()
    .time_since_epoch().count());
  for (;;) {
    ull y = rng()%(n-1)+1, c = rng()%(n-1)+1;
    ull r = 1, g = 1, x = 0, ys = 0;
    auto f = [&](ull v) {
      ull z = modMul(v, v, n);
      return z < n-c ? z+c : z-(n-c);
    };
    while (g == 1) {
      x = y;
      for (ull i = 0; i < r; ++i) y = f(y);
      for (ull k = 0; k < r && g == 1;
          k += 128) {
        ys = y;
        ull q = 1;
        for (ull i = 0; i < min(128ULL,r-k);
            ++i) {
          y = f(y);
          ull d = x > y ? x-y : y-x;
          q = modMul(q, d, n);
        }
        g = gcd(q, n);
      }
      r *= 2;
    }
    if (g == n) do {
      ys = f(ys);
      g = gcd(x > ys ? x-ys : ys-x, n);
    } while (g == 1);
    if (g < n) return g;
  }
}

vec<ull> factor64(ull n) {
  assert(n >= 1);
  if (n == 1) return {};
  if (isPrime64(n)) return {n};
  ull p = rho64(n);
  auto a = factor64(p), b = factor64(n/p);
  a.insert(a.end(), b.begin(), b.end());
  sort(a.begin(), a.end());
  return a;
}
```

5.4 Divisors

Divisors.h – Enumerate all positive divisors from a prime factorization.

Pass SORTED prime factors with repetition, not n itself; their product must fit T. Returns divisors in unspecified order; empty factors (n=1) return {1}. For each distinct prime, extends previous divisors by all its powers. O(tau(n)) time and output size.

Usage: `auto d = divisors(vec<ll>{2,2,3});`
`sort(all(d)); // {1,2,3,4,6,12}`

```
template<class T>
vec<T> divisors(const vec<T>& f) {
  vec<T> d{1};
  for (int i = 0; i < sz(f);) {
    int k = sz(d), j = i;
    T power = 1;
    while (j < sz(f) && f[j] == f[i]) {
      power *= f[i];
      for (int q = 0; q < k; ++q)
        d.pb(d[q]*power);
      ++j;
    }
    i = j;
  }
  return d;
}
```

- $n \geq 1$, $n = \prod p_i^{e_i}$:

$$\tau(n) = \prod (e_i + 1), \qquad \varphi(n) = n \prod (1 - 1/p_i).$$

  For φ, start at n and apply $v \mathrel{-}= v/p_i$ once per distinct prime.
- Sum of divisors:

$$\sigma(n) = \prod (1 + p_i + \cdots + p_i^{e_i}) = \prod \frac{p_i^{e_i+1} - 1}{p_i - 1}.$$

  Use geometric sums modulo m unless denominators are invertible; exact intermediate sums/powers may need `unsigned __int128`, even for 64-bit n.
- $n = 1$: no prime factors; divisors {1} and $\tau(1) = \sigma(1) = \varphi(1) = 1$.

5.5 Totient and Mobius

TotientMobius.h – Euler phi and Mobius mu for all integers through N.

phi[n] counts integers 1..n coprime to n; mu[n] is zero for a squared prime factor, otherwise $(-1)^k$ for k distinct primes. numberFunctions(N) returns the pair (phi,mu), both N+1 entries; index 0 unused. phi[1]=mu[1]=1. N>=1. lp stores least prime factors internally. O(N) time and memory. See divisor-sum identities in the chapter.

Usage: `auto [phi,mu] = numberFunctions(12); // phi[12]=4, mu[12]=0`

```
pair<vec<int>,vec<int>> numberFunctions(int n) {
    assert(n >= 1);
    vec<int> phi(n+1), mu(n+1), lp(n+1), primes;
    phi[1] = mu[1] = 1;
    rep(i,2,n+1) {
        if (!lp[i]) { lp[i]=i; phi[i]=i-1; mu[i]=-1; primes.
        pb(i); }
        for (int p : primes) {
            if (p > n/i) break;
            int v = i*p; lp[v] = p;
            if (i%p == 0) { phi[v] = phi[i]*p; mu[v] = 0;
            break; }
            phi[v] = phi[i]*(p-1); mu[v] = -mu[i];
        }
    }
    return {phi,mu};
}
```

- $\sum_{d|n}\varphi(d)=n$, $\sum_{d|n}\mu(d)=[n=1]$. For $\gcd(a,n)=1$, $a^{\varphi(n)}\equiv 1 \pmod n$.
- Mobius inversion: if $g(n)=\sum_{d|n}f(d)$, then $f(n)=\sum_{d|n}\mu(d)g(n/d)$. For finite arrays, $g(n)=\sum_{n|d}f(d)$ gives $f(n)=\sum_{n|d}\mu(d/n)g(d)$.
- Primitive Pythagorean triples: $(m^2-n^2, 2mn, m^2+n^2)$ with $m>n>0$, coprime m,n of opposite parity. Multiply by k>0 for all triples; interchange the legs if needed.

Combinatorial (6)

6.1 Factorials and coefficients

- $0!=1$, $n!=n(n-1)!$. Exact factorials fit int through $n=12$, ll through $n=20$; use `cpp_int` beyond these limits.
- For $n\ge 0$, $0\le k\le n$:

$$\binom{n}{k}=\frac{n!}{k!(n-k)!}=\binom{n}{n-k}.$$

Set $\binom{n}{k}=0$ for $k<0$ or $k>n$.

Binomial.h – Exact number of unordered choices.

binom(n,k) returns C(n,k), n>=0; k outside [0,n] gives zero. Result must fit T (ll by default); binom<cpp_int> supports arbitrary precision. Cancels denominators before multiplying. O(min(k,n-k) log(n+1)) arithmetic time; big-integer operations cost more.

Usage: `assert(binom(8,3) == 56);`

```
template<class T = ll> T binom(ll n, ll k) {
  assert(n >= 0);
  if (k < 0 || k > n) return T(0);
  k = min(k, n-k);
  T ans = 1;
  for (ll i = 1; i <= k; ++i) {
    ll a = n-k+i, b = i, g = gcd(a,b);
    a /= g; b /= g;
    ans = T(ans / b * a);
  }
  return ans;
}
```

ModComb.h – Many binomial coefficients modulo one prime.

Construct ModComb(lim,prime): $1\le lim\le prime$, PRIME modulus, $0\le n<lim$. choose(n,k) returns C(n,k) modulo p, or zero for k outside [0,n]. fact[i]=i!, invfact[i]=inverse factorial. The integer type is inferred from prime; products use 128 bits. Build $O(lim+\log p)$, memory $O(lim)$; each query $O(1)$. For n>=p use Lucas when p is small, not these factorials directly.

Usage:
```
ModComb c(101,1000000007LL);
assert(c.choose(100,3) == 161700);
```

"../numerical/ModPower.h"

```
template<class T = ll> struct ModComb {
  T p;
  vec<T> fact, invfact;
  ModComb(int lim, T prime)
    : p(prime), fact(lim), invfact(lim) {
    assert(p >= 2 && 1 <= lim && T(lim) <= p);
    fact[0] = 1;
    for (int i = 1; i < lim; ++i)
      fact[i] = modMul(fact[i-1], T(i), p);
    invfact[lim-1] = modPow(fact[lim-1], ull(p-2), p);
    for (int i = lim-1; i > 0; --i)
      invfact[i-1] = modMul(invfact[i], T(i), p);
  }
  T choose(ll n, ll k) const {
    assert(0 <= n && n < sz(fact));
    if (k < 0 || k > n) return T(0);
    T ans = modMul(fact[n], invfact[k], p);
    return modMul(ans, invfact[n-k], p);
  }
};
```

- Pascal: $\binom{n}{0}=\binom{n}{n}=1$, $\binom{n}{k}=\binom{n-1}{k-1}+\binom{n-1}{k}$. Rows through N in $O(N^2)$ time, $O(N)$ space; update k downwards. Also works modulo any positive m; widen sums before reducing.
- Multinomial: counts $k_i\ge 0$, $s_i=\sum_{j\le i}k_j$, $N=\sum_i k_i$; permutations of a multiset:

$$\binom{N}{k_1,\dots,k_r}=\frac{N!}{\prod_i k_i!}=\prod_{i=1}^{r}\binom{s_i}{k_i}.$$

Use the product of binomials for exact arithmetic; modulo p use `fact[N]` $\prod_i$ `invfact[`k_i`]`, with $N<lim\le p$. Empty product is 1. Products/results must fit the chosen type; use `modMul` for modular products.

6.2 Inclusion–exclusion

- For finite sets $A_1,\dots,A_k$:

$$\left|\bigcup_{i=1}^{k}A_i\right|=\sum_{\emptyset\ne S\subseteq\{1,\dots,k\}}(-1)^{|S|+1}\left|\bigcap_{i\in S}A_i\right|.$$

- Avoid all forbidden sets: include the empty-set term $|\Omega|$ and use signs $(-1)^{|S|}$. There are 2^k terms; cost depends on computing intersections. Use signed/wider accumulators; normalize modular subtraction.

6.3 Stars and bars

- n indistinguishable units, k labelled boxes, $n\ge 0$, $k\ge 1$:

$$x_i\ge 0,\ \sum x_i=n:\ \binom{n+k-1}{k-1},$$
$$x_i\ge 1,\ \sum x_i=n:\ \binom{n-1}{k-1}\quad(n\ge k).$$

Positive case is zero when $n<k$. For $k=0$, count is $[n=0]$.

- Bounds $l_i\le x_i\le u_i$, $l_i\ge 0$: set $y_i=x_i-l_i$, replace n by $n-\sum l_i$, and upper bounds by u_i-l_i. If any $u_i<l_i$, zero solutions. Apply inclusion–exclusion to violations $y_i\ge u_i-l_i+1$. Skip negative residual sums before calling `binom`.

6.4 Catalan numbers

- $C_0=1$; for $n\ge 0$:

$$C_n=\frac{1}{n+1}\binom{2n}{n}=\binom{2n}{n}-\binom{2n}{n+1}.$$

Exact binomial terms must also fit the chosen type.

$$C_{n+1}=\sum_{i=0}^{n}C_iC_{n-i}.$$

Recurrence: $O(N^2)$ time, $O(N)$ space through C_N.

- Counts: n matched parenthesis pairs; ordered full binary trees with n internal nodes; triangulations of a convex polygon with $n+2$ vertices ($n\ge 1$).
- Difference/recurrence avoid modular division. `ModComb` needs $2n<lim\le p$; division by $n+1$ modulo m requires $\gcd(n+1,m)=1$.

6.5 Permutation cycles and parity

PermCycles.h – Lengths of the disjoint cycles of a permutation.

p must be a permutation of [0,n), mapping i to p[i]. Returns cycle lengths in unspecified order; fixed points have length 1. Empty input gives empty output. With c cycles, n-c is the minimum number of arbitrary swaps. O(n) time/memory.

Usage: `auto len = cycleLengths({1,2,0,4,3}); // {3,2}`

```
vec<int> cycleLengths(const vec<int>& p) {
  vec<int> len, vis(sz(p));
  for (int i = 0; i < sz(p); ++i) if (!vis[i]) {
    int j = i, k = 0;
    do {
      vis[j] = 1; ++k;
      j = p[j];
    } while (j != i);
    len.pb(k);
  }
  return len;
}
```

- For a permutation of $[0,n)$ with c cycles, including fixed points:

$$\text{sign}=(-1)^{n-c},\qquad \text{minimum arbitrary swaps}=n-c.$$

Parity is $(n-c)$ mod 2; one transposition flips it. Repeated values or restricted swaps require a different model.

- Order under composition is the LCM of cycle lengths; empty permutation has order 1. Check LCM overflow (Number theory).

6.6 Counting reference

- Derangements (no fixed point): $D_0=1, D_1=0$, $D_n=(n-1)(D_{n-1}+D_{n-2})$ for $n\ge 2$.
- Lucas, PRIME p: write n,k in base p, then $\binom{n}{k}\equiv\prod_i\binom{n_i}{k_i} \pmod p$. Precompute `ModComb(p,p)` only if p is small enough to store.
- Unsigned Stirling first kind (permutations with k cycles): $c(n,k)=c(n-1,k-1)+(n-1)c(n-1,k)$. Second kind (partitions into k nonempty unlabelled groups): $S(n,k)=S(n-1,k-1)+kS(n-1,k)$. Both start at (0,0)=1 and are zero outside valid indices. Bell $B_n=\sum_k S(n,k)$ counts all set partitions.
- Eulerian numbers (permutations with k descents): $E(n,k)=(n-k)E(n-1,k-1)+(k+1)E(n-1,k)$; $E(0,0)=1$, invalid indices zero.
- Integer partitions, order ignored: $p(0)=1$, negative arguments zero; $p(n)=\sum_{k\ge 1}(-1)^{k+1}[p(n-k(3k-1)/2)+p(n-k(3k+1)/2)]$. Stop when both arguments are negative; through N, $O(N\sqrt{N})$ time.
- Burnside: configurations up to a symmetry group G: $|G|^{-1}\sum_{g\in G}|\operatorname{Fix}(g)|$. Necklaces: $n\ge 1$ positions, c colours, rotations only: $n^{-1}\sum_{k=0}^{n-1}c^{\gcd(n,k)}$. Modular division requires an invertible group size.
- Labelled trees on n>=2 vertices: n^{n-2}; for prescribed positive degrees summing to $2n-2$: $(n-2)!/\prod_i(d_i-1)!$. One vertex has one tree. For a graph's spanning trees, use Matrix–Tree: delete one row/column of its degree-minus-adjacency Laplacian and take the determinant.

Graph (7)

- Vertices $[0, V)$. Adjacency is directed unless stated otherwise: `vec<vec<int>> g(n)` or weighted `vec<vec<pair<int,ll>>>`. An edge is $(to, weight)$; insert both directions for an undirected graph. Edge-list algorithms receive each undirected edge once.
- Parents use -1 at roots and unreachable vertices. Distances use the sentinel documented by each algorithm: BFS -1, weighted paths infinity.

7.1 Traversal and shortest paths

Traversal.h – BFS distances, paths and bipartite colouring

Unit-cost graph g[u]=outgoing neighbours; bfs(g,src) accepts one or several sources and returns (dist,parent). dist=-1 means unreachable; parent=-1 at sources and unreachable vertices. Duplicate/empty sources are allowed. restorePath(s,t,parent) returns s..t, or empty if the chain does not reach s; parents must be acyclic. bipartite(g) expects an undirected graph and returns (ok,color), checking every component and self-loop; colors 0/1 are valid only if ok. BFS/colouring $O(V + E + |src|)$, path $O(V)$; extra space $O(V)$.

Usage:
```
vec<vec<int>> g{{1},{0,2},{1}};
auto [d,p] = bfs(g,{0}); // d={0,1,2}
auto path = restorePath(0,2,p);
auto [ok,color] = bipartite(g);
```

```
pair<vec<int>,vec<int>> bfs(const vec<vec<int>>& g,
    const vec<int>& src) {
  vec<int> dist(sz(g),-1), parent(sz(g),-1), q;
  for (int s : src) {
    assert(0 <= s && s < sz(g));
    if (dist[s] < 0) dist[s] = 0, q.pb(s);
  }
  rep(i,0,sz(q)) {
    int v = q[i];
    for (int u : g[v]) if (dist[u] < 0) {
      dist[u] = dist[v]+1; parent[u] = v;
      q.pb(u);
    }
  }
  return {dist,parent};
}
vec<int> restorePath(int s, int t, const vec<int>& parent) {
  assert(0 <= s && s < sz(parent));
  assert(0 <= t && t < sz(parent));
  vec<int> path;
  for (int v = t; v >= 0; v = parent[v]) {
    path.pb(v);
    if (v == s) {
      reverse(path.begin(),path.end());
      return path;
    }
  }
  return {};
}
pair<bool,vec<int>> bipartite(const vec<vec<int>>& g) {
  vec<int> color(sz(g),-1), q;
  rep(s,0,sz(g)) {
    if (color[s] >= 0) continue;
    color[s] = 0; q = {s};
    rep(i,0,sz(q)) {
      int v = q[i];
      for (int u : g[v]) {
        if (color[u] < 0) {
          color[u] = color[v]^1; q.pb(u);
        } else if (color[u] == color[v])
          return {false,color};
      }
    }
  }
  return {true,color};
}
```

GridBFS.h – Shortest paths through a grid

Rectangular nonempty character grid; wall blocks movement, all other cells are free. Sources are valid free (row,column) pairs; moves are orthogonal with unit cost. Returns (dist,parent) indexed by id=row*C+column; decode with id/C and id%C. dist=-1 for walls/unreachable cells; parent=-1 for sources/unreachable cells. Several sources give distance to the nearest. Follow parents to recover a path. $RC \le INT_MAX$; time $O(RC + |src|)$, space $O(RC)$.

Usage:
```
vec<string> a{"...",".#.","..."};
auto [d,p] = gridBfs(a,{{0,0}});
// d[2*3+2]=4; p contains the path.
```

```
pair<vec<int>,vec<int>> gridBfs(const vec<string>& a,
    const vec<pair<int,int>>& src,
    char wall = '#') {
  assert(!a.empty());
  int R = sz(a), C = sz(a[0]);
  assert(C > 0 && (ll)R*C <= INT_MAX);
  rep(r,0,R) assert(sz(a[r]) == C);
  vec<int> dist(R*C,-1), parent(R*C,-1), q;
  for (auto [r,c] : src) {
    assert(0 <= r && r < R && 0 <= c && c < C);
    assert(a[r][c] != wall);
    int v = r*C+c;
    if (dist[v] < 0) dist[v] = 0, q.pb(v);
  }
  const int dr[4] = {-1,1,0,0};
  const int dc[4] = {0,0,-1,1};
  rep(i,0,sz(q)) {
    int v = q[i], r = v/C, c = v%C;
    rep(k,0,4) {
      int nr = r+dr[k], nc = c+dc[k];
      if (nr < 0 || nr >= R || nc < 0 || nc >= C)
        continue;
      int u = nr*C+nc;
      if (a[nr][nc] == wall || dist[u] >= 0) continue;
      dist[u] = dist[v]+1; parent[u] = v;
      q.pb(u);
    }
  }
  return {dist,parent};
}
```

Dijkstra.h – Shortest paths with nonnegative edge weights

g[u] contains (to,weight), weight>=0; add both directions for an undirected edge. dijkstra(g,s) or dijkstra(g,src) returns (dist,parent), from the nearest source; a source starts at cost 0. dist=inf means unreachable; finite answers must be <inf. parent=-1 at sources/unreachable vertices; use restorePath from Traversal.h. Parents change only on strict improvements. Heap version: $O((V + E)\log(V + E + 1))$ time and $O(V + E)$ extra space. dijkstraDense(g,s) instead scans all vertices: $O(V^2 + E)$ time, $O(V)$ extra space; useful for dense graphs.

Usage:
```
vec<vec<pair<int,ll>>> g(3);
g[0]={{1,7}}; g[1]={{2,2}};
auto [d,p] = dijkstra(g,0); // {0,7,9}
```

```
pair<vec<ll>,vec<int>> dijkstra(
    const vec<vec<pair<int,ll>>>& g,
    const vec<int>& src,
    ll inf = LLONG_MAX/4) {
  assert(inf > 0);
  vec<ll> dist(sz(g),inf);
  vec<int> parent(sz(g),-1);
  using P = pair<ll,int>;
  priority_queue<P,vec<P>,greater<P>> pq;
  for (int s : src) {
    assert(0 <= s && s < sz(g));
    if (dist[s] != 0) dist[s] = 0, pq.push({0,s});
  }
  while (!pq.empty()) {
    auto [du,u] = pq.top(); pq.pop();
    if (du != dist[u]) continue;
    for (auto [v,w] : g[u]) {
      assert(w >= 0);
      if (w >= inf-du) continue;
      ll nd = du+w;
      if (nd < dist[v]) {
        dist[v] = nd; parent[v] = u;
        pq.push({nd,v});
      }
    }
  }
  return {dist,parent};
}

pair<vec<ll>,vec<int>> dijkstra(
    const vec<vec<pair<int,ll>>>& g,
    int s, ll inf = LLONG_MAX/4) {
  return dijkstra(g,vec<int>{s},inf);
}
pair<vec<ll>,vec<int>> dijkstraDense(
    const vec<vec<pair<int,ll>>>& g,
    int s, ll inf = LLONG_MAX/4) {
  int n=sz(g);
  assert(0<=s && s<n && inf>0);
  vec<ll> d(n,inf);
  vec<int> parent(n,-1), used(n);
  d[s]=0;
  rep(i,0,n) {
    int u=-1;
    rep(v,0,n) if (!used[v] && (u<0 || d[v]<d[u])) u=v;
    if (u<0 || d[u]==inf) break;
    used[u]=1;
    for (auto [v,w] : g[u]) {
      assert(w>=0);
      if (w<inf-d[u] && d[u]+w<d[v]) {
        d[v]=d[u]+w; parent[v]=u;
      }
    }
  }
  return {d,parent};
}
```

ZeroOneBFS.h – Shortest paths with weights 0 or 1

g[u] contains (to,weight), weight is 0 or 1. src contains one or several sources, initially at distance 0. Returns (dist,parent) from the nearest source: INT_MAX means unreachable; parent=-1 at sources/unreachable vertices. Sources may repeat or be empty. Weight 0 goes to the deque front, 1 to the back; ordinary BFS does not work for mixed weights. Use restorePath from Traversal.h. Time $O(V + E + |src|)$, extra space $O(V + E)$.

Usage:
```
vec<vec<pair<int,int>>> g(3);
g[0]={{1,0},{2,1}}; g[1]={{2,0}};
auto [d,p] = zeroOneBfs(g,{0}); // {0,0,0}
```

```
pair<vec<int>,vec<int>> zeroOneBfs(
    const vec<vec<pair<int,int>>>& g,
    const vec<int>& src) {
  vec<int> d(sz(g),INT_MAX), parent(sz(g),-1);
  deque<pair<int,int>> q;
  for (int s : src) {
    assert(0 <= s && s < sz(g));
    if (d[s] != 0) {
      d[s] = 0; q.pb({0,s});
    }
  }
  while (!q.empty()) {
    auto [dv,v] = q.front(); q.pop_front();
    if (dv != d[v]) continue;
    for (auto [u,w] : g[v]) {
      assert(w == 0 || w == 1);
      if (dv+w >= d[u]) continue;
      d[u] = dv+w; parent[u] = v;
      if (w) q.pb({d[u],u});
      else q.push_front({d[u],u});
    }
  }
  return {d,parent};
}
```

```
  }
  return {d,parent};
}
```

BellmanFord.h – Shortest paths and reachable negative cycles

Create fresh nodes(V) and edges (from,to,intWeight); call BF::bellmanFord(nodes,edges,s). Reads nodes[v].dist and .prev (previous vertex). BF::inf means unreachable; -inf means a reachable negative cycle can reduce the cost to v without bound. Sorts edges in place; prev is meaningful only for finite distances and may not retain the usual source-parent convention when negative cycles exist. All intermediate finite sums must fit $(-inf, inf)$; $V(E+1)\max|w| < inf$ suffices, inf=LLONG_MAX. Time $O(V(E+1)+E\log(E+1))$, extra space $O(\log(E+1))$.

Usage: `vec<BF::Node> nodes(3);`
`vec<BF::Ed> edges{{0,1,4},{1,2,-2}};`
`BF::bellmanFord(nodes,edges,0);`
`ll answer=nodes[2].dist; // 2`

```
namespace BF {
const ll inf = LLONG_MAX;
struct Ed {
  int a, b, w, s() {
    return a < b ? a : -a;
  }
};
struct Node { ll dist = inf; int prev = -1; };

void bellmanFord(vec<Node>& nodes,
    vec<Ed>& eds, int s) {
  nodes[s].dist = 0;
  sort(eds.begin(), eds.end(), [](Ed a, Ed b) {
    return a.s() < b.s();
  });

  int lim = sz(nodes) / 2 + 2;
  // /3+100 com vértices baralhados
  rep(i,0,lim)
    for (Ed ed : eds) {
      Node cur = nodes[ed.a],
        &dest = nodes[ed.b];
      if (abs(cur.dist) == inf) continue;
      ll d = cur.dist + ed.w;
      if (d < dest.dist) {
        dest.prev = ed.a;
        dest.dist = (i < lim-1 ? d : -inf);
      }
    }
  rep(i,0,lim)
    for (Ed e : eds) {
      if (nodes[e.a].dist == -inf)
        nodes[e.b].dist = -inf;
    }
}
} // namespace BF
```

FloydWarshall.h – All-pairs shortest paths, including negative cycles

Square ll matrix m: m[u][v]=edge cost, FW::inf if absent; take the minimum of parallel edges. Replaces m with shortest distances, with diagonal at most 0. inf means unreachable; -inf means a path through a negative cycle. Allows negative edges; suitable for small V. Require $V\max|w| < inf = 2^{62}$; time $O(V^3)$, extra space $O(1)$.

Usage: `vec<vec<ll>> d(3,vec<ll>(3,FW::inf));`
`d[0][1]=4; d[1][2]=-2;`
`FW::floydWarshall(d); // d[0][2]=2`

```
namespace FW {
const ll inf = 1LL << 62;
void floydWarshall(vec<vec<ll>>& m) {
  int n = sz(m);
  rep(i,0,n)
    m[i][i] = min(m[i][i], 0LL);
  rep(k,0,n)
    rep(i,0,n)
      rep(j,0,n)
        if (m[i][k] != inf &&
            m[k][j] != inf) {
          auto newDist = max(
            m[i][k] + m[k][j], -inf);
          m[i][j] = min(m[i][j], newDist);
        }
  rep(k,0,n) if (m[k][k] < 0)
    rep(i,0,n)
      rep(j,0,n)
        if (m[i][k] != inf &&
            m[k][j] != inf)
          m[i][j] = -inf;
}
} // namespace FW
```

7.2 DAG and minimum spanning forest

Kahn.h – Topological order: prerequisites before dependants

For u before v, put v in g[u]. Returns vertices in topological order if the directed graph is acyclic; fewer than V vertices means a cycle prevents a complete order. in counts unfinished prerequisites. A FIFO queue gives any valid order; use a min-heap for lexicographically smallest order. Time $O(V+E)$, extra space $O(V)$.

Usage: `vec<vec<int>> g{{2},{2},{}};`
`auto order=kahn(g); // {0,1,2}`
`bool dag=sz(order)==sz(g);`

```
vec<int> kahn(const vec<vec<int>>& g) {
  vec<int> in(sz(g)), q;
  for (auto& adj : g) for (int v : adj) ++in[v];
  rep(v,0,sz(g))
    if (!in[v]) q.pb(v);
  rep(i,0,sz(q))
    for (int v : g[q[i]]) if (!--in[v])
      q.pb(v);
  return q;
}
```

- DAG DP: process a complete topological order, ignoring unreachable states. For $u \to v$, relax $d[v]$ with $d[u]+w$ or add $ways[u]$. Negative edges are allowed in a DAG. Time $O(V+E)$.

Kruskal.h – Minimum spanning forest of an undirected graph

n vertices; e[i]=(u,v,weight), each undirected edge supplied once. Returns (totalCost,chosenEdgeIDs); IDs index the original e, which is unchanged. Disconnected input gives one tree per component: components=n-chosen.size(). Accepts negative weights, parallel edges and self-loops. T is the weight/sum type; prefer ll. Uses UF from UnionFind.h. Time $O(V+E\log(E+1))$, space $O(V+E)$.

Usage: `vec<tuple<int,int,ll>> e{{0,1,4},{1,2,2},{0,2,9}};`
`auto [cost,ids]=kruskal(3,e); // cost=6`
`"../data-structures/UnionFind.h"`

```
template<class T>
pair<T,vec<int>> kruskal(int n,
    const vec<tuple<int,int,T>>& e) {
  vec<int> ord(sz(e)), ids;
  iota(ord.begin(),ord.end(),0);
  sort(ord.begin(),ord.end(),[&](int a,int b) {
    return get<2>(e[a]) < get<2>(e[b]);
  });
  UF uf(n);
  T cost = 0;
  for (int i : ord) {
    auto [u,v,w] = e[i];
    if (uf.join(u,v)) {
      cost += w; ids.pb(i);
    }
  }
  return {cost,ids};
}
```

7.3 Connectivity and constraints

StrongComponents.h – Strongly connected components and their DAG

Directed adjacency g[u]. kosaraju(g) returns comp[v], grouping mutually reachable vertices. Component IDs are contiguous and increase along every edge between different components. condensation(g,comp) returns the DAG of components without duplicates. Both DFS passes are iterative, safe on long chains. Neither changes g. Each function uses $O(V+E)$ time/space; TwoSAT relies on this component ordering.

Usage: `vec<vec<int>> g{{1},{0,2},{3},{2}};`
`auto comp=kosaraju(g); // {0,0,1,1}`
`auto dag=condensation(g,comp); // {{1},{}}`

```
vec<int> kosaraju(const vec<vec<int>>& g) {
  int n = sz(g), count = 0;
  vec<vec<int>> rev(n);
  rep(v,0,n)
    for (int u : g[v]) rev[u].pb(v);
  vec<int> seen(n), next(n), order, comp(n,-1), st;
  rep(s,0,n) if (!seen[s]) {
    seen[s] = 1; st.pb(s);
    while (!st.empty()) {
      int v = st.back();
      if (next[v] == sz(g[v])) {
        order.pb(v); st.pop_back();
      } else {
        int u = g[v][next[v]++];
        if (!seen[u]) seen[u] = 1, st.pb(u);
      }
    }
  for (int i = n; i--;) {
    int s = order[i];
    if (comp[s] >= 0) continue;
    comp[s] = count; st.pb(s);
    while (!st.empty()) {
      int v = st.back(); st.pop_back();
      for (int u : rev[v]) if (comp[u] < 0) {
        comp[u] = count; st.pb(u);
      }
    }
    ++count;
  }
  return comp;
}
vec<vec<int>> condensation(const vec<vec<int>>& g,
    const vec<int>& comp) {
  int count = 0;
  for (int c : comp) count = max(count,c+1);
  vec<vec<int>> groups(count), dag(count);
  rep(v,0,sz(g))
    groups[comp[v]].pb(v);
  vec<int> mark(count,-1);
  rep(c,0,count)
    for (int v : groups[c]) for (int u : g[v]) {
      int d = comp[u];
      if (d != c && mark[d] != c) {
        mark[d] = c; dag[c].pb(d);
      }
    }
  return dag;
}
```

Lowlink.h – Bridges and articulation points

Undirected edges e[i]=(u,v), each supplied once; n vertices. bridges contains edge IDs whose removal increases component count; cut[v]=1 marks vertices with the same property. Handles disconnected graphs, self-loops and parallel edges. tin is DFS discovery time; low is the earliest discovery reachable below a vertex using a back edge. Parent edges are tracked by ID. Iterative DFS; time/space $O(V+E)$.

Usage: `vec<pair<int,int>> e{{0,1},{1,2},{2,0},{1,3}};`
`Lowlink b(4,e); // bridges={3}, cut[1]=1`

```
struct Lowlink {
  vec<int> bridges, cut;
  Lowlink(int n, const vec<pair<int,int>>& e)
    : cut(n) {
    vec<vec<pair<int,int>>> g(n);
    rep(i,0,sz(e)) {
      auto [u,v] = e[i];
      g[u].pb({v,i});
      g[v].pb({u,i});
    }
    vec<int> tin(n,-1), low(n), par(n,-1), pe(n,-1);
    vec<int> it(n), child(n), st;
    int timer = 0;
    rep(s,0,n) if (tin[s] < 0) {
      tin[s] = low[s] = timer++;
      st.pb(s);
      while (!st.empty()) {
        int v = st.back();
        if (it[v] < sz(g[v])) {
          auto [u,id] = g[v][it[v]++];
          if (id == pe[v]) continue;
          if (tin[u] < 0) {
            par[u] = v; pe[u] = id; ++child[v];
            tin[u] = low[u] = timer++;
            st.pb(u);
          } else low[v] = min(low[v],tin[u]);
        } else {
          st.pop_back();
          int p = par[v];
          if (p < 0) cut[v] = child[v] > 1;
          else {
            low[p] = min(low[p],low[v]);
            if (low[v] > tin[p])
              bridges.pb(pe[v]);
            if (par[p] >= 0 && low[v] >= tin[p])
              cut[p] = 1;
          }
        }
      }
    }
};
```

TwoSAT.h – Satisfy clauses with at most two literals

TwoSAT(n): variables [0,n), literal 2*v means v=true, 2*v+1 means v=false; negate with xor 1. either(a,b) adds a OR b; either(a,a) forces a; implication a=>b is either(a xor 1,b). XOR adds (a OR b) and (not-a OR not-b). solve() returns false if impossible; after true, answer[v] is one valid 0/1 assignment. Re-solving after adding clauses is allowed. Clauses of 3+ literals are unsupported; does not count solutions or optimise true variables. Uses iterative Kosaraju. Add $O(1)$ amortised; solve $O(V+E)$ time/space.

Usage: `TwoSAT s(2);`
`s.either(0,2); s.either(1,3); // exactly one`
`s.either(0,0);`
`bool ok=s.solve(); // answer={1,0}`

"StrongComponents.h"

```
struct TwoSAT {
  vec<vec<int>> g;
  vec<int> answer;
  TwoSAT(int n) : g(2*n) {}
  void either(int a, int b) {
    g[a^1].pb(b);
    g[b^1].pb(a);
  }
  bool solve() {
    vec<int> comp = kosaraju(g);
    answer.resize(sz(g)/2);
    rep(v,0,sz(answer)) {
      int a = comp[2*v], b = comp[2*v+1];
      if (a == b) return false;
      answer[v] = a > b;
    }
    return true;
  }
};
```

Hierholzer.h – Euler trail: use each edge exactly once

n vertices, edges e[i]=(u,v), supplied once even if undirected. directed selects orientation. Returns (vertices,edgeIDs) in trail order: E+1 vertices, E IDs; empty pair means impossible from src. With no edges returns one vertex src and no edge IDs. Self-loops/parallel edges allowed, vertices may repeat. Undirected: 0 or 2 odd degrees, start at an odd vertex if 2. Directed: out-in is +1 at start and -1 at end, or all balanced. A circuit starts at a vertex incident to an edge. Checks degrees and that every edge is used; time/space $O(V+E)$.

Usage: `vec<pair<int,int>> e{{0,1},{1,2}};`
`auto [v,ids]=eulerTrail(3,e,0,false);`
`// v={0,1,2}, ids={0,1}`

```
pair<vec<int>,vec<int>> eulerTrail(int n,
    const vec<pair<int,int>>& e,
    int src, bool directed) {
  if (src < 0 || src >= n) return {};
  vec<vec<pair<int,int>>> g(n);
  vec<int> deg(n);
  rep(i,0,sz(e)) {
    auto [u,v] = e[i];
    g[u].pb({v,i}); ++deg[u];
    if (directed) --deg[v];
    else g[v].pb({u,i}), ++deg[v];
  }
  int pos = 0, neg = 0;
  rep(v,0,n) {
    if (directed) {
      if (deg[v] == 1 && v == src) ++pos;
      else if (deg[v] == -1) ++neg;
      else if (deg[v]) return {};
    } else pos += deg[v]&1;
  }
  if (directed && pos != neg) return {};
  if (!directed && pos &&
      (pos != 2 || !(deg[src]&1))) return {};
  vec<int> it(n), used(sz(e)), path, ids;
  vec<pair<int,int>> st{{src,-1}};
  while (!st.empty()) {
    auto [v,id] = st.back();
    while (it[v] < sz(g[v]) &&
        used[g[v][it[v]].second]) ++it[v];
    if (it[v] == sz(g[v])) {
      path.pb(v);
      if (id >= 0) ids.pb(id);
      st.pop_back();
    } else {
      auto [u,k] = g[v][it[v]++];
      used[k] = 1; st.pb({u,k});
    }
  }
  if (sz(ids) != sz(e)) return {};
  reverse(path.begin(),path.end());
  reverse(ids.begin(),ids.end());
  return {move(path),move(ids)};
}
```

7.4 Flow and matching

- Several terminals: add a super-source/sink. Vertex capacity: split $v_{in} \to v_{out}$ with that capacity, redirect incoming/outgoing edges. Unit edge capacities model edge-disjoint paths; vertex splitting models internally vertex-disjoint paths. An "infinite" capacity must exceed every relevant finite cut and fit the sum type.
- Matching chooses disjoint pairs. Maximum matching optimises their number; assignment/min-cost flow optimises cost; stable matching prevents mutually preferred blocking pairs. These are different objectives.

DinicFlow.h – Maximum flow and minimum s-t cut

add(u,v,capacity) creates a directed edge and its zero-capacity reverse. flow(s,t) sends maximum additional flow and changes residual capacities in g; repeated calls continue the same network. Returns units, not distance. Save the (u,index) handle returned by add: original capacity minus g[u][index].cap is its net flow. After a full flow call, minCut(s) returns a 0/1 vec of vertices reachable from s through positive residual capacity; original edges crossing 1 to 0 form a minimum cut. rev indexes the reverse edge; lev is BFS depth, it is DFS cursor. Require distinct valid s,t, nonnegative capacities, total flow fitting ll. Time $O(V^2E)$, space $O(V+E)$; recursive DFS depth $O(V)$.

Usage: `DinicFlow f(3);`
`auto edge=f.add(0,1,5); f.add(1,2,3);`
`ll sent=f.flow(0,2); // 3`
`auto side=f.minCut(0); // {1,1,0}`

```
struct DinicFlow {
  struct Edge { int to, rev; ll cap; };
  vec<vec<Edge>> g;
  vec<int> lev, it;
  DinicFlow(int n) : g(n), lev(n), it(n) {}
  pair<int,int> add(int u, int v, ll cap) {
    assert(cap >= 0);
    int a = sz(g[u]), b = sz(g[v]);
    g[u].pb({v,b+(u==v),cap});
    g[v].pb({u,a,0});
    return {u,a};
  }
  vec<int> minCut(int s) const {
    vec<int> side(sz(g)), q{s}; side[s]=1;
    rep(i,0,sz(q)) for (auto e : g[q[i]])
      if (e.cap>0 && !side[e.to])
        side[e.to]=1, q.pb(e.to);
    return side;
  }
  bool bfs(int s, int t) {
    fill(lev.begin(), lev.end(), -1);
    queue<int> q;
    q.push(s); lev[s] = 0;
    while (!q.empty()) {
      int u = q.front(); q.pop();
      for (auto e : g[u])
        if (e.cap > 0 && lev[e.to] < 0) {
          lev[e.to] = lev[u]+1;
          q.push(e.to);
        }
    }
    return lev[t] >= 0;
  }
  ll dfs(int u, int t, ll f) {
    if (u == t) return f;
    for (int& i = it[u]; i < sz(g[u]); ++i) {
      auto& e = g[u][i];
      if (e.cap <= 0 || lev[e.to] != lev[u]+1)
        continue;
      ll sent = dfs(e.to, t, min(f,e.cap));
      if (sent > 0) {
        e.cap -= sent; g[e.to][e.rev].cap += sent;
        return sent;
      }
    }
    return 0;
  }
  ll flow(int s, int t) {
    assert(0 <= s && s < sz(g));
    assert(0 <= t && t < sz(g) && s != t);
    ll ans = 0;
    while (bfs(s,t)) {
      fill(it.begin(), it.end(), 0);
      while (ll sent = dfs(s,t,
          LLONG_MAX)) ans += sent;
    }
```

```
    return ans;
  }
};
```

MinCostFlow.h – Send a required or maximum flow at minimum cost

add(u,v,cap,cost): directed capacity and cost per unit. flow(s,t,limit) maximises additional units up to limit, then minimises their total cost; returns (sent,totalCost). For exactly k units, pass k and check sent==k. Changes residual capacities; repeated calls continue the network and return incremental results. Reverse edges cancel earlier choices. Potentials h allow Dijkstra despite negative costs; pv/pe remember the previous vertex/edge. Negative edges allowed, but no negative residual cycle anywhere. Require s!=t, integral cap>=0, limit>=0, all intermediate sums/products fitting ll and distance/potential magnitudes below INF. Time $O(V + VE + A(E + V)\log(V + E + 1))$ for A augmentations; space $O(V + E)$.

Usage: `MinCostFlow f(3);`
`f.add(0,1,2,3); f.add(1,2,2,1);`
`auto [sent,cost]=f.flow(0,2,2); // {2,8}`

```
struct MinCostFlow {
  struct Edge { int to, rev; ll cap, cost; };
  const ll INF = LLONG_MAX/4;
  vec<vec<Edge>> g;
  MinCostFlow(int n) : g(n) {}
  void add(int u, int v, ll cap, ll cost) {
    assert(cap >= 0 && -INF < cost && cost < INF);
    int a = sz(g[u]), b = sz(g[v]);
    g[u].pb({v,b+(u==v),cap,cost});
    g[v].pb({u,a,0,-cost});
  }
  pair<ll,ll> flow(int s, int t,
      ll lim = LLONG_MAX) {
    int n = sz(g);
    assert(0 <= s && s < n && 0 <= t && t < n);
    assert(s != t && lim >= 0);
    vec<ll> h(n), d(n);
    vec<int> pv(n,-1), pe(n,-1);
    rep(k,0,n) {
      bool change = false;
      rep(u,0,n)
        for (auto e : g[u]) if (e.cap > 0)
          if (h[u]+e.cost < h[e.to]) {
            h[e.to] = h[u]+e.cost;
            change = true;
          }
      if (!change) break;
      assert(k+1 < n); // Sem ciclo negativo.
    }
    ll sent = 0, totalCost = 0;
    while (sent < lim) {
      fill(d.begin(), d.end(), INF);
      priority_queue<pair<ll,int>,
        vec<pair<ll,int>>,
        greater<pair<ll,int>>> q;
      d[s] = 0; q.push({0,s});
      while (!q.empty()) {
        auto [du,u] = q.top(); q.pop();
        if (du != d[u]) continue;
        rep(i,0,sz(g[u])) {
          auto e = g[u][i];
          if (e.cap <= 0) continue;
          ll reduced = du+e.cost+h[u]-h[e.to];
          if (reduced < d[e.to]) {
            d[e.to] = reduced;
            pv[e.to] = u; pe[e.to] = i;
            q.push({reduced,e.to});
          }
        }
      }
      if (d[t] == INF) break;
      rep(v,0,n)
        if (d[v] < INF) h[v] += d[v];
      ll amount = lim-sent;
      for (int v = t; v != s; v = pv[v])
        amount = min(amount,g[pv[v]][pe[v]].cap);
      for (int v = t; v != s; v = pv[v]) {
        auto& e = g[pv[v]][pe[v]];
        e.cap -= amount;
        g[v][e.rev].cap += amount;
      }
      sent += amount; totalCost += amount*(h[t]-h[s]);
    }
    return {sent,totalCost};
  }
};
```

BipartiteMatching.h – Maximum bipartite matching and minimum vertex cover

n left and m right vertices, independently numbered from 0. add(u,v) marks a compatible left/right pair. solve() rebuilds a maximum-cardinality matching: l[u]=right partner, r[v]=left partner, -1 if unmatched; returns number of pairs. After solve, cover() returns (leftIDs,rightIDs) of a minimum vertex cover, touching every edge; their complements form a maximum independent set. Do not modify g between solve and cover. Isolated vertices/duplicate edges allowed; this does not optimise costs or preferences. Hopcroft–Karp: $O((V + E)\sqrt{V})$ time, $O(V + E)$ space; cover linear. DFS depth $O(V)$.

Usage: `BipartiteMatching b(2,3);`
`b.add(0,1); b.add(1,1); b.add(1,2);`
`int pairs=b.solve(); // 2`
`auto [left,right]=b.cover();`

```
struct BipartiteMatching {
  int n, m;
  vec<vec<int>> g;
  vec<int> l, r, d;
  BipartiteMatching(int a, int b)
    : n(a), m(b), g(a), l(a,-1), r(b,-1),
      d(a+1) {}
  void add(int u, int v) { g[u].pb(v); }
  bool bfs() {
    fill(d.begin(), d.end(), n+1);
    queue<int> q;
    rep(u,0,n) if (l[u] < 0)
      d[u] = 0, q.push(u);
    while (!q.empty()) {
      int u = q.front(); q.pop();
      if (d[u] >= d[n]) continue;
      for (int v : g[u]) {
        int w = r[v] < 0 ? n : r[v];
        if (d[w] == n+1) {
          d[w] = d[u]+1;
          if (w < n) q.push(w);
        }
      }
    }
    return d[n] <= n;
  }
  bool dfs(int u) {
    if (u == n) return true;
    for (int v : g[u]) {
      int w = r[v] < 0 ? n : r[v];
      if (d[w] == d[u]+1 && dfs(w)) {
        l[u] = v; r[v] = u; return true;
      }
    }
    d[u] = n+1;
    return false;
  }
  int solve() {
    fill(l.begin(), l.end(), -1);
    fill(r.begin(), r.end(), -1);
    int ans = 0;
    while (bfs()) rep(u,0,n)
      if (l[u] < 0 && dfs(u)) ++ans;
    return ans;
  }
  pair<vec<int>,vec<int>> cover() const {
    vec<int> a(n), b(m);
    queue<int> q;
    rep(u,0,n) if (l[u] < 0)
      a[u] = 1, q.push(u);
    while (!q.empty()) {
      int u = q.front(); q.pop();
      for (int v : g[u])
        if (l[u] != v && !b[v]) {
          b[v] = 1;
          int w = r[v];
          if (w >= 0 && !a[w])
            a[w] = 1, q.push(w);
        }
    }
    pair<vec<int>,vec<int>> ans;
    rep(u,0,n)
      if (!a[u]) ans.first.pb(u);
    rep(v,0,m)
      if (b[v]) ans.second.pb(v);
    return ans;
  }
};
```

WeightedMatching.h – Minimum-cost assignment of every left vertex

cost[n][m] is a complete bipartite cost matrix, n<=m. Returns (minimumTotalCost,match), with left u assigned to right match[u]; right vertices are used at most once. Negative costs allowed; negate for maximum cost. Missing edges need a sufficiently large finite penalty and a final check that none was chosen; use MinCostFlow for sparse constraints or partial assignments. Does not modify cost. dist/pot maintain reduced costs; rev is the reverse matching. Every intermediate difference/sum must fit ll. Empty left side returns cost 0 and an empty matching. Time $O(n^2m)$, extra space $O(n + m)$.

Usage: `vec<vec<ll>> cost{{4,1,9},{2,8,3}};`
`auto [total,match]=weightedMatching(cost);`
`// total=3, match={1,0}`

```
pair<ll, vec<int>> weightedMatching(const vec<vec<ll>>& C) {
  int i = sz(C), m = i ? sz(C[0]) : 0, c, s, r;
  assert(i<=m);
  for (auto& row : C) assert(sz(row)==m);
  vec<ll> dist(m), pot(m);
  vec<int> match(i), rev(m, -1), cols(m), prev(m);
  ll d = 0, nd, cost = 0;
  while (i--) {
    rep(c,0,m) dist[c] = C[i][c], cols[c] = c, prev[c] = i;
    for (s = 0;;) {
      rep(j,s,m) {
        c = cols[j], nd = dist[c] - pot[c];
        if (j == s || d > nd) d = nd, swap(cols[s], cols[j]);
      }
      if ((r = rev[c = cols[s++]]) == -1) break;
      rep(j,0,m) if (dist[j] > (nd = C[r][j]-C[r][c]+dist[c])
      )
        dist[j] = nd, prev[j] = r;
    }
    cost += dist[c];
    while (s--) pot[cols[s]] = dist[cols[s]] - d;
    for (; r != i; swap(c, match[r]))
      r = rev[c] = prev[c];
  }
  return {cost, match};
}
```

StableMatching.h – Stable pairing under two-sided preferences

a[u] lists acceptable receivers in decreasing preference; b[v] lists acceptable proposers. Sides may differ in size; lists may be incomplete, but must contain distinct valid IDs and no ties. A pair is acceptable only if both list each other. Returns each proposer's receiver, -1 if unmatched. Gale–Shapley is proposer-optimal among stable matchings: no mutually acceptable pair prefer each other to their current partner (being unmatched is worst). Stability is not maximum total score. rank[v][u] is receiver v's preference index; next[u] is its next untried proposal. Time $O(nm + |a| + |b| + n + m)$ and space $O(nm + n + m)$; inputs unchanged.

Usage: `vec<vec<int>> a{{0,1},{0}}, b{{1,0},{0}};`
`auto partner=stableMatching(a,b); // {1,0}`

```
vec<int> stableMatching(const vec<vec<int>>& a,
    const vec<vec<int>>& b) {
  int n=sz(a), m=sz(b);
  vec<vec<int>> rank(m,vec<int>(n,n));
  rep(v,0,m) rep(i,0,sz(b[v])) {
    int u=b[v][i];
    assert(0<=u && u<n && rank[v][u]==n);
    rank[v][u]=i;
  }
  vec<int> l(n,-1), r(m,-1), next(n), free(n);
  iota(all(free),0);
  while (!free.empty()) {
    int u=free.back(); free.pop_back();
    while (next[u]<sz(a[u])) {
      int v=a[u][next[u]++];
      assert(0<=v && v<m);
      int w=r[v];
      if (rank[v][u]==n) continue;
      if (w<0 || rank[v][u]<rank[v][w]) {
        l[u]=v; r[v]=u;
        if (w>=0) l[w]=-1, free.pb(w);
        break;
      }
    }
  }
  return l;
}
```

7.5 Trees and repeated successors

JumpTable.h – Repeated successors and k-th ancestors

next[v] is the successor, -1 if absent; it may contain cycles. jmp[k][v] stores the successor after 2^k steps. jump(v,k) returns the destination after k steps, or -1 after leaving the graph. Choose $1 \le lg \le 64$ and $steps < 2^{lg}$; lg=64 supports all unsigned 64-bit step counts. For a tree pass parents with parent[root]=-1. Build/space $O(V\,lg)$, each jump $O(lg)$.

Usage: `JumpTable t(vec<int>{1,2,0},64);`
`int v=t.jump(0,5); // 2`

```
using ull = unsigned long long;

struct JumpTable {
  vec<vec<int>> jmp;
  JumpTable(const vec<int>& next, int lg)
    : jmp(lg,next) {
    assert(1 <= lg && lg <= 64);
    for (int p : next) assert(-1<=p && p<sz(next));
    rep(j,1,lg)
      rep(v,0,sz(next)) {
        int p = jmp[j-1][v];
        jmp[j][v] = p < 0 ? -1 : jmp[j-1][p];
      }
  }
  int jump(int v, ull k) const {
    assert(-1<=v && v<sz(jmp[0]));
    assert(sz(jmp) == 64 || (k >> sz(jmp)) == 0);
    for (int j = 0; k && v >= 0; ++j, k >>= 1)
      if (k & 1) v = jmp[j][v];
    return v;
  }
};
```

TreeTour.h – Flatten rooted subtrees into array intervals

g is an undirected connected tree; root selects its root. Empty input allowed. par[root]=-1, dep[v] counts edges from root. ord[i] is the vertex at position i; v's subtree is exactly ord[tin[v]..tout[v]), including v. Store a vertex value at position tin[v] for Fenwick/segment-tree queries. Reverse ord processes children before parents. Iterative build, time/space $O(V)$.

Usage: `vec<vec<int>> g{{1},{0,2,3},{1},{1}};`
`TreeTour t(g);`
`// subtree(1): [t.tin[1],t.tout[1]) = [1,4)`

```
struct TreeTour {
  vec<int> par, dep, tin, tout, ord;
  TreeTour(const vec<vec<int>>& g, int root = 0)
    : par(sz(g),-1), dep(sz(g)),
      tin(sz(g)), tout(sz(g)) {
    if (g.empty()) return;
    vec<int> st{root}, size(sz(g),1);
    while (!st.empty()) {
      int v = st.back(); st.pop_back();
      tin[v] = sz(ord); ord.pb(v);
      for (int u : g[v]) if (u != par[v]) {
        par[u] = v; dep[u] = dep[v]+1;
        st.pb(u);
      }
    }
    for (int i = sz(ord); i--;) {
      int v = ord[i];
      tout[v] = tin[v]+size[v];
      if (par[v] >= 0)
        size[par[v]] += size[v];
    }
  }
};
```

TreeLCA.h – Lowest common ancestor and tree distance

Undirected connected tree g, chosen root. lca(a,b) returns the deepest common ancestor; dist(a,b) counts edges. ancestor(v,k) returns the k-th parent or -1 if absent. Inherits par/dep/tin/tout/ord from TreeTour; jt.jmp uses [level][vertex]. Build can be empty; queries require valid vertices. For weighted distances separately precompute root-distance D: D[a]+D[b]-2*D[lca(a,b)]. Build/space $O(V \log V)$; queries $O(\log V)$.

Usage: `vec<vec<int>> g{{1},{0,2,3},{1},{1}};`
`TreeLCA t(g);`
`int c=t.lca(2,3); // 1`
`int a=t.ancestor(2,3); // -1`
`"TreeTour.h", "JumpTable.h"`

```
struct TreeLCA : TreeTour {
  JumpTable jt;
  static int levels(int n) {
    int d = 1;
    for (ll on = 1; on < n; on *= 2) ++d;
    return d;
  }
  TreeLCA(const vec<vec<int>>& g, int root = 0)
    : TreeTour(g,root), jt(par,levels(sz(g))) {}
  int ancestor(int v, ull k) const {
    return k > ull(dep[v]) ? -1 : jt.jump(v,k);
  }
  int lca(int a, int b) const {
    if (dep[a] < dep[b]) swap(a,b);
    a = jt.jump(a,ull(dep[a]-dep[b]));
    if (a == b) return a;
    for (int i = sz(jt.jmp); i--;) {
      int c = jt.jmp[i][a], d = jt.jmp[i][b];
      if (c != d) a = c, b = d;
    }
    return jt.jmp[0][a];
  }
  int dist(int a, int b) const {
    int c = lca(a,b);
    return (dep[a]-dep[c])+(dep[b]-dep[c]);
  }
};
```

HeavyLight.h – Tree path queries via array intervals

Undirected connected tree g; root chosen as in TreeTour. Inherits par/dep/tin/tout/ord, reordered so heavy paths are consecutive; subtree(v) remains [tin[v],tout[v]). The example uses SegTree (SegmentTree.h), storing vertex values at tin[v]. path(u,v,f) calls f(l,r) for disjoint pieces covering the path; combine queries or apply updates in f. With edges=true, exclude the LCA and store each edge at its child's position. Segment order is unspecified: use only commutative aggregates unless adding direction handling. Build/space $O(V)$; $O(\log V)$ callbacks, hence typically $O(\log^2 V)$ per query/update.

Usage: `HeavyLight h(g,0);`
`ll ans=SegTree::unit;`
`h.path(u,v,[&](int l,int r){ans=max(ans,seg.query(l,r));});`
`"TreeTour.h"`

```
struct HeavyLight : TreeTour {
  vec<int> head;
  HeavyLight(const vec<vec<int>>& g, int root = 0)
    : TreeTour(g,root), head(sz(g)) {
    if (g.empty()) return;
    vec<int> heavy(sz(g),-1);
    rep(v,0,sz(g))
      for (int u : g[v]) if (par[u] == v) {
        int w = heavy[v];
        if (w < 0 ||
            tout[u]-tin[u] > tout[w]-tin[w])
          heavy[v] = u;
      }
    vec<int> st{root}; ord.clear();
    while (!st.empty()) {
      int v = st.back(); st.pop_back();
      for (int u = v; u >= 0; u = heavy[u]) {
        head[u] = v;
        tin[u] = sz(ord); ord.pb(u);
        tout[u] = tin[u]+1;
        for (int w : g[u])
          if (par[w] == u && w != heavy[u])
            st.pb(w);
      }
    }
    for (int i = sz(ord); i--;) {
      int v = ord[i];
      if (par[v] >= 0)
        tout[par[v]] = max(tout[par[v]],tout[v]);
    }
  }
  template<class F>
  void path(int u, int v, F f,
      bool edges = false) const {
    while (head[u] != head[v]) {
      if (dep[head[u]] < dep[head[v]]) swap(u,v);
      f(tin[head[u]],tin[u]+1);
      u = par[head[u]];
    }
    if (dep[u] > dep[v]) swap(u,v);
    int l = tin[u]+int(edges), r = tin[v]+1;
    if (l < r) f(l,r);
  }
};
```

TreeDiameter.h – Farthest pair in a nonnegative weighted tree

g[u] contains (to,weight), both directions of each tree edge. All weights must be nonnegative; use weight 1 if unweighted. Returns (length,endpointA,endpointB); empty tree returns (0,-1,-1), endpoints may coincide. Two iterative traversals find a farthest vertex, then a farthest vertex from it. T is inferred from weights and must hold every path sum; prefer ll. Time/space $O(V)$.

Usage: `vec<vec<pair<int,ll>>> g{{{1,4}},{{0,4},{2,2}},{{1,2}}};`
`auto [length,a,b]=treeDiameter(g); // length=6`

```
template<class T>
tuple<T,int,int> treeDiameter(
    const vec<vec<pair<int,T>>>& g) {
  int n = sz(g);
  if (!n) return {T{},-1,-1};
  auto far = [&](int s) {
    vec<T> d(n);
    vec<int> par(n,-1), q{s};
    int best = s;
    rep(i,0,sz(q)) {
      int v = q[i];
      if (d[v] > d[best]) best = v;
      for (auto [u,w] : g[v]) if (u != par[v]) {
        par[u] = v; d[u] = d[v]+w;
        q.pb(u);
      }
    }
    return pair{best,d[best]};
  };
  int a = far(0).first;
  auto [b,d] = far(a);
  return {d,a,b};
}
```

CompressTree.h – Virtual tree connecting a small subset of tree vertices

Preprocess the original tree with TreeLCA. compressTree(t,subset) adds LCAs and compresses paths with no selected branching point. Returns (parentIndex,originalVertex) for at most 2*|subset|-1 vertices in DFS order; parentIndex=-1 at the root, otherwise indexes this returned vector. Empty subset returns empty; duplicate IDs are ignored. A virtual edge stands for a whole original path: its unweighted length is dep[child]-dep[parent], or subtract weighted root-distances. Useful when many queries mention only a few vertices. Time $O(k \log k + k \log V)$, extra space $O(k)$, after LCA preprocessing.

Usage: `TreeLCA t(g);`
`auto vt=compressTree(t,{u,v});`
`// vt[i].first is an index, vt[i].second an original ID.`
`"TreeLCA.h"`

```
vec<pair<int,int>> compressTree(const TreeLCA& t,
    vec<int> subset) {
  if (subset.empty()) return {};
  auto cmp=[&](int a,int b){return t.tin[a]<t.tin[b];};
  sort(all(subset),cmp);
  subset.erase(unique(all(subset)),subset.end());
  int k=sz(subset);
  rep(i,1,k) subset.pb(t.lca(subset[i-1],subset[i]));
  sort(all(subset),cmp);
  subset.erase(unique(all(subset)),subset.end());
  vec<pair<int,int>> ans;
  vec<int> st;
  for (int v : subset) {
    while (!st.empty() && t.tout[ans[st.back()].second]<=t.
    tin[v])
      st.pop_back();
    int p=st.empty() ? -1 : st.back();
    st.pb(sz(ans)); ans.pb({p,v});
  }
  return ans;
}
```

CentroidDecomposition.h – Balanced tree decomposition for distance-to-active queries

Input is an undirected connected unweighted tree. parent[c] is c's parent in the centroid tree, -1 at its root. path[v] lists (centroid,distance) for all centroid ancestors of original vertex v, including v itself. Each list has O(log V) entries. Example: initialise best[c]=INF; activating v applies best[c]=min(best[c],d) for (c,d) in path[v]. Nearest active distance from u is min(best[c]+d) over path[u]. This recipe supports activations only; deletions need multisets or another container. Iterative component traversals avoid linear recursion depth. Build/space $O(V \log V)$; each activation/query $O(\log V)$.

Usage: `CentroidDecomposition cd(g);`
`vec<int> best(sz(g),INT_MAX/2);`
`for (auto [c,d]:cd.path[v]) best[c]=min(best[c],d);`
`int ans=INT_MAX/2;`
`for (auto [c,d]:cd.path[u]) ans=min(ans,best[c]+d);`

```
struct CentroidDecomposition {
  vec<int> parent;
  vec<vec<pair<int,int>>> path;
  CentroidDecomposition(const vec<vec<int>>& g)
    : parent(sz(g),-1), path(sz(g)) {
    int n=sz(g);
    if (!n) return;
    vec<int> used(n), par(n,-1), sub(n);
    vec<pair<int,int>> tasks{{0,-1}};
    while (!tasks.empty()) {
      auto [s,p]=tasks.back(); tasks.pop_back();
      vec<int> ord{s}; par[s]=-1;
      rep(i,0,sz(ord)) {
        int v=ord[i]; sub[v]=1;
        for (int u:g[v]) if (!used[u] && u!=par[v])
          par[u]=v, ord.pb(u);
      }
      for (int i=sz(ord);i--;) {
        int v=ord[i];
        if (par[v]>=0) sub[par[v]]+=sub[v];
      }
      int c=-1;
      for (int v:ord) {
        int largest=sz(ord)-sub[v];
        for (int u:g[v]) if (!used[u] && par[u]==v)
          largest=max(largest,sub[u]);
        if (largest<=sz(ord)/2) {c=v; break;}
      }
      parent[c]=p;
      vec<tuple<int,int,int>> walk{{c,-1,0}};
      while (!walk.empty()) {
        auto [v,pv,d]=walk.back(); walk.pop_back();
        path[v].pb({c,d});
        for (int u:g[v]) if (!used[u] && u!=pv)
          walk.pb({u,v,d+1});
      }
      used[c]=1;
      for (int u:g[c]) if (!used[u]) tasks.pb({u,c});
    }
  }
};
```

- Tree DP: reverse TreeTour::ord processes children first. Rerooting adds the parent's contribution; exclude a child's own contribution before passing information to it. Prefix/suffix combinations avoid requiring an inverse; associative constant-time merges give $O(V)$ total time.

7.6 Graph counting and degree sequences

- Matrix–Tree theorem: start with a zero $V \times V$ matrix L. For each arc $a \to b$, do $L[a][b]-=1$, $L[b][b]+=1$; for an undirected edge add both arcs. Delete row/column r: the determinant counts spanning out-arborescences rooted at r (all vertices reachable from r). For undirected graphs any root gives the same number of spanning trees. Use `detMod` from IntDeterminant.h on that minor for the answer modulo any positive modulus.
- Erdős–Gallai: nonnegative sorted degrees $d_1 \geq \cdots \geq d_n$ form a simple undirected graph iff their sum is even and, for every $1 \leq k \leq n$, $\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k)$. These are degrees only; it does not construct the graph.

Geometry (8)

8.1 Conventions and formulas

- Prefer integer predicates when the input is integral; use `toReal` to obtain `P2` for real constructions. Point differences, products and sums must fit T. Prefer ll to int for integer predicates. Finite real inputs/intermediates; nearly parallel lines or collinear triples can amplify rounding errors.
- No universal eps: choose by scale and by quantity (length, squared length, area). Keep sort comparisons exact. Angles in radians; $\pi = \text{acos}(-1)$, degrees $\times \pi/180$.
- Triangle sides a, b, c, opposite angles A, B, C: $a^2 = b^2 + c^2 - 2bc\cos A$; $a/\sin A = b/\sin B = c/\sin C = 2R$, R the circumradius of a nondegenerate triangle.
- Area $\Delta = bc\sin A/2 = \sqrt{s(s-a)(s-b)(s-c)}$, $s = (a+b+c)/2$; require valid triangle sides. Circle arc length $r\theta$, sector area $r^2\theta/2$.
- Clamp acos arguments to $[-1, 1]$ and tiny negative radicands to 0 only when caused by rounding, not to hide invalid input.

8.2 Points, lines and segments

Point.h – 2D points and vectors

- Point<T>(x,y) stores a point or vector; use ll for integer predicates and long double for real constructions. Arithmetic must fit T. Addition/subtraction are coordinate-wise; multiplication/division use a scalar.
- dot(p) is the scalar product; cross(p) the signed determinant. a.cross(b,c) is twice the oriented triangle area, positive counterclockwise. dist2() is squared length; (a-b).dist() is Euclidean distance.
- dist/angle return double. unit/normal require nonzero real vectors; perp() rotates 90 degrees; rotate(angle) uses radians. Sorting compares (x,y) exactly, without epsilon. All operations are $O(1)$.

Usage: `Point<ll> a(1,2), b(4,6);`
`ll d2 = (b-a).dist2(); // 25`

```
template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T x=0, T y=0) : x(x), y(y) {}
  bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y);
   }
  bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y);
   }
  P operator+(P p) const { return P(x+p.x, y+p.y); }
  P operator-(P p) const { return P(x-p.x, y-p.y); }
  P operator*(T d) const { return P(x*d, y*d); }
  P operator/(T d) const { return P(x/d, y/d); }
  T dot(P p) const { return x*p.x + y*p.y; }
  T cross(P p) const { return x*p.y - y*p.x; }
  T cross(P a, P b) const { return (a-*this).cross(b-*this);
  }
  T dist2() const { return x*x + y*y; }
  double dist() const { return sqrt((double)dist2()); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2(y, x); }
  P unit() const { return *this/dist(); } // makes dist()=1
  P perp() const { return P(-y, x); } // rotates +90 degrees
  P normal() const { return perp().unit(); }
  // returns point rotated 'a' radians ccw around the origin
  P rotate(double a) const {
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
  friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << "," << p.y << ")"; }
```

```
};
```

Predicates.h – Orientation and segment membership

- side: -1 right, 0 collinear, 1 left. onSeg includes endpoints, including a=b. toReal converts to Point<long double>.
- $O(1)$; all arithmetic must fit T. eps ≥ 0 is in squared-length units; default 0. Keep sorting exact; real predicates may round.

Usage: `int s = side(a,b,p); // side of line a->b`
`bool hit = onSeg(a,b,p); // segment [a,b]`
`P2 q = toReal(p); // convert coordinates`
"Point.h"

```
using ld = long double;
using P2 = Point<ld>;

template<class T> P2 toReal(Point<T> p) {
  return P2(ld(p.x),ld(p.y));
}
template<class T> int geoSgn(T x, T eps=T{}) {
  return (x > eps)-(x < -eps);
}
template<class T>
int side(Point<T> a, Point<T> b, Point<T> p,
    T eps=0) {
  return geoSgn(a.cross(b,p),eps);
}
template<class T>
bool onSeg(Point<T> a, Point<T> b, Point<T> p,
    T eps=0) {
  return !side(a,b,p,eps) &&
    (p-a).dot(p-b) <= eps;
}
```

lineIntersection.h – Intersection of two infinite lines

- Lines through distinct endpoint pairs. Returns {1,point}, {0,(0,0)} if parallel, or {-1,(0,0)} if coincident. $O(1)$.
- Real points for fractional intersections; integer division truncates. Products of three coordinates must fit the type. Large coordinates can cancel in the numerator; nearly parallel real lines amplify rounding errors.

Usage: `auto [status,p] = lineInter(a,b,c,d);`
`// a,b and c,d define the lines; use P2 for fractional points`
"Point.h"

```
template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
  auto d = (e1 - s1).cross(e2 - s2);
  if (d == 0) // if parallel
    return {-(s1.cross(e1, s2) == 0), P(0, 0)};
  auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
  return {1, (s1 * p + e1 * q) / d};
}
```

LineSegInter.h – Intersection of two closed segments

- Segment intersection: 0/1 points, or 2 endpoints of overlap, lexicographically sorted. Degenerate segments, touching and overlap valid.
- $O(1)$ time/space; integer predicates exact within Point bounds. Output is approximate P2; real predicates compare with zero.

Usage: `auto pts = segInter(a,b,c,d);`
`// endpoints a,b and c,d; intersection points`
"Predicates.h", "lineIntersection.h"

```
template<class T>
vec<P2> segInter(Point<T> a, Point<T> b,
    Point<T> c, Point<T> d) {
  if (side(a,b,c)*side(a,b,d) < 0 &&
      side(c,d,a)*side(c,d,b) < 0) {
    P2 A=toReal(a), B=toReal(b);
    P2 C=toReal(c), D=toReal(d);
    if ((B-A).dist2() > (D-C).dist2()) {
      swap(A,C); swap(B,D);
    }
    auto hit = lineInter(P2(),B-A,C-A,D-A);
    return {A+hit.second};
  }
  vec<Point<T>> q;
  for (auto p : {a,b})
    if (onSeg(c,d,p)) q.pb(p);
  for (auto p : {c,d})
    if (onSeg(a,b,p)) q.pb(p);
  sort(q.begin(),q.end());
  q.erase(unique(q.begin(),q.end()),q.end());
  vec<P2> ans;
  for (auto p : q) ans.pb(toReal(p));
  return ans;
}
```

Projection2.h – Projections and distances

- Project/reflect p on line ab; distances from p to line/segment ab are nonnegative. Lines require $a \neq b$; segments allow $a = b$.
- $O(1)$ time/space. Real arithmetic; intermediate values must be finite.

Usage: `P2 q = project(a,b,p), r = reflect(a,b,p);`
`ld d = lineDist(a,b,p), e = segDist(a,b,p);`
`// a,b define line/segment; p is query point`
"Predicates.h"

```
P2 project(P2 a, P2 b, P2 p) {
  P2 u = b-a;
  assert(u.dist2() > 0);
  return a+u*((p-a).dot(u)/u.dist2());
}
P2 reflect(P2 a, P2 b, P2 p) {
  return project(a,b,p)*2-p;
}
ld lineDist(P2 a, P2 b, P2 p) {
  assert(!(a == b));
  return abs((b-a).cross(p-a))/
    sqrtl((b-a).dist2());
}
ld segDist(P2 a, P2 b, P2 p) {
  if (a == b) return sqrtl((p-a).dist2());
  P2 u = b-a;
  ld t = clamp((p-a).dot(u)/u.dist2(),
    ld(0),ld(1));
  return sqrtl((p-(a+u*t)).dist2());
}
```

8.3 Polygons

PolyArea.h – Twice the signed polygon area

- Twice signed area; positive for CCW. Empty/point/segment gives 0; divide by 2.
- $O(n)$ time, $O(1)$ extra space. Simple polygon; products and sum must fit T.

Usage: `auto twice = polyArea2(p);`
`// p: polygon vertices in boundary order`
"Predicates.h"

```
template<class T>
T polyArea2(
    const vec<Point<T>>& p) {
  T ans = 0;
  rep(i,0,sz(p))
    ans += p[i].cross(p[(i+1)%sz(p)]);
  return ans;
}
```

PolygonCentroid.h – Centre of mass of a uniform polygon

p contains simple-polygon vertices in boundary order, clockwise or counterclockwise. Returns the area centroid; nullopt for fewer than 3 vertices or zero signed area. Finite real arithmetic; nearly zero area is ill-conditioned.

Usage: `auto centre = polygonCentroid(p);`
Memory: $\mathcal{O}(1)$ extra
Time: $\mathcal{O}(n)$
"Predicates.h"

```
optional<P2> polygonCentroid(const vec<P2>& p) {
    if (sz(p) < 3) return nullopt;
    P2 sum(0,0);
    ld area2 = 0;
    rep(i,0,sz(p)) {
        P2 a = p[i], b = p[(i+1)%sz(p)];
        ld cross = a.cross(b);
        area2 += cross;
        sum = sum+(a+b)*cross;
    }
    if (area2 == 0) return nullopt;
    return sum/(3*area2);
}
```

PointInPoly.h – Point location in a simple polygon

- -1 outside, 0 boundary, 1 inside. Simple polygon, either orientation; n=0/1/2.
- $O(n)$ time, $O(1)$ space. Exact signs; follow Point's bounds; reals may round.

Usage: `int where = pointInPoly(p,q);`
`// p: boundary-order vertices; q: query point`
"Predicates.h"

```
template<class T> int pointInPoly(
    const vec<Point<T>>& p, Point<T> q) {
  bool in = false;
  rep(i,0,sz(p)) {
    auto a = p[i], b = p[(i+1)%sz(p)];
    if (onSeg(a,b,q)) return 0;
    if ((a.y > q.y) != (b.y > q.y))
      if ((a.cross(b,q) > 0) == (b.y > a.y))
        in = !in;
  }
  return in ? 1 : -1;
}
```

Hull.h – Convex hull in counterclockwise order

- Monotone chain; unique vertices in CCW order, no collinear interiors/closing copy. Collinear input gives endpoints; n=0/1 valid.
- $O(n \log n)$ time, $O(n)$ space. Exact signs; follow Point's bounds. Reals may round near degeneracies.

Usage: `auto h = hull(p); // p: unordered input points`
"Predicates.h"

```
template<class T>
vec<Point<T>> hull(vec<Point<T>> p) {
  sort(p.begin(), p.end());
  p.erase(unique(p.begin(),p.end()),p.end());
  int n = sz(p), k = 0;
  if (n < 2) return p;
  vec<Point<T>> h(2*n);
  for (auto q : p) {
    while (k >= 2 &&
      h[k-2].cross(h[k-1],q) <= 0) --k;
    h[k++] = q;
  }
  for (int i = n-2, t = k+1; i >= 0; --i) {
    while (k >= t &&
      h[k-2].cross(h[k-1],p[i]) <= 0) --k;
    h[k++] = p[i];
  }
  h.resize(k-1);
  return h;
}
```

PointInConvex.h – Point location in a convex polygon

- -1 outside, 0 boundary, 1 inside. Strictly convex CCW hull; n=0/1/2 allowed.
- $O(\log(n+1))$ time, $O(1)$ space. Exact signs; follow Point's bounds. Reals may round near degeneracies.

Usage: `int where = pointInConvex(h,q);`
`// h: CCW convex hull; q: query point`
"Predicates.h"

```
template<class T> int pointInConvex(
    const vec<Point<T>>& h, Point<T> q) {
  int n = sz(h);
```

```
  if (n < 3)
    return n && onSeg(h[0],h.back(),q) ? 0 : -1;
  auto a = h[0].cross(h[1],q);
  auto b = h[0].cross(h.back(),q);
  if (a < 0 || b > 0) return -1;
  if (a == 0) return onSeg(h[0],h[1],q) ? 0 : -1;
  if (b == 0)
    return onSeg(h[0],h.back(),q) ? 0 : -1;
  int l = 1, r = n-1;
  while (r-l > 1) {
    int m = (l+r)/2;
    if (h[0].cross(h[m],q) >= 0) l = m;
    else r = m;
  }
  return side(h[l],h[r],q);
}
```

HullDiameter2.h – Farthest pair on a convex hull

- Maximum squared distance by rotating calipers. Strictly convex CCW hull, no closing copy; n=0/1 gives 0, n=2 supported.
- $O(n)$ time, $O(1)$ extra space. Exact signs; follow Point's bounds. Reals may round near degeneracies.

Usage: `auto d2 = hullDiameter2(h);`

```
// h:  CCW convex hull; d2:  squared diameter
"Predicates.h"
template<class T>
T hullDiameter2(
    const vec<Point<T>>& h) {
  int n = sz(h), j = 1;
  if (n < 2) return 0;
  if (n == 2) return (h[0]-h[1]).dist2();
  T ans = 0;
  rep(i,0,n) {
    int k = (i+1)%n;
    while (h[i].cross(h[k],h[(j+1)%n]) >
        h[i].cross(h[k],h[j])) j = (j+1)%n;
    ans = max(ans,(h[i]-h[j]).dist2());
    ans = max(ans,(h[i]-h[(j+1)%n]).dist2());
  }
  return ans;
}
```

ConvexCut.h – Clip a polygon by a half-plane

- Keep the LEFT of $a \to b$, inclusive. Convex polygon; preserves vertex order. Empty/point/segment inputs/outputs allowed.
- $a \neq b$; no repeated closing vertex. Exact signs (eps=0); real arithmetic rounds. $O(n)$ time and extra space.

Usage: `auto q = convexCut(p,a,b);`

```
// p:  polygon vertices; q:  part left of a->b
"Predicates.h"
vec<P2> convexCut(const vec<P2>& p,
    P2 a, P2 b) {
  assert(!(a == b));
  vec<P2> out;
  auto add = [&](P2 q) {
    if (out.empty() || !(out.back() == q))
      out.pb(q);
  };
  int n = sz(p);
  rep(i,0,n) {
    P2 u = p[i], v = p[(i+1)%n];
    ld s = a.cross(b,u), t = a.cross(b,v);
    if (s >= 0) add(u);
    if (n == 2 && i == 1) continue;
    if ((s < 0 && t > 0) || (s > 0 && t < 0))
      add(u+(v-u)*(s/(s-t)));
  }
  if (sz(out) > 1 && out[0] == out.back())
    out.pop_back();
  return out;
}
```

- After clipping, remove collinear vertices with hull before using queries that require a strictly convex polygon.

8.4 Circles

Circle2.h – Circle intersections and circumcircle

- Circle–line/circle intersections, $O(1)$. 0/1/2 points; nullopt treats the circles as coincident within eps (infinite case). Radius zero works.
- Finite data, radius ≥ 0, distinct line endpoints; eps ≥ 0 is absolute distance tolerance. Scale must suit long double.
- Circumcircle returns nullopt if collinear; near-collinearity is ill-conditioned. $O(1)$.

Usage: `Circle c{center,radius};`

```
auto p = circleLine(c,a,b); // line a->b
auto q = circleCircle(c,d); // d:  other circle
auto e = circumcircle(a,b,p0); // three points
"Predicates.h"
struct Circle { P2 c; ld r; };

vec<P2> circleLine(Circle c, P2 a, P2 b,
    ld eps = 1e-12L) {
  P2 u = (b-a)/sqrtl((b-a).dist2());
  P2 v = u.perp();
  ld h = (a-c.c).dot(v), d = abs(h);
  if (d > c.r+eps) return {};
  P2 p = c.c+v*h;
  if (d >= c.r-eps) return {p};
  P2 q = u*sqrt(max(ld(0),(c.r-d)*(c.r+d)));
  return {p-q,p+q};
}

optional<vec<P2>> circleCircle(Circle a,
    Circle b, ld eps = 1e-12L) {
  P2 v = b.c-a.c;
  ld d = sqrtl(v.dist2()), sum = a.r+b.r;
  ld diff = abs(a.r-b.r);
  if (a.r == 0 || b.r == 0) {
    if (abs(d-sum) > eps) return vec<P2>{};
    return vec<P2>{a.r == 0 ? a.c : b.c};
  }
  if (d <= eps) {
    if (diff <= eps) return nullopt;
    return vec<P2>{};
  }
  if (d > sum+eps || d < diff-eps)
    return vec<P2>{};
  ld x = (d+(a.r-b.r)/d*sum)/2;
  P2 u = v/d, p = a.c+u*x;
  if (d >= sum-eps || d <= diff+eps)
    return vec<P2>{p};
  ld h = sqrt(max(ld(0),(a.r-x)*(a.r+x)));
  P2 q = u.perp()*h;
  return vec<P2>{p-q,p+q};
}

optional<Circle> circumcircle(P2 a, P2 b, P2 c) {
  P2 u = b-a, v = c-a;
  ld s = max({abs(u.x),abs(u.y),
    abs(v.x),abs(v.y)});
  if (s == 0) return nullopt;
  ld d = (u.cross(v)/s)/s*2;
  if (d == 0) return nullopt;
  u = u/s; v = v/s;
  P2 w = (u.perp()*v.dist2()-
    v.perp()*u.dist2())/d*s;
  return Circle{a+w,sqrtl(w.dist2())};
}
```

CircleTangents.h – Common tangents of two circles

Returns pairs of tangency points (on a, on b). External tangents by default; inner=true asks for internal tangents. Radii are nonnegative; a zero radius represents a point. Empty means none, or coincident centres (infinite case not enumerated). At tangency the two points coincide: the line is perpendicular to b.c-a.c. Uses long double; nearly tangent inputs are sensitive to rounding.

Usage: `auto lines = circleTangents(a,b);`

Time: $\mathcal{O}(1)$

```
"Circle2.h"
vec<pair<P2,P2>> circleTangents(
      Circle a, Circle b, bool inner = false) {
    assert(a.r >= 0 && b.r >= 0);
    ld r2 = inner ? -b.r : b.r;
    P2 d = b.c-a.c;
    ld dr = a.r-r2, d2 = d.dist2();
    ld h2 = d2-dr*dr;
    if (d2 == 0 || h2 < 0) return {};
    vec<pair<P2,P2>> out;
    for (int sign : {-1,1}) {
      P2 v = (d*dr+d.perp()*(sqrtl(h2)*sign))/d2;
      out.pb({a.c+v*a.r,b.c+v*r2});
    }
    if (h2 == 0) out.pop_back();
    return out;
}
```

CirclePolygonArea.h – Intersection area of a disk and a polygon

c is the disk; p is a simple polygon in boundary order. Either orientation works; returns nonnegative area. Repeated adjacent vertices and a repeated closing vertex are allowed. Does not change p. Finite long double coordinates/radius; rounding near tangencies.

Usage: `ld area = circlePolygonArea(c,p);`

Memory: $\mathcal{O}(1)$ extra

Time: $\mathcal{O}(n)$

```
"Circle2.h"
ld circlePolygonArea(Circle c, const vec<P2>& p) {
    assert(c.r >= 0);
    if (c.r == 0 || sz(p) < 3) return 0;
    auto angle = [](P2 a,P2 b) {
      return atan2l(a.cross(b),a.dot(b));
    };
    auto sector = [&](P2 a,P2 b) {
      return angle(a,b)*c.r*c.r/2;
    };
    auto edge = [&](P2 a,P2 b) {
      P2 d = b-a;
      if (d.dist2() == 0) return ld(0);
      ld u = d.dot(a)/d.dist2();
      ld v = (a.dist2()-c.r*c.r)/d.dist2();
      ld det = u*u-v;
      if (det <= 0) return sector(a,b);
      ld l = max(ld(0),-u-sqrtl(det));
      ld r = min(ld(1),-u+sqrtl(det));
      if (l >= r) return sector(a,b);
      P2 x = a+d*l, y = a+d*r;
      return sector(a,x)+x.cross(y)/2+sector(y,b);
    };
    ld sum = 0;
    rep(i,0,sz(p)) sum += edge(p[i]-c.c,p[(i+1)%sz(p)]-c.c);
    return abs(sum);
}
```

EnclosingCircle.h – Minimum enclosing circle

- Randomized minimum enclosing circle. Empty input returns radius -1. Duplicates and collinear points are allowed.
- Boundary tolerance is absolute eps (≥ 0) plus machine roundoff by scale; minimality is approximate within that scale. Scale/conditioning must suit long double. Final radius covers every point.
- Expected $O(n)$, worst $O(n^3)$ time; $O(n)$ space, including the input copy.

Usage: `Circle c = enclosingCircle(p);`
`// p: points; c.c: center; c.r: radius`
"Circle2.h"

```
Circle enclosingCircle(vec<P2> p,
    ld eps = 1e-12L) {
  static mt19937 rng(random_device{}());
  shuffle(p.begin(),p.end(),rng);
  Circle c{P2(0,0),-1};
  auto out = [&](P2 a) {
    ld s = max({abs(a.x),abs(a.y),
      abs(c.c.x),abs(c.c.y),c.r});
    ld e = eps+
      16*numeric_limits<ld>::epsilon()*s;
    ld d = sqrtl((a-c.c).dist2());
    return c.r < 0 || d > c.r+e;
  };
  auto diameter = [](P2 a, P2 b) {
    return Circle{a+(b-a)/2,
      sqrtl((b-a).dist2())/2};
  };
  rep(i,0,sz(p)) if (out(p[i])) {
    c = {p[i],0};
    rep(j,0,i) if (out(p[j])) {
      c = diameter(p[i],p[j]);
      rep(k,0,j)
        if (out(p[k])) {
          auto q = circumcircle(p[i],p[j],p[k]);
          if (q) c = *q;
          else {
            c = diameter(p[i],p[j]);
            for (int v : {i,j}) {
              auto d = diameter(p[v],p[k]);
              if (d.r > c.r) c = d;
            }
          }
        }
    }
  }
  for (P2 a : p)
    c.r = max(c.r,sqrtl((a-c.c).dist2()));
  return c;
}
```

8.5 Point sets and area unions

NearestPair.h – Closest pair of input points

- Returns squared distance and two input IDs. Fewer than two points gives (0,(-1,-1)). Ties arbitrary; input unchanged, duplicates valid.
- Point arithmetic must fit; floating-point values must remain finite. No epsilon or sqrt.
- $O(n \log n)$ time, $O(n)$ extra space.

Usage: `auto [d2,ids] = nearestPair(p);`
`// p: input points; ids: pair of 0-based IDs`
"Predicates.h"

```
template<class T>
auto nearestPair(const vec<Point<T>>& p) {
  if (sz(p) < 2) return pair{T(0),pair{-1,-1}};
  using Item = pair<Point<T>,int>;
  vec<Item> a, tmp(sz(p));
  rep(i,0,sz(p))
    a.pb({p[i],i});
  sort(a.begin(),a.end());
  T best = (p[0]-p[1]).dist2();
  pair<int,int> ids{0,1};
  auto test = [&](const Item& u, const Item& v) {
    T d = (u.first-v.first).dist2();
    if (d < best)
      best = d, ids = {u.second,v.second};
  };
  auto byY = [](const Item& u, const Item& v) {
    return u.first.y < v.first.y;
  };
  auto go = [&](auto&& self, int l,int r)->void {
    if (r-l <= 3) {
      rep(i,l,r)
        rep(j,i+1,r)
          test(a[i],a[j]);
      sort(a.begin()+l,a.begin()+r,byY);
      return;
    }
    int m = l+(r-l)/2;
    T x = a[m].first.x;
    self(self,l,m); self(self,m,r);
    merge(a.begin()+l,a.begin()+m,
      a.begin()+m,a.begin()+r,tmp.begin()+l,byY);
    copy_n(tmp.begin()+l,r-l,a.begin()+l);
    int k = 0;
    rep(i,l,r) {
      T dx = a[i].first.x-x;
      if (dx*dx >= best) continue;
      for (int j = k; j--;) {
        T dy = a[i].first.y-tmp[j].first.y;
        if (dy*dy >= best) break;
        test(a[i],tmp[j]);
      }
      tmp[k++] = a[i];
    }
  };
  go(go,0,sz(a));
  return pair{best,ids};
}
```

ManhattanMST.h – Sparse candidate edges for a Manhattan MST

p contains integer points with $|coordinate| \leq 10^{15}$. Returns at most 4n edges (weight,u,v), using ORIGINAL 0-based IDs. These edges contain an MST of the complete graph whose weights are abs(dx)+abs(dy). Run Kruskal on them; they are not the MST itself. Copies its input; duplicates and empty input are supported.

Usage: `auto candidates = manhattanEdges(points);`
Memory: $\mathcal{O}(n)$
Time: $\mathcal{O}(nlogn)$
"Point.h"

```
vec<tuple<ll,int,int>> manhattanEdges(vec<Point<ll>> p) {
    for (auto q : p) {
        assert(-1000000000000000LL <= q.x && q.x <=
        1000000000000000LL);
        assert(-1000000000000000LL <= q.y && q.y <=
        1000000000000000LL);
    }
    vec<int> id(sz(p));
    iota(all(id),0);
    vec<tuple<ll,int,int>> edges;
    rep(k,0,4) {
        sort(all(id),[&](int i,int j) {
            return p[i].x+p[i].y < p[j].x+p[j].y;
        });
        map<ll,int> sweep;
        for (int i : id) {
            auto it = sweep.lower_bound(-p[i].y);
            while (it != sweep.end()) {
                int j = it->second;
                auto d = p[i]-p[j];
                if (d.y > d.x) break;
                edges.pb({d.x+d.y,i,j});
                it = sweep.erase(it);
            }
            sweep[-p[i].y] = i;
        }
        for (auto& q : p)
            if (k&1) q.x = -q.x;
            else swap(q.x,q.y);
    }
    return edges;
}
```

RectUnion.h – Union area of axis-aligned rectangles

- Exact union area of integer rectangles (x1,y1,x2,y2), interpreted as $[x1, x2) \times [y1, y2)$. Ordered bounds; empty rectangles are ignored.
- Lengths, products and total area must fit ll. Input unchanged; ll coordinates.
- $O(n \log n)$ time, $O(n)$ extra space.

Usage: `ll area = rectUnion(rect);`
`// rect: vec<array<ll,4>>, {x1,y1,x2,y2}`

```
ll rectUnion(const vec<array<ll,4>>& rect) {
  vec<tuple<ll,ll,ll,int>> ev;
  vec<ll> y;
  for (auto [x1,y1,x2,y2] : rect)
    if (x1 < x2 && y1 < y2) {
      ev.pb({x1,y1,y2,1});
      ev.pb({x2,y1,y2,-1});
      y.pb(y1); y.pb(y2);
    }
  if (ev.empty()) return 0;
  sort(ev.begin(),ev.end());
  sort(y.begin(),y.end());
  y.erase(unique(y.begin(),y.end()),y.end());
  int n = sz(y)-1;
  vec<int> cover(4*n);
  vec<ll> len(4*n);
  auto add = [&](auto&& self, int p,int l,int r,
      int a, int b, int d)->void {
    if (b <= l || r <= a) return;
    if (a <= l && r <= b) cover[p] += d;
    else {
      int m = l+(r-l)/2;
      self(self,2*p,l,m,a,b,d);
      self(self,2*p+1,m,r,a,b,d);
    }
    len[p] = cover[p] ? y[r]-y[l] :
      (r-l == 1 ? 0 : len[2*p]+len[2*p+1]);
  };
  ll area = 0;
  ll prev = get<0>(ev[0]);
  for (auto [x,y1,y2,d] : ev) {
    area += len[1]*(x-prev);
    int a = int(lower_bound(y.begin(),y.end(),y1)
      -y.begin());
    int b = int(lower_bound(y.begin(),y.end(),y2)
      -y.begin());
    add(add,1,0,n,a,b,d); prev = x;
  }
  return area;
}
```

8.6 Spatial grid / radius queries

- For radius $r > 0$, use square cells of side r with key $(\lfloor x/r \rfloor, \lfloor y/r \rfloor)$. For integer data, use `floorDiv` from Mathematics; truncation is wrong for negatives.
- Store point IDs per cell. Check the cell and its 8 neighbours, then filter with squared distance $\leq r^2$ in a type where the squared distances fit. To enumerate pairs once, query previous points before inserting the new point. For $r = 0$, group identical coordinates instead.
- Hash lookup is expected $O(1)$; total work includes every candidate tested. Dense cells can make it $O(n^2)$; cell indices and neighbour offsets must fit their type. This enumerates neighbours; closest pair alone does not.

Strings (9)

- Indices start at 0; intervals $[l, r)$. Strings are byte sequences, not Unicode code points. Lengths, node/pattern counts are below INT_MAX.
- Substring is contiguous; subsequence may skip positions.

9.1 Matching and hashing

KMP.h – Exact pattern occurrences and prefix borders.

- kmpMatches(text,pattern) returns all starting indices, including overlaps; an empty pattern matches all n+1 boundaries. Strings or vectors; no special separator is required.
- prefixFunction(s) returns pi[i]: length of the longest proper prefix of s[0,i+1) that is also its suffix. Borders: follow pi[n-1], pi[k-1], etc. Candidate period p=n-pi[n-1] tiles s iff n%p=0.
- Prefix function $O(m)$; matching $O(n+m)$ time, $O(m)$ plus output space.

Usage: `auto at=kmpMatches(string("ababa"),string("aba")); // {0,2}`

```
template<class S>
vec<int> prefixFunction(const S& s) {
  vec<int> p(sz(s));
  rep(i,1,sz(s)) {
    int j = p[i-1];
    while (j && s[i] != s[j]) j = p[j-1];
    if (s[i] == s[j]) ++j;
    p[i] = j;
  }
  return p;
}
template<class S, class P>
vec<int> kmpMatches(const S& s, const P& p) {
  vec<int> ans;
  int n = sz(s), m = sz(p), j = 0;
  if (!m) {
    ans.resize(n+1);
    iota(ans.begin(),ans.end(),0);
    return ans;
  }
  vec<int> pi = prefixFunction(p);
  rep(i,0,n) {
    while (j && s[i] != p[j]) j = pi[j-1];
    if (s[i] == p[j]) ++j;
    if (j == m) {
      ans.pb(i-m+1); j = pi[j-1];
    }
  }
  return ans;
}
```

ZFunction.h – Prefix-match length at every position.

- zFunction(s) accepts a string/vector and returns z[i], the common prefix length of s and s[i,n). z[0]=0; empty input gives empty output. Useful for prefix occurrences, borders and periodicity.
- The matched block [l,r) is reused to avoid comparing twice. $O(n)$ time/space.

Usage: `auto z=zFunction(string("aaaa")); // {0,3,2,1}`

```
template<class S> vec<int> zFunction(const S& s) {
  int n = sz(s), l = 0, r = 0;
  vec<int> z(n);
  rep(i,1,n) {
    if (i < r) z[i] = min(r-i,z[i-l]);
    while (z[i] < n-i && s[z[i]] == s[i+z[i]])
      ++z[i];
    if (i+z[i] > r) l = i, r = i+z[i];
  }
  return z;
}
```

DoubleHash.h – Constant-time substring fingerprints (probabilistic).

- Construct from a string or vec<int>. get(l,r) returns {hash1,hash2,length} for $[l,r)$. Different keys prove inequality; equal keys may collide. h/p are prefix hashes/powers of size n+1.
- h.concat(x,y) represents x followed by y; the length of y must not exceed the input length of h. Strings use unsigned bytes. Integer IDs must lie in $[0,10^9+7)$: jointly compress ALL compared sequences, including signed/64-bit integers, before constructing hashes.
- All objects share fixed bases: deterministic and simple, but vulnerable to deliberately constructed collisions. For exact equality, verify matches or use an exact string algorithm. Build/space $O(n)$; get/concat $O(1)$.

Usage: `DoubleHash h("abc"), t("ac");`
`bool same=h.concat(h.get(0,1),h.get(2,3))==t.get(0,2);`

```
struct DoubleHash {
  using Key = array<ll,3>; // hash1, hash2, comprimento
  static constexpr array<ll,2> mod{1000000007,1000000009};
  static constexpr array<ll,2> base{911382323,972663749};
  vec<array<ll,2>> h, p;
  explicit DoubleHash(const vec<int>& s)
    : h(sz(s)+1), p(sz(s)+1, {1,1}) {
    rep(i,0,sz(s)) {
      assert(0 <= s[i] && s[i] < mod[0]);
      rep(k,0,2) {
        h[i+1][k] = (h[i][k]*base[k]+s[i])%mod[k];
        p[i+1][k] = p[i][k]*base[k]%mod[k];
      }
    }
  }
  explicit DoubleHash(const vec<unsigned char>& s)
    : DoubleHash(vec<int>(s.begin(),s.end())) {}
  explicit DoubleHash(string_view s)
    : DoubleHash(vec<unsigned char>(s.begin(),s.end())) {}
  Key get(int l, int r) const {
    assert(0 <= l && l <= r && r < sz(h));
    Key ans{0,0,r-l};
    rep(k,0,2)
      ans[k] = (h[r][k]-h[l][k]*p[r-l][k]%mod[k]+mod[k])%mod[
      k];
    return ans;
  }
  Key concat(Key x, Key y) const {
    assert(0 <= y[2] && y[2] < sz(p));
    assert(0 <= x[2] && x[2] <= INT_MAX-y[2]);
    rep(k,0,2)
      x[k] = (x[k]*p[y[2]][k]+y[k])%mod[k];
    x[2] += y[2]; return x;
  }
};
```

9.2 Dictionary and multiple patterns

AlphabetTrie.h – Dictionary membership and prefix counts, with duplicates.

- Trie accepts lowercase a-z words, including empty. insert(s) adds one copy; count(s) counts exact copies, contains(s) tests membership, countPrefix(s) counts all inserted words starting with s.
- Root=0. walk(s) returns its node ID, or -1 if absent. node[v].next contains child IDs (-1 absent); terminal counts word endings; pass counts words passing through. countPrefix("") counts all insertions.
- Dynamic nodes; insertion may invalidate references into node. No erase. Insertions/nodes $< INT_MAX$. Calls $O(|s|+1)$ (insert amortised); space $O(26L)$ for L distinct prefixes plus root.

Usage: `Trie t; t.insert("cat"); t.insert("cat");`
`int copies=t.count("cat"); // 2`

```
struct Trie {
  struct Node {
    array<int,26> next;
    int terminal = 0, pass = 0;
    Node() { next.fill(-1); }
  };
  vec<Node> node = vec<Node>(1);
  void insert(const string& s) {
    assert(node[0].pass < INT_MAX);
    int v = 0; ++node[v].pass;
    for (char c : s) {
      int x = c-'a'; assert(0 <= x && x < 26);
      if (node[v].next[x] == -1) {
        assert(node.size() < INT_MAX);
        int id = sz(node); node.emplace_back();
        node[v].next[x] = id;
      }
      v = node[v].next[x]; ++node[v].pass;
    }
    ++node[v].terminal;
  }
  int walk(const string& s) const {
    int v = 0;
    for (char c : s) {
      int x = c-'a'; assert(0 <= x && x < 26);
      v = node[v].next[x];
      if (v == -1) break;
    }
    return v;
  }
  int count(const string& s) const {
    int v = walk(s); return v == -1 ? 0 : node[v].terminal;
  }
  bool contains(const string& s) const { return count(s) != 0
  ; }
  int countPrefix(const string& s) const {
    int v = walk(s); return v == -1 ? 0 : node[v].pass;
  }
};
```

Aho.h – Aho–Corasick: many pattern matches in one text scan.

- Use to find many patterns while reading a text once. add(word) returns its ID; add all words, then build once. **Aho(K,first):** K bytes starting at first; default a-z. Aho a(256,0) accepts every byte.
- Start v=0; v=go(v,ch) reads one byte. count(v) returns patterns ending here; report(v,emit) calls emit(id) for each one. Duplicate words have distinct IDs. Empty words match every boundary, including v=0 before scanning.
- Node next stores transitions; link falls back to a suffix, out skips to a matching suffix; ids holds pattern IDs, cnt counts matches. Never add after build.
- K letters, V nodes, P patterns: add amortized $O(K(|s|+1))$; build/space $O(KV+P)$. go/count $O(1)$; report $O(1+out)$. Output lists are not copied along links; total counts use ll.

Usage: `Aho a; a.add("aba"); a.build();`
`int v = 0; ll total = 0; // text: input string`
`for (char c : text) { v=a.go(v,c); total+=a.count(v); }`
`a.report(v,[&](int id) { cout << id << ' '; });`

```
struct Aho {
  int K, BASE;
  struct Node {
    vec<int> next;
    int link = 0, out = -1, cnt = 0;
    vec<int> ids;
    explicit Node(int k) : next(k,-1) {}
  };
  vec<Node> t;
  explicit Aho(int alphabet = 26,
    int first = 'a')
    : K(alphabet), BASE(first) {
    assert(BASE >= 0 && K > 0 && K <= 256-BASE);
    t.emplace_back(K);
  }
  int patterns = 0;
  bool built = false;
  int code(unsigned char ch) const {
    int c = int(ch)-BASE;
    assert(0 <= c && c < K); return c;
  }
  int add(string_view s) {
    assert(!built);
    int v = 0;
    for (unsigned char ch : s) {
      int c = code(ch), u = t[v].next[c];
      if (u < 0) {
        t[v].next[c] = u = sz(t);
        t.emplace_back(K);
      }
```

```
      v = u;
    }
    t[v].ids.pb(patterns);
    ++t[v].cnt;
    return patterns++;
  }
  void build() {
    assert(!built); built = true;
    queue<int> q; q.push(0);
    while (!q.empty()) {
      int v = q.front(); q.pop();
      if (v) {
        int p = t[v].link;
        t[v].out = t[p].ids.empty() ?
          t[p].out : p;
        t[v].cnt += t[p].cnt;
      }
      rep(c,0,K) {
        int u = t[v].next[c];
        int w = v ? t[t[v].link].next[c] : 0;
        if (u < 0) t[v].next[c] = w;
        else { t[u].link = w; q.push(u); }
      }
    }
  }
  int go(int v, unsigned char ch) const {
    assert(built); return t[v].next[code(ch)];
  }
  int count(int v) const {
    assert(built); return t[v].cnt;
  }
  void report(int v, const function<void(int)>& emit) const {
    assert(built);
    for (int u = v; u >= 0; u = t[u].out)
      for (int id : t[u].ids) emit(id);
  }
};
```

9.3 Suffixes, palindromes and rotations

Suffixes.h – Sorted suffixes, inverse ranks and adjacent LCP.

- Use for repeated/common substrings or many suffix comparisons. Input: string s; unsigned-byte order.
- sa[i] is the start of suffix i in sorted order; rank[start] gives its position there. lcp[i] is the common-prefix length of sa[i] and sa[i+1]. No empty suffix: sizes n, n, max(0,n-1). max(lcp) is the longest repeated length (0 if lcp is empty).
- Doubling sorts pairs of ranks, then a linear scan builds lcp. $O(n \log n)$ time, $O(n)$ extra space.

Usage: `Suffixes a(s); RMQ<int> rmq(a.lcp);`

```
// suffixes i!=j:  rmq.query(min(a.rank[i],a.rank[j]),
// max(a.rank[i],a.rank[j])); i==j gives n-i.
```

```
struct Suffixes {
  vec<int> sa, rank, lcp;
  Suffixes(string_view s) : sa(sz(s)),
    rank(sz(s)), lcp(max(0,sz(s)-1)) {
    int n = sz(s), c = 0;
    if (!n) return;
    iota(sa.begin(),sa.end(),0);
    sort(sa.begin(),sa.end(),[&](int a,int b) {
      return (unsigned char)s[a] <
        (unsigned char)s[b];
    });
    rep(i,0,n) {
      if (!i || s[sa[i]] != s[sa[i-1]]) ++c;
      rank[sa[i]] = c-1;
    }
    vec<int> y(n), r(n), cnt(n);
    for (int k = 1; c < n; k *= 2) {
      int p = 0;
      rep(i,n-k,n) y[p++] = i;
      for (int v : sa)
        if (v >= k) y[p++] = v-k;
      fill(cnt.begin(),cnt.begin()+c,0);
      for (int v : y) ++cnt[rank[v]];
      rep(i,1,c)
        cnt[i] += cnt[i-1];
      for (int i = n; i--;)
        sa[--cnt[rank[y[i]]]] = y[i];
      c = 1; r[sa[0]] = 0;
      rep(i,1,n) {
        int a = sa[i-1], b = sa[i];
        int ra = a < n-k ? rank[a+k] : -1;
        int rb = b < n-k ? rank[b+k] : -1;
        if (rank[a] != rank[b] || ra != rb) ++c;
        r[b] = c-1;
      }
      rank.swap(r);
      if (c == n) break;
    }
    for (int i = 0, h = 0; i < n; ++i) {
      int rnk = rank[i];
      if (rnk == n-1) { h = 0; continue; }
      int j = sa[rnk+1];
      while (h < n-i && h < n-j &&
        s[i+h] == s[j+h]) ++h;
      lcp[rnk] = h;
      if (h) --h;
    }
  }
};
```

- LCP of suffixes i and j: if i=j, return n-i; otherwise take the minimum of lcp between their ranks with RMQ from Data structures. An empty lcp array has no valid nonempty query; constructing RMQ on it is still valid.

Palindromes.h – Manacher: all odd/even palindrome radii.

- Use to find palindromes around every centre. Input: string/vector s. Returns {odd,even} radius arrays.
- odd[i]=k means $[i-k+1, i+k)$, length 2k-1; even[i]=k means $[i-k, i+k)$, length 2k. Thus "aba" has odd[1]=2; "aa" has even[1]=1.
- Manacher reuses the furthest known palindrome [l,r]. Empty input gives two empty arrays; $O(n)$ time/space.

Usage: `auto [odd,even] = palindromes(s);`

```
template<class S>
array<vec<int>,2> palindromes(const S& s) {
  int n = sz(s);
  vec<int> odd(n), even(n);
  for (int i = 0, l = 0, r = -1; i < n; ++i) {
    int k = i > r ? 1 : min(odd[l+(r-i)],r-i+1);
    while (k <= i && k < n-i && s[i-k] == s[i+k]) ++k;
    odd[i] = k;
    if (i+k-1 > r) { l = i-k+1; r = i+k-1; }
  }
  for (int i = 0, l = 0, r = -1; i < n; ++i) {
    int k = i > r ? 0 : min(even[l+(r-i)+1],r-i+1);
    while (k < i && k < n-i && s[i-k-1] == s[i+k]) ++k;
    even[i] = k;
    if (i+k-1 > r) { l = i-k; r = i+k-1; }
  }
  return {odd,even};
}
```

LeastRotation.h – Lexicographically smallest cyclic rotation.

- Use to give circular strings/sequences a common form. **Input:** string/vector s. Returns the first start index of the smallest rotation; "baba" gives 1, empty gives 0.
- Read the rotation as s[(start+i)%n]. i,j are candidate starts; k counts matching symbols; at wraps the index and treats char as an unsigned byte. Other types keep their order. $O(n)$ time, $O(1)$ extra space.

Usage: `int start = leastRotation(s);`

```
template<class S>
int leastRotation(const S& s) {
  int n = sz(s);
  ll i = 0, j = 1, k = 0;
  auto at = [&](ll pos) {
    auto c = s[size_t(pos%n)];
    if constexpr (is_same_v<decltype(c),char>)
      return (unsigned char)c;
    else return c;
  };
  while (i < n && j < n && k < n) {
    auto a = at(i+k), b = at(j+k);
    if (a == b) { ++k; continue; }
    if (b < a) {
      i += k+1;
      if (i <= j) i = j+1;
    } else {
      j += k+1;
      if (j <= i) j = i+1;
    }
    k = 0;
  }
  return n ? int(min(i,j)) : 0;
}
```

Sequences and DP (10)

10.1 Arrays and windows

CoordinateCompression.h – Map ordered coordinates to compact indices.

- compressCoordinates(a) returns (values,rank), without changing a. values is sorted and unique; values[rank[i]]=a[i]. Equal values get equal ranks. Compress ALL values that will be compared together.
- Preserves order, not distances: widths use values[j]-values[i], not j-i. Requires consistent < and == (no NaN). Empty input is valid. $O(n \log n)$ time, $O(n)$ space.

Usage: `auto [values,rank]=compressCoordinates(vec<int>{50,10,50}); // values={10,50}, rank={1,0,1}`

```
template<class T>
pair<vec<T>,vec<int>> compressCoordinates(const vec<T>& a) {
  vec<T> values = a;
  sort(values.begin(),values.end());
  values.erase(unique(values.begin(),values.end()),
    values.end());
  vec<int> rank;
  for (const T& x : a) {
    auto it = lower_bound(values.begin(),values.end(),x);
    rank.pb(int(it-values.begin()));
  }
  return {values,rank};
}
```

TwoPointers.h – Longest valid sliding window.

- longestKDistinct(a,k) returns the first longest $[l, r)$ containing at most k distinct values; k>=0. Frequencies use map: $O(n \log(D+1))$ time and $O(D)$ space, D the most distinct values stored.
- longestSumAtMost(a,limit) returns the first longest $[l, r)$ of sum at most limit. Requires nonnegative values, limit>=0 and total sum fitting ll. $O(n)$ time, $O(1)$ extra space. Negatives break the rule.
- Empty answer is [0,0). Expand the right endpoint; remove left values until valid. Both endpoints only move forwards.

Usage: `auto [l,r]=longestKDistinct(vec<int>{1,2,1,3},2); // [0,3)`

```
template<class T>
pair<int,int> longestKDistinct(const vec<T>& a, int k) {
  assert(k >= 0);
  map<T,int> freq;
  pair<int,int> best{0,0};
  for (int l = 0, r = 0; r < sz(a); ) {
    ++freq[a[r++]];
    while (sz(freq) > k) {
      auto it = freq.find(a[l++]);
      if (--it->second == 0) freq.erase(it);
```

```
    }
    if (r-l > best.second-best.first) best = {l,r};
  }
  return best;
}
pair<int,int> longestSumAtMost(const vec<ll>& a, ll limit) {
  assert(limit >= 0);
  ll sum = 0;
  pair<int,int> best{0,0};
  for (int l = 0, r = 0; r < sz(a); ) {
    assert(a[r] >= 0);
    sum += a[r++];
    while (sum > limit) sum -= a[l++];
    if (r-l > best.second-best.first) best = {l,r};
  }
  return best;
}
```

MonotonicStack.h – Nearest strictly smaller neighbour.

- nearestSmaller(a) returns the index of the nearest STRICTLY smaller value to the left (-1 if absent). With right=true, searches right (absent=n). Returns boundaries/indices, not values.
- Stack values increase: pop >= removes ties. To accept <= neighbours, change pop to >; reverse comparisons for greater neighbours. Counting subarray contributions with duplicates needs one strict boundary and one non-strict boundary. $O(n)$ time/space.

Usage: `auto left=nearestSmaller(vec<int>{3,1,2,2}); // {-1,-1,1,1}`

```
template<class T>
vec<int> nearestSmaller(const vec<T>& a, bool right = false)
{
  int n = sz(a);
  vec<int> ans(n,right ? n : -1), st;
  rep(t,0,n) {
    int i = right ? n-1-t : t;
    while (!st.empty() && a[st.back()] >= a[i])
      st.pop_back();
    if (!st.empty()) ans[i] = st.back();
    st.pb(i);
  }
  return ans;
}
```

MonotonicDeque.h – Minimum of every fixed-size sliding window.

- slidingMin(a,k), k>0, returns indices: ans[l] locates a minimum in $[l, l+k)$. Empty if k>n; ties choose the rightmost. Use a[ans[l]] for the value. Change >= to <= for sliding maxima.
- q stores candidates: expire old indices at the front, remove dominated ones at the back. $O(n)$ time, $O(\min(n,k))$ space plus output.

Usage: `auto at=slidingMin(vec<int>{4,2,5,1,3},3); // {1,3,3}`

```
template<class T>
vec<int> slidingMin(const vec<T>& a, int k) {
  assert(k > 0);
  deque<int> q;
  vec<int> ans;
  rep(r,0,sz(a)) {
    while (!q.empty() && q.front() <= r-k)
      q.pop_front();
    while (!q.empty() && a[q.back()] >= a[r])
      q.pop_back();
    q.pb(r);
    if (r >= k-1) ans.pb(q.front());
  }
  return ans;
}
```

10.2 Subsequences and dynamic programming

Subsequence.h – Longest increasing / nondecreasing subsequence.

- lis(a) returns chosen INDICES in increasing order. lis(a,true) allows equal values. Indices need not be adjacent; values are a[id]. Empty input gives empty output, tie-breaking arbitrary.
- With length k+1, tail[k] minimises the last value; par reconstructs predecessors. Input needs only <. $O(n \log n)$ time, $O(n)$ space.

Usage: `auto ids=lis(vec<int>{3,1,2,2,4}); // length 3`

```
template<class T> vec<int> lis(const vec<T>& a,
    bool nondecreasing = false) {
  vec<int> tail, par(sz(a),-1);
  rep(i,0,sz(a)) {
    int l = 0, r = sz(tail);
    while (l < r) {
      int m = l + (r-l)/2;
      bool extend = nondecreasing
        ? !(a[i] < a[tail[m]]) : a[tail[m]] < a[i];
      if (extend) l = m+1;
      else r = m;
    }
    if (l > 0) par[i] = tail[l-1];
    if (l == sz(tail)) tail.pb(i);
    else tail[l] = i;
  }
  vec<int> ans;
  int v = tail.empty() ? -1 : tail.back();
  for (; v >= 0; v = par[v]) ans.pb(v);
  reverse(ans.begin(),ans.end());
  return ans;
}
```

ClassicDP.h – Compact classical DP routines (values/counts, no reconstruction).

- maxSubarray(a), nonempty ll array: returns (linear,circular) maximum NONEMPTY subarray sums; $O(n)$ time, $O(1)$ space. All sums fit ll.
- knapsack01(w,v,W): nonnegative integer weights, ll values, capacity W>=0. Each item at most once; returns maximum value with weight <=W, possibly choosing nothing. Descending capacities prevent reuse. $O(nW)$ time/$O(W)$ space; values/sums fit ll.
- minCoins(coins,S): positive unlimited coins, S>=0; minimum coin count or -1 if impossible. coinWays(coins,S,mod): DISTINCT positive coins; unordered combinations modulo positive mod. Coin loop outside avoids counting permutations. $O(|coins|S)$ time, $O(S)$ space.
- lcsLength(a,b): length of longest common SUBSEQUENCE of two strings, not necessarily contiguous; $O(nm)$ time, $O(m)$ space.
- gridPaths(g,mod): rectangular grid of . / #, moves right/down from top-left to bottom-right. Returns count modulo positive mod; empty grid or blocked endpoint gives 0. $O(nm)$ time, $O(m)$ space.

Usage: `auto [linear,circular]=maxSubarray({5,-8,6}); // 6,11`
`ll best=knapsack01({2,3,4},{4,5,7},5); // 9`

```
// Array não vazio -> {melhor soma linear, melhor soma
circular}.
pair<ll,ll> maxSubarray(const vec<ll>& a) {
  assert(!a.empty());
  ll endMax = a[0], best = a[0], total = a[0];
  ll endMin = a[0], worst = a[0];
  rep(i,1,sz(a)) {
    endMax = max(a[i],endMax+a[i]);
    endMin = min(a[i],endMin+a[i]);
    best = max(best,endMax);
    worst = min(worst,endMin);
    total += a[i];
  }
  return {best,best < 0 ? best : max(best,total-worst)};
}

// Cada item uma vez; capacidade até W; pode não escolher
nada.
ll knapsack01(const vec<int>& w, const vec<ll>& v, int W) {
  assert(sz(w) == sz(v) && 0 <= W && W < INT_MAX);
  vec<ll> dp(W+1);
  rep(i,0,sz(w)) {
    assert(w[i] >= 0);
    for (int x = W; x >= w[i]; --x)
      dp[x] = max(dp[x],dp[x-w[i]]+v[i]);
  }
  return dp[W];
}

// Moedas positivas ilimitadas -> mínimo, ou -1 se impossível
.
int minCoins(const vec<int>& coins, int S) {
  assert(0 <= S && S < INT_MAX);
  for (int c : coins) assert(c > 0);
  vec<int> dp(S+1,S+1); dp[0] = 0;
  for (int s = 1; s <= S; ++s)
    for (int c : coins) if (c <= s && dp[s-c] < S+1)
      dp[s] = min(dp[s],dp[s-c]+1);
  return dp[S] == S+1 ? -1 : dp[S];
}

// Moedas positivas DISTINTAS ilimitadas -> combinações sem
ordem.
int coinWays(const vec<int>& coins, int S, int mod =
1000000007) {
  assert(0 <= S && S < INT_MAX && mod > 0);
  vec<int> dp(S+1); dp[0] = 1%mod;
  for (int c : coins) {
    assert(c > 0);
    for (int s = c; s <= S; ++s)
      dp[s] = int((dp[s]+ll(dp[s-c]))%mod);
  }
  return dp[S];
}

// Pode saltar caracteres; devolve apenas o comprimento da
LCS.
int lcsLength(const string& a, const string& b) {
  int n = sz(a), m = sz(b);
  vec<int> prev(m+1), cur(m+1);
  for (int i = 1; i <= n; ++i) {
    for (int j = 1; j <= m; ++j)
      cur[j] = a[i-1] == b[j-1] ? prev[j-1]+1
        : max(prev[j],cur[j-1]);
    swap(prev,cur);
  }
  return prev[m];
}

// Só baixo/direita; '.' livre, '#' bloqueado; devolve
caminhos.
int gridPaths(const vec<string>& g, int mod = 1000000007) {
  assert(mod > 0);
  int n = sz(g), m = n ? sz(g[0]) : 0;
  for (const auto& row : g) assert(sz(row) == m);
  if (!n || !m) return 0;
  vec<int> dp(m); dp[0] = 1%mod;
  rep(r,0,n)
    rep(c,0,m)
      if (g[r][c] == '#') dp[c] = 0;
      else if (c) dp[c] = int((dp[c]+ll(dp[c-1]))%mod);
  return dp[m-1];
}
```

SubsetBits.h – Reachable sums using each item at most once.

- subsetBits<N>(weights) returns bits for achievable sums $[0, N)$, including 0. N>0 is a compile-time bound; weights are nonnegative int/ll. Weights >=N can be ignored. Does not reconstruct items or count ways.

- For weight x, dp<<x adds x to old sums, OR keeps the option to skip. $O(n\lceil N/B\rceil)$ time, $O(\lceil N/B\rceil)$ words (B bits/word).

Usage: `auto possible=subsetBits<11>(vec<int>{3,7}); // possible[10]=1`

```
template<size_t N, class T>
bitset<N> subsetBits(const vec<T>& w) {
  static_assert(N > 0);
  bitset<N> dp;
  dp[0] = 1;
  for (T x : w) {
    assert(x >= 0);
    if (uintmax_t(x) < N) dp |= dp << size_t(x);
  }
  return dp;
}
```

SOS.h – Aggregate over all submasks or all supermasks.

- a has size $2^k > 0$ and initially stores a value for each exact mask. sos(a) replaces a[mask] by the sum over its submasks; sos(a,true) uses supermasks. Includes the mask itself; modifies a, returns void.
- Optional op must be associative and commutative (default addition); all intermediate values fit T. small/big differ in one bit. $O(k2^k)$ operations, $O(1)$ extra space.

Usage: `vec<ll> a{1,2,3,4}; sos(a); // {1,3,4,10}`

```
template<class T, class F = plus<T>>
void sos(vec<T>& a, bool supersets = false,
    F op = {}) {
  int n = sz(a);
  assert(n > 0 && !(n & (n-1)));
  for (int bit = 1; bit < n; bit <<= 1) {
    rep(mask,0,n) {
      if (mask & bit) continue;
      int small = mask, big = mask | bit;
      if (supersets) a[small] = op(a[small],a[big]);
      else a[big] = op(a[big],a[small]);
    }
  }
}
```

DigitDP.h – Digit DP: count numbers by digit-sum remainder.

- digitSumCount(bound,d,mod) counts integers in [0,bound] whose digit sum is divisible by d, modulo mod. bound is a nonempty decimal string (may be huge); d/mod are positive ints. Leading zeros and 0 itself are accepted. Subtract one modulo mod to exclude 0.
- dp[sum][smaller] counts prefixes: sum is digit sum modulo d; smaller means already below bound. Start dp[0][0]=1. Append digit and update (sum+digit)%d and smaller. The next layer starts at zero.
- For other digit restrictions, change the state/transition and accepted final states. Add started if leading zeros must not count as real digits (e.g. adjacency restrictions). Range [L,R] is F(R)-F(L-1). $O(10d|bound|)$ time, $O(d)$ space; does not list numbers.

Usage: `int count=digitSumCount("20",3); // 7, including 0`

```
int digitSumCount(const string& bound, int d, int mod =
1000000007) {
  assert(!bound.empty() && d > 0 && mod > 0);
  vec<array<int,2>> dp(d); dp[0][0] = 1%mod;
  for (char ch : bound) {
    assert('0' <= ch && ch <= '9'); int limit = ch-'0';
    vec<array<int,2>> next(d);
    rep(s,0,d) rep(smaller,0,2) rep(digit,0,10) {
      if (!smaller && digit > limit) continue;
      int ns = int((ll(s)+digit)%d), nt = smaller || digit <
      limit;
      next[ns][nt] = int((next[ns][nt]+ll(dp[s][smaller]))%
      mod);
    }
    dp.swap(next);
  }
  return int((ll(dp[0][0])+dp[0][1])%mod);
}
```

DivideAndConquerDP.h – Faster partition-DP layer with monotone optimal split points.

- partitionLayer(prev,cost,INF) returns cur of size n+1, where cur[j]=min over 0<=k<j of prev[k]+cost(k,j). cost(k,j) is the cost of nonempty interval $[k, j)$; cur[0]=INF (one more nonempty group). Initialise prev[0]=0, other entries INF; call once per group.
- REQUIRES the leftmost optimal split not to decrease as j grows. Prove this for the cost (often via a Monge inequality); not valid for arbitrary DP. rec computes the midpoint, then restricts candidate splits for each half. Unreachable prev entries are skipped.
- INF is a positive sentinel; every finite value/intermediate must lie strictly between -INF and INF. Cost evaluation must be O(1) for $O(n \log n)$ per layer; $O(n)$ space. No reconstruction.

Usage: `vec<ll> prev(n+1,INFLL); prev[0]=0; // Squared sums: monotone split points when array values are nonnegative.`

```
auto cur=partitionLayer(prev,[&](int l,int r) {
ll s=prefix[r]-prefix[l]; return s*s;
},INFLL);
```

```
template<class F>
vec<ll> partitionLayer(const vec<ll>& prev, F cost, ll inf =
INFLL) {
  assert(!prev.empty() && inf > 0);
  int n = sz(prev)-1;
  vec<ll> cur(n+1,inf);
  auto rec = [&](auto&& self, int l, int r, int lo, int hi)
  -> void {
    if (l >= r) return;
    int mid = l+(r-l)/2, best = lo;
    rep(k,lo,min(mid,hi)) if (prev[k] != inf) {
      __int128 value = (__int128)prev[k]+cost(k,mid);
      assert(-(__int128)inf < value && value < inf);
      if (value < cur[mid]) { cur[mid] = ll(value); best = k;
      }
    }
    self(self,l,mid,lo,best+1);
    self(self,mid+1,r,best,hi);
  };
  rec(rec,1,n+1,0,n);
  return cur;
}
```

XorBasis.h – Representable XORs of subsets, in 64 bits.

- add(x) inserts an unsigned 64-bit value; returns true iff it increases rank. contains(x) tests whether some subset XOR equals x. maximum(x=0) returns the largest unsigned x XOR subset-XOR. To merge bases, add each nonzero pivot of the other basis.
- pivot[b] has highest set bit b; eliminate highest bits first. rank independent values generate 2^{rank} distinct XORs (do not evaluate 1ULL<<rank when rank=64). Does not reconstruct the subset.
- $O(64)$ per operation, $O(64)$ space; all 64 bits supported.

Usage: `XorBasis b; b.add(3); b.add(5); bool yes=b.contains(6); uint64_t best=b.maximum(); // true,6`

```
struct XorBasis {
  array<uint64_t,64> pivot{};
  int rank = 0;
  bool add(uint64_t x) {
    for (int b = 63; b >= 0; --b) if ((x>>b)&1) {
      if (!pivot[b]) { pivot[b] = x; ++rank; return true; }
      x ^= pivot[b];
    }
    return false;
  }
  bool contains(uint64_t x) const {
    for (int b = 63; b >= 0; --b) if ((x>>b)&1) x ^= pivot[b]
    ;
    return x == 0;
  }
  uint64_t maximum(uint64_t x = 0) const {
    for (int b = 63; b >= 0; --b) x = max(x,x^pivot[b]);
    return x;
  }
};
```

10.3 Intervals, cycles and line queries

CoverIntervals.h – Minimum number of intervals covering a target.

- coverIntervals(seg,L,R) receives intervals $[l, r)$ and target $[L, R)$. Returns original interval IDs in coverage order; nullopt if impossible, empty if L=R. Integers/reals allowed, l<=r and L<=R, no NaN.
- Greedily extend the current covered prefix as far as possible. Input is unchanged; $O(n \log n)$ time, $O(n)$ extra space. Closed integer intervals [a,b] become [a,b+1) if b+1 fits.

Usage: `auto ids=coverIntervals(vec<pair<int,int>>{{0,3},{2,5}},0,5);`

```
template<class T> optional<vec<int>> coverIntervals(
    const vec<pair<T,T>>& seg, T L, T R) {
  if (L == R) return vec<int>{};
  vec<int> ord(sz(seg)), ans;
  iota(ord.begin(),ord.end(),0);
  sort(ord.begin(),ord.end(),[&](int a,int b) {
    return seg[a] < seg[b];
  });
  int i = 0;
  while (L < R) {
    T reach = L;
    int id = -1;
    while (i < sz(ord) &&
        seg[ord[i]].first <= L) {
      int j = ord[i++];
      if (reach < seg[j].second) {
        reach = seg[j].second; id = j;
      }
    }
    if (id < 0) return nullopt;
    ans.pb(id); L = reach;
  }
  return ans;
}
```

Histogram.h – Largest rectangle under a histogram, with variable widths.

- largestRectangle(h,w) receives integer heights h>=0 and widths w>0 of equal length. Returns (area,l,r) covering bars $[l, r)$; empty or all-zero heights give (0,0,0), ties arbitrary.
- x stores n+1 prefix widths; the stack waits for a smaller bar to close each candidate. For unit widths, use w(n,1). After compression, widths are original coordinate differences. Sums/areas must fit ll. $O(n)$ time/space.

Usage: `auto [area,l,r]=largestRectangle( vec<int>{2,1,4,4},vec<int>(4,1));`

```
template<class H, class W>
tuple<ll,int,int> largestRectangle(
    const vec<H>& h, const vec<W>& w) {
  int n = sz(h), left = 0, right = 0;
  vec<ll> x(n+1);
  rep(i,0,n)
    x[i+1] = x[i]+ll(w[i]);
  vec<int> st;
  ll best = 0;
  for (int r = 0; r <= n; ++r) {
    while (!st.empty() &&
        (r == n || h[st.back()] >= h[r])) {
      int j = st.back(); st.pop_back();
      int l = st.empty() ? 0 : st.back()+1;
      ll area = ll(h[j])*(x[r]-x[l]);
      if (area > best) {
        best = area; left = l; right = r;
      }
    }
    if (r < n) st.pb(r);
  }
  return {best,left,right};
}
```

```
}
```

FunctionalCycle.h – Floyd: cycle entry distance and cycle length.

- A deterministic process eventually repeats: find how many steps before the cycle and how long one lap takes, without a visited array.
- floydCycle(start,step) returns (mu,len); step(state) must return the next state. Slow advances once, fast twice. Default equality is ==; optional same(a,b) must be an equivalence preserved by step; lengths are under same. No states returned.
- O(mu+len) calls, O(1) stored states; count state costs too. The process must become periodic and lengths must fit ll.

Usage: `auto [mu,len] = floydCycle(0, [&](int v){ return next[v]; });`

```
template<class T, class F,
  class Eq = equal_to<T>>
pair<ll,ll> floydCycle(T start, F step, Eq same = {}) {
  // Meet inside the cycle.
  T slow = step(start), fast = step(step(start));
  while (!same(slow,fast)) {
    slow = step(slow);
    fast = step(step(fast));
  }
  // Restart one pointer to find the cycle entry.
  ll mu = 0, len = 1;
  slow = start;
  while (!same(slow,fast)) {
    slow = step(slow);
    fast = step(fast);
    ++mu;
  }
  // Walk once around the cycle.
  fast = step(slow);
  while (!same(slow,fast)) {
    fast = step(fast);
    ++len;
  }
  return {mu,len};
}
```

LineHull.h – Dynamic convex hull trick: maximum over inserted lines.

- DP asking for max(k[j]*x+m[j]) over earlier choices. Call add(k,m) to insert a line, query(x) to get the maximum y. ll inputs/result; slopes may arrive in any order, duplicates allowed.
- query needs at least one line. Internally lines are ordered by slope; p is the last integer x where that line wins. updateEnd updates it and flags a redundant next line. Breakpoints and arithmetic use ___int128; returned y must fit ll.
- Add amortized O(log n), query O(log n), O(n) space; no removals.

Usage: `LineHull h; h.add(2,1); h.add(-1,3); ll best = h.query(4); // 9`

```
struct HullLine {
  ll k, m; mutable __int128 p;
  bool operator<(const HullLine& o) const {
    return k < o.k;
  }
  bool operator<(ll x) const {return p < x;}
};

struct LineHull : multiset<HullLine,less<>> {
  static constexpr __int128 inf = (__int128)1 << 126;
  static __int128 floor(__int128 a, __int128 b) {
    return a/b - ((a%b != 0) && ((a<0) != (b<0)));
  }
  bool updateEnd(iterator x, iterator y) {
    if (y == end()) {
      x->p = inf; return false;
    }
    if (x->k == y->k)
      x->p = x->m > y->m ? inf : -inf;
    else x->p = floor((__int128)y->m-x->m,(__int128)x->k-y->k
    );
    return x->p >= y->p;
  }
  void add(ll k, ll m) {
    auto y = insert({k,m,0});
    auto z = next(y), x = y;
    // Remove redundant lines on the right.
    while (updateEnd(y,z)) z = erase(z);
    if (x != begin()) {
      --x;
      if (updateEnd(x,y)) {
        y = erase(y); // New line is redundant.
        updateEnd(x,y);
      }
    }
    while (x != begin()) { // Then prune the left.
      y = x; --x;
      if (x->p < y->p) break;
      y = erase(y);
      updateEnd(x,y);
    }
  }
  ll query(ll x) const {
    assert(!empty());
    auto l = *lower_bound(x);
    __int128 ans = (__int128)l.k*x+l.m;
    assert(LLONG_MIN <= ans && ans <= LLONG_MAX);
    return ll(ans);
  }
};
```

- For minimum line queries negate coefficients and the result; all negations must fit ll (beware LLONG_MIN).

GregorianDays.h – Gregorian date to signed day number.

- gregorianDays(y,m,d) returns days since 1970-01-01 (zero). Input is a valid date; months/days start at 1, unlike array indices. Supports year 0 and negative years; $-10^{12} \le y \le 10^{12}$.
- Subtract results for elapsed days; add 1 for inclusive counting. Proleptic Gregorian calendar: ignores historical calendar changes. Weekday Monday=0: ((days+3)%7+7)%7. $O(1)$ time/space.

Usage: `ll days=gregorianDays(2024,3,1)-gregorianDays(2024,2,28); // 2`

`"../math/IntDivision.h"`

```
ll gregorianDays(ll y, int m, int d) {
  y -= m <= 2;
  ll era = floorDiv(y,400LL), z = y-era*400;
  int day = (153*(m+(m>2 ? -3 : 9))+2)/5+d-1;
  ll off = z*365+z/4-z/100+day;
  return era*146097+off-719468;
}
```

10.4 Operações com bits

```
// x unsigned; 0 <= i,n < 32
x << i                  // x * 2^i (mod 2^32)
x >> i                  // floor(x / 2^i)

x & (1U << i)           // testar bit i
x | (1U << i)           // ligar bit i
x & ~(1U << i)          // desligar bit i
x ^ (1U << i)           // inverter bit i

(x >> i) & 1U           // valor do bit i
(1U << n) - 1           // n bits ligados

x & (x - 1)             // remover último bit 1
x & -x                  // obter último bit 1
x && !(x & (x - 1))     // potência de 2

__builtin_popcount(x) // número de bits 1
__builtin_ctz(x)      // zeros à direita
__builtin_clz(x)      // zeros à esquerda

mask ^ ((1U << n)-1)  // inverter n bits baixos
```

- ctz/clz exigem $x \neq 0$. Para 64 bits usar unsigned long long, 1ULL e builtins terminados em ll; índices de deslocamento < 64. Todos os 32/64 bits ligados: `~0U` / `~0ULL`.
- Para contar zeros com segurança quando x=0, usar `x ? __builtin_ctzll(x) : 64` ou `x ? __builtin_clzll(x) : 64` para unsigned long long.
- XOR: $a \oplus a = 0$, $a \oplus 0 = a$; associativo/comutativo. $a \oplus b = a + b - 2(a \,\&\, b)$; alargar o tipo da aritmética normal.
- Submáscaras: começar s=m, processar s, parar em 0; caso contrário `s=(s-1)&m`. Todas as máscaras/submáscaras: $O(3^k)$; SOS: $O(k2^k)$.

10.5 Guia de complexidade

Tamanho do input	Complexidade típica
$n \le 10$	$O(n!)$
$n \le 20$	$O(2^n)$
$n \le 500$	$O(n^3)$
$n \le 5000$	$O(n^2)$
$n \le 10^6$	$O(n \log n)$ ou $O(n)$
n grande	$O(\log n)$ ou $O(1)$

- Limites aproximados: verificar tempo disponível, custo por transição e total dos testes. $500^3 = 1.25 \cdot 10^8$ operações podem ser demasiado lentas.
- Verificar também memória: ll usa 8 bytes; um ll $dp[mask][v]$ usa cerca de $8n2^n$ bytes, mais containers e dados de reconstrução.